RKNN3 SDK 开发指南

文件标识：RK-JC-YF-416

发布版本：V1.2.0

日期：2026-10-22

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档面向 RKNN3 SDK 的使用者，系统介绍 SDK 的模型转换、量化、评估与板端部署方法，支持协处理器模式（RK1820 / RK1828）和非协处理器模式（RK3572）。文档按基础、转换与评估、板端部署、高级功能、诊断优化五个部分组织。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V1.0.0	HPC / NN	2026-01-01	1. 初版（RK182X）	熊伟
V1.0.4	HPC / NN	2026-05-12	1. 增加RK3572支持 2. 增加LLM推理暂停与恢复 3. 增加KV Cache导入导出 4. 支持LoRA 5. 支持模型加密部署	熊伟
V1.1.0	HPC / NN	2026-08-08	1. 文档结构重构 2. 更新rkllm3-server用法 3. 量化说明与API一致性修正	熊伟
V1.2.0	HPC / NN	2026-10-22	1. 增加Paged Attention使用说明 2. 增加MatMul C API使用说明 3. 增加RK182X多卡LLM部署说明 4. 增加MoE模型转换说明 5. 增加LLM结构化输出（Grammar）说明	熊伟

目录

RKNN3 SDK 开发指南

1. 概述与开发准备

1.1 RKNN3 SDK概述

1.1.1 SDK定位和典型应用场景

1.1.2 支持的硬件平台和部署模式

1.1.3 软件栈与组件职责

1.1.4 Toolkit / Runtime / Toolkit Lite / Server的关系

1.1.5 术语、缩略语和文档约定

1.2 SDK组件与环境约束

1.2.1 版本兼容矩阵

1.2.2 主机端系统要求

1.2.3 板端环境要求

1.2.4 环境验证

1.3 开发流程与接口选择

1.3.1 通用模型开发流程

1.3.2 LLM开发流程

1.3.3 MLLM开发流程

1.3.4 接口选择

1.3.5 模型转换和部署产物说明

2. 模型转换、量化与评估

2.1 通用模型转换

2.1.1 模型准备

2.1.2 转换配置

2.1.3 模型加载和构建

2.1.4 RKNN模型导出

2.1.5 转换结果验证

2.2 LLM和MLLM模型转换

2.2.1 Hugging Face模型准备

2.2.2 ONNX、配置、Tokenizer和Embedding导出

2.2.3 LLM和MLLM模型转换

2.2.3.1 LLM 模型转换

2.2.3.2 MLLM 模型转换

2.2.4 模型和Tokenizer适配

2.2.5 转换结果验证

2.2.6 模型分段导出
2.2.7 MoE模型转换
2.3 模型量化
2.3.1 量化基本概念
2.3.1.1 量化定义
2.3.1.2 量化计算
2.3.1.3 量化粒度
2.3.1.4 量化算法
2.3.2 Toolkit量化流程概览
2.3.3 内部量化流程
2.3.3.1 数据类型
2.3.3.2 内部量化算法
2.3.3.3 校准数据准备
2.3.3.4 具体示例
2.3.4 RKQuantizer量化流程
2.3.4.1 数据类型
2.3.4.2 RKQuantizer量化算法
2.3.4.3 校准数据准备
2.3.4.4 混合量化设置
2.3.4.5 具体示例
2.3.5 其他量化模型转换
2.4 模型评估
2.4.1 模型推理验证
2.4.2 精度评估
2.4.3 性能评估
2.4.4 内存评估
3. 板端部署
3.1 Runtime部署基础
3.1.1 模型和上下文生命周期
3.1.2 张量和数据排列格式
3.1.3 输入输出内存
3.1.4 同步与异步推理
3.1.5 错误处理和资源释放
3.2 通用模型C API部署
3.2.1 完整调用流程
3.2.2 模型初始化

3.2.3 输入输出准备
3.2.4 推理执行
3.2.5 输出处理
3.2.6 完整示例
3.2.7 调试方法
3.3 LLM和MLLM Session部署
3.3.1 完整调用流程
3.3.2 Session生命周期
3.3.3 输入和回调
3.3.4 推理执行与中断
3.3.5 Chat Template
3.3.6 多模态输入
3.3.7 完整示例
3.4 Toolkit Lite Python部署
3.4.1 Python运行环境
3.4.2 RKNN3Lite对象初始化与释放
3.4.3 加载和初始化模型
3.4.4 张量和内存管理
3.4.5 执行推理
3.4.6 完整示例
3.5 RKLLM3 Server集成
3.5.1 Server适用场景
3.5.2 服务部署概览
3.5.3 客户端调用方式
3.5.4 与原生C/Python接口的选择
3.5.5 Server文档入口
3.6 RK182X多卡LLM部署
3.6.1 多卡拓扑和运行边界
3.6.2 运行参数与资源约束
3.6.3 每个stage的Runtime初始化
3.6.4 Host侧数据转发和推理控制
3.6.5 外置输入和回调
3.6.6 命令行部署示例
3.7 MatMul C API使用
3.7.1 接口与运行环境
3.7.2 数据布局、内存与调用流程

3.7.3 W16A16核心调用示例
3.7.4 量化参数准备
3.7.5 共享固定B的多shape使用
3.7.6 返回值、资源生命周期与文档入口
4. 高级功能与扩展
4.1 Runtime高级功能
4.1.1 NPU多核
4.1.2 动态Shape
4.1.3 Cacheable内存一致性
4.1.4 Internal内存复用
4.2 LLM和MLLM高级功能
4.2.1 Prompt、Token和Embedding输入
4.2.2 Multimodal和Aux输入
4.2.3 Callback机制
4.2.4 Function Calling
4.2.5 KVCache管理
4.2.6 Paged Attention
4.2.7 KVCache Checkpoint
4.2.8 LoRA
4.2.9 Tie Word Embedding
4.2.10 暂停、恢复和状态查询
4.2.11 结构化输出（Grammar）
4.3 扩展与安全
4.3.1 自定义算子
4.3.1.1 后处理算子
4.3.1.1.1 插件实现步骤
4.3.1.1.2 插件编译和加载使用
4.3.1.1.3 示例
4.3.1.2 自定义 CPU 算子
4.3.1.2.1 插件实现步骤
4.3.1.2.2 插件编译和加载使用
4.3.1.2.3 示例
4.3.2 模型加密部署
5. 问题诊断与优化
5.1 精度问题诊断

5.1.1 内部量化精度诊断

5.1.2 RKQuantizer 量化精度诊断

5.2 性能分析与优化

5.3 内存分析与优化

5.4 常见问题与错误处理

1. 概述与开发准备

1.1 RKNN3 SDK概述

1.1.1 SDK定位和典型应用场景

RKNN3 SDK 是瑞芯微面向 RK1820 / RK1828 协处理器与 RK3572 SoC 的 AI 模型部署开发套件，提供从模型转换、量化评估到板端部署的完整工具链，支持 CNN / LLM / MLLM 等多种模型类型在 NPU 上的高效推理。

典型应用场景包括：

- **视觉识别与检测**：在 RK182X 或 RK3572 上部署 MobileNet / YOLO 等 CNN 模型，完成图像分类、目标检测等任务。
- **大语言模型推理**：部署 Qwen 等大语言模型，支持多轮对话、流式生成、KVCache 加速、LoRA 微调适配等。
- **多模态理解**：部署 Qwen2.5-VL 等视觉语言模型，支持图文理解、视频理解等。
- **OpenAI 兼容服务**：通过 rkllm3-server 提供 OpenAI 兼容 API，与现有应用生态对接。

SDK 支持两种部署模式：

- **协处理器模式（RK182X）**：主控 SoC（如 RK3588 / RK3576）通过 PCIe / USB / Ethernet 与 RK1820 / RK1828 协处理器互联，应用运行在主控 SoC 上，通过 RKNN3 API 控制协处理器执行 AI 推理。
- **非协处理器模式（RK3572）**：RK3572 SoC 集成高性能 NPU，应用直接运行在 RK3572 上，通过 RKNN3 API 控制本地 NPU 执行 AI 推理。

1.1.2 支持的硬件平台和部署模式

本文档适用如下硬件平台：

- **RK1820 / RK1828（协处理器模式）**：作为 AI 计算加速单元，由主控 SoC 调度，通过 PCIe / USB / Ethernet 接口参与计算。
- **RK3572（非协处理器模式）**：集成高性能 NPU 的 SoC 平台，NPU 与 CPU / GPU 共享内存空间。

两种模式的主要区别：

对比项	协处理器模式（RK182X）	非协处理器模式（RK3572）
主控 SoC	RK3588 / RK3576 等	无（RK3572 自身即主控）
NPU	RK1820 / RK1828 协处理器	RK3572 集成 NPU
通信方式	PCIe / USB / Ethernet	统一内存架构
通信服务	rknn3_transfer_proxy	rknn3-server
支持系统	Android / Linux / Windows	Android / Linux（仅 aarch64）

1.1.3 软件栈与组件职责

RKNN3 软件栈整体架构分为 PC 侧、设备侧 API 与应用服务、Runtime 与 NPU 驱动三层。

协处理器模式架构

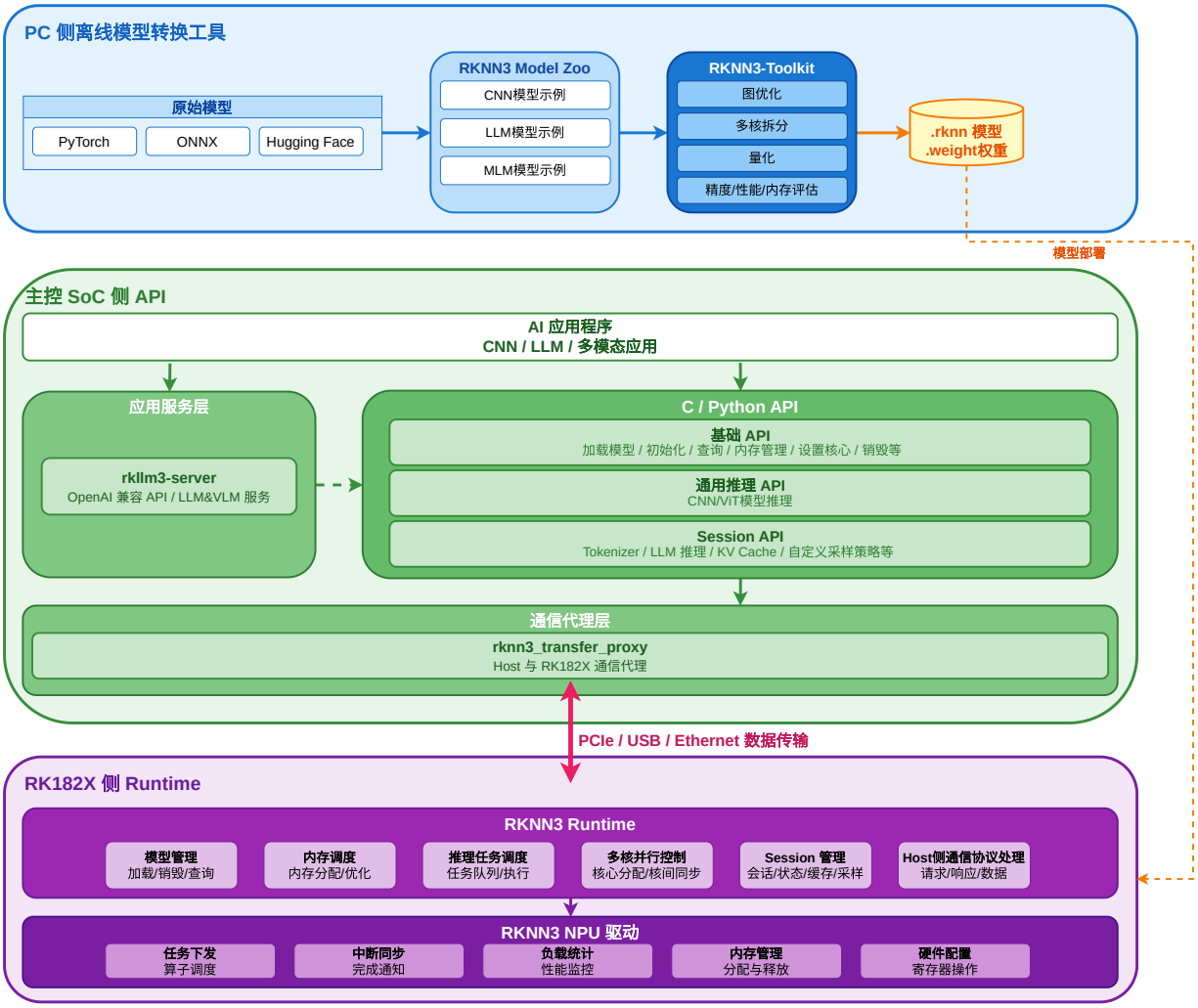


图1-1 协处理器模式架构

非协处理器模式架构

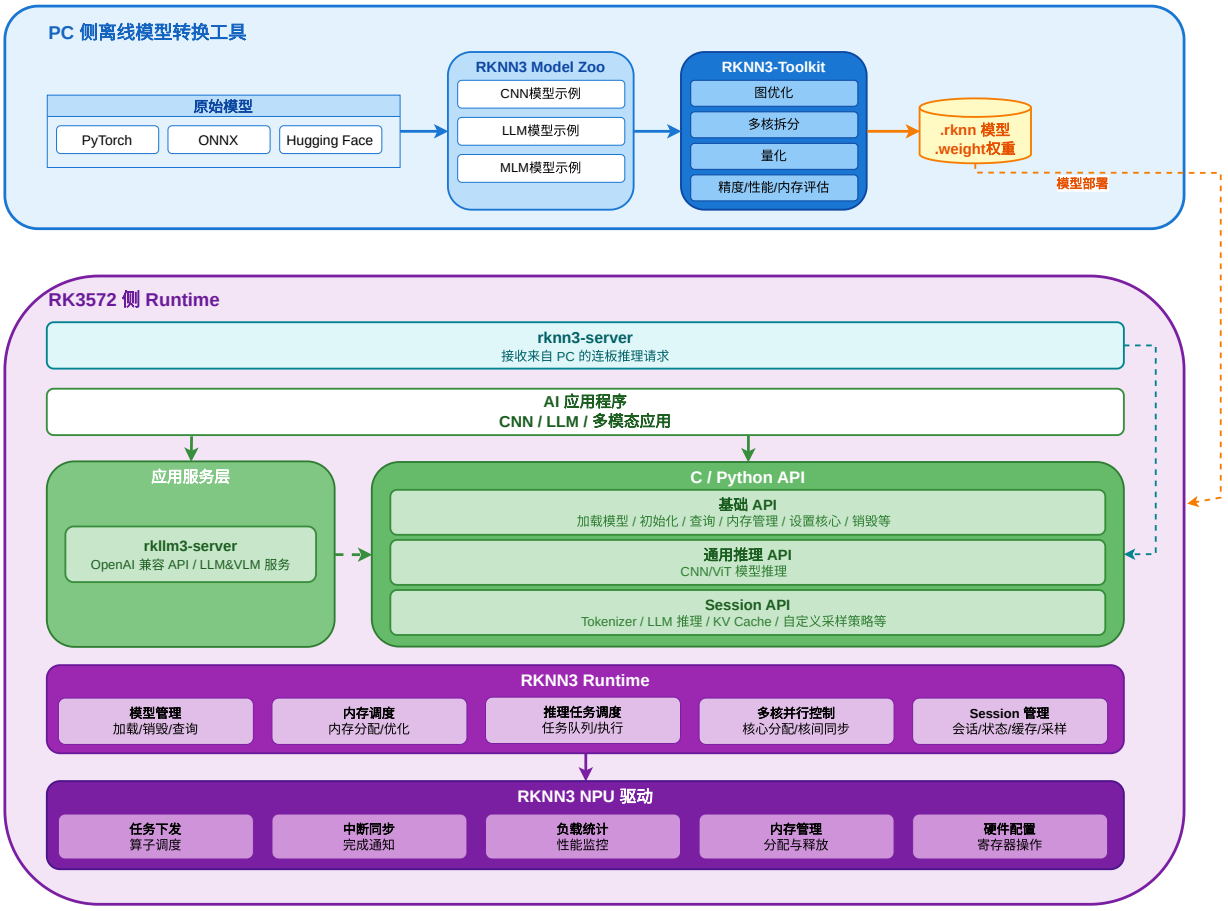


图1-2 非协处理器模式架构

各组件职责如下：

1. PC 侧

- **RKNN3 Model Zoo**：提供模型转换与部署示例，涵盖 CNN（MobileNetV2 / ResNet50 / YOLOv5/v6/v8 等）、LLM（Qwen2.5 / Qwen3 等）、MLLM（FastVLM / Qwen2.5-VL / Qwen3-VL 等）多种模型类型。
- **RKNN3-Toolkit**：模型转换核心工具，将 ONNX 模型转换为 `.rknn` 和 `.weight` 格式，完成图优化、多核拆分、量化、精度分析与可视化分析等功能。对于 LLM / MLLM 模型，需先通过 Model Zoo 的导出脚本将模型转换为 ONNX，再使用 Toolkit 转换为 RKNN。

2. 设备侧 API 与应用服务

- **RKNN3 API**：核心接口层，支持 C 和 Python 两种语言，按功能分为：
- **基础 API**：模型加载、初始化、查询、内存管理、设置运行核心、销毁等基础操作。
- **通用推理 API**：面向 CNN / ViT 等通用视觉模型的推理接口。
- **Session API**：面向 LLM 的推理接口，支持 KVCache、自定义采样策略等功能。
- **rkllm3-server**：提供 OpenAI 兼容 API 服务，支持文本、图片和语音输入，简化大模型集成与调用。

3. 通信服务

- **协处理器模式**：使用 **rknn3_transfer_proxy**，提供主控 SoC 与 RK1820 / RK1828 协处理器之间的通信代理，支持 PCIe / USB / Ethernet三种连接方式。
- **非协处理器模式**：使用 **rknn3-server**，运行在 RK3572 侧，处理来自 PC 端的连板推理请求，管理 NPU 资源。

4. Runtime 与 NPU 驱动

- **RKNN3 Runtime**：负责模型管理、内存调度、推理任务调度、多核并行控制以及 Session 会话管理。
- **RKNN3 NPU 驱动**：负责任务下发、中断同步、负载统计等底层操作。

在协处理器模式中，应用运行在主控 SoC 上，通过 RKNN3 API 与 rknn3_transfer_proxy 控制 RK1820 / RK1828 协处理器上的模型加载与推理执行；在非协处理器模式中，应用直接运行在 RK3572 上，通过 RKNN3 API 控制本地 NPU。用户可以选择直接使用 RKNN3 API 进行模型推理，或通过 rkllm3-server 使用 OpenAI 兼容接口调用大模型服务。

1.1.4 Toolkit / Runtime / Toolkit Lite / Server的关系

RKNN3 SDK 提供四类核心组件，分别对应模型生命周期的不同阶段，用户根据使用场景选择：

组件	运行位置	主要职责	典型场景
RKNN3-Toolkit	PC	模型转换、量化、评估、连板调试	模型转换与评估阶段
RKNN3 Runtime	板端（RK182X / RK3572）	模型执行、内存管理、多核调度	板端推理（C API）
RKNN3-Toolkit Lite	板端	Python 推理接口	板端 Python 快速验证
rkllm3-server	板端	OpenAI 兼容 API 服务	大模型服务化部署

各组件的关系与数据流向：

1. **模型转换阶段**：用户在 PC 上使用 RKNN3-Toolkit 将 ONNX 模型转换为 `.rknn` 和 `.weight` 文件，期间可进行量化、性能评估、精度分析。
2. **板端部署阶段**：将转换产物部署到板端，通过 RKNN3 Runtime（C API）或 RKNN3-Toolkit Lite（Python）加载并运行模型；对于 LLM / MLLM，也可通过 rkllm3-server 提供 OpenAI 兼容服务。
3. **连板调试阶段**：RKNN3-Toolkit 支持连板推理，在 PC 侧发起推理、模型在板端 NPU 执行，用于模型验证和精度分析。

> **说明**：RKNN3-Toolkit 与 Runtime / Toolkit Lite 的版本需保持一致，否则可能出现模型转换正常但板端无法运行的问题，详见 1.2.1 版本兼容矩阵。

1.1.5 术语、缩略语和文档约定

本节介绍文档中常用的术语与缩略语。

硬件与部署

- **主控 SoC**：运行用户应用的主处理器（如 RK3588 / RK3576），仅在协处理器模式下涉及。
- **协处理器**：由主控 SoC 调度、通过 PCIe / USB / Ethernet 接口参与计算的专用加速单元（如 RK1820 / RK1828），仅在协处理器模式下涉及。
- **RK182X**：RK1820 与 RK1828 协处理器的统称，两者架构相同，文档中以 RK182X 泛指。

- **连板推理**：PC 经过主控 SoC 或通过 USB / 网络（adb connect）连接开发板，推理由 PC 侧 RKNN3-Toolkit 发起、模型在 NPU 上执行，用于模型验证和精度分析。

模型与张量

- **RKNN 模型**：运行在 NPU 上的模型文件，包括 `.rknn`（模型结构）和 `.weight`（权重数据）两个文件。
- **LLM（大语言模型）**：Large Language Model，基于 Transformer 架构的大型语言模型，如 Qwen2.5 / Qwen3 等，用于文本生成、对话等任务。
- **MLLM（多模态大语言模型）**：Multimodal Large Language Model，在 LLM 基础上扩展支持图像、音频、视频等多种模态输入的大模型，如 Qwen2.5-VL / Qwen3-VL 等。
- **Tensor（张量）**：表示多维数组的数据结构，包含数据内存和属性信息（shape / dtype / layout 等）。
- **数据类型（dtype）**：张量元素的数据类型，如 FP32 / FP16 / INT8 / UINT8 等。
- **数据布局（layout）**：张量在内存中的排列方式，如 NCHW / NHWC / NC1HWC2 等。
- **NC1HWC2**：NPU 优化的内存布局格式，将通道维度分块存储（FP16 为 16 子通道，INT8 为 32 子通道），提升 NPU 访问效率。

量化

- **量化模型**：使用 INT8 等低精度数据类型的模型，需要 scale 和 zero_point 参数进行量化/反量化。
- **非量化模型**：使用 FP16 数据类型的模型，无需量化参数。

API 与功能

- **基础 API**：面向 CNN / ViT 等通用视觉模型的推理接口。
- **Session API**：面向 LLM 大语言模型的专用接口，提供会话管理、KVCache / LoRA、采样控制等功能。
- **核心掩码（core_mask）**：用于指定模型运行在哪些 NPU 核心上，如 0x3 表示使用 core 0 和 core 1。
- **动态 Shape**：同一模型支持多组输入尺寸组合，运行时可通过 `rknn3_set_shape()` 切换。
- **KVCache**：大语言模型中缓存的 key-value 注意力状态，用于加速自回归生成。
- **LoRA**：低秩适配（Low-Rank Adaptation），一种高效的模型微调方法。

文档约定

- 代码块中以 `#` 开头的行为注释。
- `<>` 中的内容为参数占位符，需根据实际情况替换。
- 接口函数名、参数名、文件名等用反引号（`）包裹。
- 文档中涉及的 API 详细参数说明请参考对应的 API 参考手册。

1.2 SDK组件与环境约束

> **说明**：本章介绍 SDK 各组件的版本兼容性、功能职责与环境约束，具体的 Toolkit 安装、硬件连接、板端环境部署等操作步骤请参考《RKNN3_SDK_快速入门》。

1.2.1 版本兼容矩阵

RKNN3 SDK 的各组件（RKNN3-Toolkit、Runtime、通信服务、rkllm3-server）版本需保持一致，否则可能出现模型转换正常但板端无法运行、推理结果异常等问题。

组件概览与版本一致性

组件	运行位置	功能描述	必须一致的版本
RKNN3-Toolkit	PC	模型转换、量化、评估	与 Runtime、通信服务同版本
RKNN3 Runtime（librknn3_api.so）	板端	模型执行、内存管理、多核调度	与 Toolkit 同版本
rknn3_transfer_proxy	板端	主控 SoC 与 RK1820 / RK1828 协处理器之间的通信代理（协处理器模式）	与 Toolkit / Runtime 同版本
rknn3-server	板端	处理来自 PC 的连板推理请求，管理 RK3572 NPU 资源（非协处理器模式）	与 Toolkit / Runtime 同版本
rkllm3-server	板端	OpenAI 兼容 API 服务	与 Toolkit / Runtime 同版本

需要检查版本一致性的场景

- 切换到新的开发机器或 Docker 环境时
- 更新或重新安装 RKNN3-Toolkit 后（必须同步更新通信服务与 Runtime）
- 从他人共享的项目/SDK 拉取代码或模型时
- 出现运行错误或模型加载失败时

> **说明**：RKNN 模型文件中记录了转换时使用的 Toolkit 版本。若与板端 Runtime 版本不一致，会有运行错误的风险，建议用与 Runtime 同版本的 Toolkit 重新转换模型。版本查询方法见 1.2.4 环境验证。

1.2.2 主机端系统要求

RKNN3-Toolkit 运行在 PC 端，用于模型转换、量化与评估。主机端环境需满足以下要求：

项目	要求
操作系统	Ubuntu 20.04 / Ubuntu 22.04 / macOS（Beta）
Python	3.10 / 3.12（V1.1.0 起仅支持 cp310 / cp312）
架构	x86_64（Ubuntu）/ ARM64（macOS Apple Silicon）
核心依赖	onnx>=1.16 / onnxruntime>=1.22.0 / torch>=2.2 / numpy>=2.0

> **说明**：完整依赖列表以 Toolkit 对应的 requirements 文件为准，上述仅列出需重点关注的几个核心依赖。Python 支持面在 V1.1.0 收敛为 3.10 / 3.12，旧版本的 cp36/cp37/cp38/cp39/cp311 requirements 文件不再提供。若依赖库版本冲突导致安装失败，可尝试更换 pip 源（如 `https://pypi.tuna.tsinghua.edu.cn/simple/`）或使用 `pip install <package> --no-deps` 跳过依赖检查，但需自行确保运行依赖完整。GRQ 量化（RKQuantizer）建议在 CUDA 或 MPS 环境下运行，CPU 下耗时会很长。

1.2.3 板端环境要求

板端环境根据部署模式有所不同：

协处理器模式（RK182X）

项目	要求
主控 SoC	RK3588 / RK3576 等
协处理器	RK1820 / RK1828
主控系统	Android / Linux / Windows
通信服务	rknn3_transfer_proxy
Runtime	librknn3_api.so + librknn3_api_rkcp.so
通信接口	PCIe / USB / Ethernet
NPU 核心数	8

非协处理器模式（RK3572）

项目	要求
SoC	RK3572
系统	Android / Linux
通信服务	rknn3-server
Runtime	librknn3_api.so + librknn3_api_native.so
NPU 核心数	1（单核）

> **说明**：RK3572 仅支持 aarch64 架构的 Linux，不支持 x86_64；RK182X 协处理器模式的 Runtime 库与主控 SoC 架构相关，需选择与主控匹配的库文件。

1.2.4 环境验证

环境部署完成后，建议按以下方法验证各组件是否就绪：

1. 验证 Toolkit 安装

```
# 进入 Python 交互模式，导入 RKNN 类
python
>>> from rknn.api import RKNN
# 无报错即表示安装成功
```

2. 验证板端通信服务

- 协处理器模式：在主控 SoC 终端执行 `rknn3_transfer_proxy devices`，能列出协处理器设备即正常。
- 非协处理器模式：在 RK3572 终端执行 `rknn3-server -v`，能输出版本号即正常。

3. 验证设备连接

```
# 在 PC 端查询 adb 连接的设备
adb devices
# 能列出设备 ID 即连接正常
```

4. 验证版本一致性

确认 Toolkit、Runtime、通信服务版本一致，版本查询方法如下：

```
# 查询 rknn3-toolkit 版本
pip list | grep rknn3-toolkit

# 查询 rknn 模型转换时使用的 rknn3-toolkit 版本
strings xxxx.rknn | grep rknn3-toolkit

# 查询 Runtime 库版本
strings /usr/lib/librknn3_api.so | grep -i "librknn3_api version"

# 协处理器模式（RK182X）：查询 rknn3_transfer_proxy 版本
strings /usr/bin/rknn3_transfer_proxy | grep "Transfer version"

# 非协处理器模式（RK3572）：查询 rknn3-server 版本
/usr/bin/rknn3-server -v
```

> **Android 平台路径**：Android 平台的库与可执行文件路径与 Linux 不同，查询方法相同但路径需替换： - Runtime 库：`/system/lib64/librknn3_api.so` 或 `/vendor/lib64/librknn3_api.so` - rknn3-server：`/system/bin/rknn3-server` 或 `/vendor/bin/rknn3-server` - rknn3_transfer_proxy：`/system/bin/rknn3_transfer_proxy` 或 `/vendor/bin/rknn3_transfer_proxy`

1.3 开发流程与接口选择

1.3.1 通用模型开发流程

CNN / ViT 等视觉模型的典型开发流程分为模型转换、模型评估、板端部署运行三个阶段，整体流程如下图：



图1-3 通用模型开发流程

1. **模型转换**

在 PC 侧使用 RKNN3-Toolkit 完成：

a. 准备原始模型（ONNX格式）。 b. 在RKNN3-Toolkit中配置归一化、量化与目标平台等参数。 c. 通过 `rknn.build()` 完成模型构建，可开启量化以获得更高性能与更低内存占用。 d. 通过 `rknn.export_rknn()` 导出 `.rknn` 和 `.weight` 模型文件。

1. **模型评估**

在 PC 侧通过 USB 或网络（adb connect）连接开发板，使用 RKNN3-Toolkit 的 Python API 进行连板评估：

a. 通过 `rknn.init_runtime()` 连板初始化运行时环境，使用 `rknn.inference()` 进行连板推理，验证功能正确性、完善前后处理逻辑。 b. 精度分析：通过 `rknn.inference(accuracy_analysis=True)` 在 simulator 或连板上分析量化误差及板端推理误差的来源。 c. 性能分析：通过 `rknn.eval_perf()` 评估算子耗时，优化运行核心调度。

1. **板端部署运行**

在板端应用中使用 RKNN3 Runtime 的 C API 集成推理：

a. 通过 `rknn3_init()` 初始化上下文，通过 `rknn3_load_model_from_path()` 加载 `.rknn` 和 `.weight` 文件。 b. 通过 `rknn3_model_init()` 完成模型初始化配置（含多核掩码 `run_core_mask` 等），通过 `rknn3_query()` 查询输入输出属性。 c. 通过 `rknn3_create_mem()` 分配输入输出内存，通过 `rknn3_mem_sync()` 同步数据，调用 `rknn3_run()` 执行推理。 d. 实现输入前处理（数据类型转换、布局转换）和输出后处理（反量化、业务逻辑），集成到实际业务流程。

> **说明**：模型转换、量化、评估的具体操作见第 2 章（2.1-2.4 节），板端部署见第 3 章（3.1-3.5 节）。

1.3.2 LLM开发流程

LLM模型的开发流程与CNN模型存在一些差异，主要体现在对HuggingFace模型的特殊处理，以及LLM模型在板端部署时所使用的Session API与CNN模型的不同。



图1-4 LLM模型开发流程

LLM模型开发流程分为三阶段：

1. **模型转换**

在 PC 侧使用 RKNN3-Toolkit 完成：

a. 将HuggingFace模型导出为ONNX，可选用RKQuantizer进行GRQ量化。 b. 通过 `rknn.load_llm()` 加载LLM模型。 c. 通过 `rknn.build()` 构建模型，可开启量化以获得更高性能与更低内存占用。 d. 通过 `rknn.export_rknn()` 导出 `.rknn` / `.weight` / `.embed.bin` / `.tokenizer.gguf` 等部署产物。

1. **模型评估**

在 PC 侧通过 USB 或网络（adb connect）连接开发板，使用 RKNN3-Toolkit 的 Python API 进行连板评估：

a. 通过 `rknn.init_runtime()` 连板初始化运行时环境，使用 `rknn.inference()` 进行连板推理，验证LLM模型功能与生成质量。 b. 验证多轮对话、流式生成等功能是否正常。

1. **板端部署运行**

在板端使用 RKNN3 Runtime 的 C API（Session 接口）或 rkllm3-server 集成推理：

a. 通过 `rknn3_init()` / `rknn3_load_model_from_path()` 加载模型，通过 `rknn3_model_init()` 完成初始化。 b. 通过 `rknn3_session_init()` 创建会话，配置Chat Template、回调、采样参数。 c. 通过 `rknn3_session_run()` 进行会话推理，支持KVCache / LoRA、多模态输入等。

> **说明**：LLM模型转换的详细要求（ONNX输入约定、KVCache、多种Shape等）见 2.2 LLM和MLLM模型转换；Session部署见 3.3 LLM和MLLM Session部署。

1.3.3 MLLM开发流程

MLLM（Multimodal Large Language Model，多模态大语言模型）在 LLM 基础上扩展了视觉、音频等非文本输入能力，其中 VLM（视觉语言模型）是最常见的形态。

MLLM 由多模态编码部分（如 Vision / Audio）与 LLM 部分组成，整体开发流程分为模型转换、模型评估、板端部署运行三阶段，每阶段都需分别处理多模态编码部分和 LLM 部分：

多模态编码部分开发流程

多模态编码部分本质是通用模型（如 Vision 模型），其开发流程参考 1.3.1 通用模型开发流程，通过 `rknn.load_onnx()` 加载、`rknn.build()` 构建、`rknn.export_rknn()` 导出，板端通过通用 C API 运行得到多模态 embedding。

LLM 部分开发流程

LLM 部分的开发流程与 1.3.2 LLM 开发流程一致，差异在于推理时需将多模态编码器输出的 embedding 与文本 prompt 一起作为输入。

> **说明**：MLLM 多模态编码部分转换见 2.2.2 多模态编码部分导出与 2.1 通用模型转换；多模态输入部署见 3.3.6 多模态输入。

1.3.4 接口选择

RKNN3 SDK 提供三种板端调用方式，用户根据场景选择：

方式	语言	适用模型	适用场景
RKNN3 Runtime（C API）	C / C++	通用模型、LLM / MLLM	生产部署、性能敏感场景
RKNN3-Toolkit Lite	Python	通用模型、LLM / MLLM	板端快速验证、原型开发
rkllm3-server	HTTP API	LLM / MLLM	服务化部署、OpenAI 生态对接

三者差异：

- **性能**：C API 最优；rkllm3-server 次之（含网络与序列化开销）；Toolkit Lite 因 Python 封装开销性能略低。
- **开发便捷性**：rkllm3-server 最便捷（零编程，启动服务即可通过 HTTP 调用）；Toolkit Lite 次之（Python 编程，无需交叉编译）；C API 需交叉编译集成。
- **功能完整性**：C API 支持完整高级功能（自定义采样、KVCache 管理、LoRA 动态切换等）；rkllm3-server 支持常用配置；Toolkit Lite 面向验证，高级功能覆盖较少。

选择建议：

- **通用模型**：生产部署用 C API；板端验证用 Toolkit Lite。
- **LLM / MLLM**：需对接 OpenAI 生态或快速服务化用 rkllm3-server；需深度控制推理流程用 C API 的 Session 接口。

1.3.5 模型转换和部署产物说明

模型转换完成后生成的部署产物如下：

通用模型（CNN / ViT）

文件	说明
`.rknn`	RKNN 模型结构文件
`.weight`	RKNN 模型权重文件

LLM / MLLM 模型

文件	说明
`xxx.rknn`	RKNN 模型结构文件
`xxx.weight`	RKNN 模型权重文件
`xxx.embed.bin`	Embedding 层权重，Tied word embeddings 模式下不需要
`xxx.tokenizer.gguf`	Tokenizer 文件，板端部署时用于初始化分词器
`xxx.safetensors`	外置 ROPE 缓存文件（部分模型需要，如 Gemma4）
`xxx_per_layer_inputs.embed.bin`	PLE（per_layer_inputs embed）文件，每层额外输入（部分模型需要，如 Gemma4）

> **说明**：部署时需将上述产物传输到板端，C API 部署需 `.rknn` + `.weight`（LLM 还需 `.embed.bin` + `.tokenizer.gguf`），Server 部署按 3.5 RKLLM3 Server集成 的参数指定对应文件。

2. 模型转换、量化与评估

2.1 通用模型转换

本节介绍 CNN / ViT 等通用模型的转换流程，使用 RKNN3-Toolkit 将 ONNX 模型转换为 RKNN 模型。整体流程为：准备 ONNX 模型 → 配置转换参数 → 加载并构建模型 → 导出 RKNN 模型 → 验证转换结果。



图2-1 通用模型转换流程图

2.1.1 模型准备

通用模型转换以 ONNX 格式为输入。用户需先从原始训练框架（PyTorch / TensorFlow 等）将模型导出为 ONNX：

- **ONNX 导出**：在原始框架侧完成导出，建议 opset 版本 ≥ 11。若原始模型为 PyTorch，可参考 RKNN3 Model Zoo 中导出ONNX模型的脚本。
- **输入约束**：RKNN3-Toolkit 加载 ONNX 时，需明确模型的输入 name / shape / dtype 等信息，用于后续 `config` 配置与 `load_onnx` 加载。
- **模型检查**：导出后建议用 ONNX 工具（如 `onnxruntime` 或 Netron）检查模型结构、算子支持情况，确保导出正确。

> **说明**：RKNN3-Toolkit 当前仅支持 ONNX 模型加载，其他框架（TensorFlow / PyTorch 等）需先导出为 ONNX。

2.1.2 转换配置

模型转换时，`rknn.config()` 和 `rknn.build()` 接口的参数会影响模型转换结果。其中 `rknn.config()` 配置归一化、量化、目标平台等参数，`rknn.build()` 控制是否量化及量化数据集。

确定转换参数的基本流程

可参考以下步骤确定转换参数：

1. 确定 NPU 平台（RK1820 / RK1828 / RK3572），设置 `target_platform`。
2. 确认输入是 RGB 还是 BGR，设置 `quant_img_RGB2BGR`。
3. 确认归一化参数，设置 `mean_values` 和 `std_values`。
4. 确认模型输入尺寸，用于 `load_onnx()` 加载。
5. 确认量化比特数，设置 `quantized_dtype`。
6. 确认量化算法，设置 `quantized_algorithm`。
7. 确认是否量化，设置 `rknn.build()` 的 `do_quantization`，量化时通过 `dataset` 指定校正数据集。

参数细化说明

- **target_platform**：目标 NPU 平台，支持 `rk1820`、`rk1828`、`rk3572`。
- **quant_img_RGB2BGR**：量化图像是否做 RGB2BGR 转换，默认 `False`，仅在量化阶段生效。模型采用 RGB 训练则设为 `False` 或不设置，推理时输入 RGB 图片；模型采用 BGR 训练则设为 `True`，推理时输入 BGR 图片。若量化数据采用 npy 格式，建议不要使用此参数，避免使用混乱。**注意**：此参数只影响量化过程中读取校正集图像时的通道转换，不影响推理时的输入处理，推理时对输入数据只做减均值、除标准差操作，通道转换需用户自行完成。
- **mean_values / std_values**：输入的均值与归一化值，两者格式一致。单输入 N 通道为 `[[channel_1, ..., channel_n]]`，多输入为 `[[输入1各通道], [输入2各通道], ...]`。归一化公式为 `output = (input - mean) / std`，Toolkit 在推理时按此公式对输入做归一化。配置后 C API 推理阶段无需再做均值归一化，减少部署耗时。
- **quantized_dtype**：量化类型，支持 `w4a16` / `w6a16` / `w8a16` / `w16a16` / `w8a8`，默认 `w16a16`。
- **quantized_algorithm**：量化算法，默认 `normal`。`normal` 速度快，推荐校正集 20-100 张；`mmse` 精度更高但速度慢、内存占用大，推荐校正集 20-50 张；`kl_divergence` 耗时介于两者之间，适用于 feature 分布不均匀场景，推荐校正集 20-100 张。建议先用 `normal`，效果不佳再尝试其他算法。
- **quantized_method**：量化方法，支持 `layer` / `channel` / `group{SIZE}`，默认 `None`（`quantized_dtype` 不为 `w16a16` 时必须配置）；`group{SIZE}` 仅在 `w4a16` / `w6a16` / `w8a16` 下有效，且仅支持 MatMul / Linear 类型算子。
- **optimization_level**：模型优化等级，默认 `3`（打开所有优化），值为 2/1 关闭部分优化，0 关闭所有优化。
- **input_attrs**：输入属性，可设置 dtype / layout / shape / stride 等。
- **dynamic_input**：动态输入配置，指定多组输入 shape 模拟动态输入，默认 `None`。

量化校正数据集的格式与填写方式见 2.3 模型量化。更详细的参数说明请参考《RKNN3_Toolkit_Python_API_参考》。

示例：

```
rknn.config(
    mean_values=[[103.94, 116.78, 123.68]],
    std_values=[[58.82, 58.82, 58.82]],
    quant_img_RGB2BGR=False,
    input_attrs={'pixel': {'dtype': 'uint8', 'layout': 'NHWC'}},
    target_platform='rk1820')
```

2.1.3 模型加载和构建

1. 初始化 RKNN 对象

转换流程的第一步是初始化 RKNN 对象，可传入 `verbose` 控制日志输出：

```
from rknn.api import RKNN
rknn = RKNN(verbose=True, verbose_file='./mobilenet_build.log')
```

2. 加载 ONNX 模型

通过 `rknn.load_onnx()` 加载 ONNX 模型，需提供模型文件路径：

```
rknn.load_onnx(model='./arcface.onnx')
```

3. 构建 RKNN 模型

通过 `rknn.build()` 构建模型，可选择是否量化：

- `do_quantization`：是否量化，默认 `True`。
- `dataset`：量化校准数据集，文本文件格式，每行一个校正样本路径。

dataset.txt 示例：

```
./imgs/ILSVRC2012_val_00000665.JPEG
./imgs/ILSVRC2012_val_00001123.JPEG
./imgs/ILSVRC2012_val_00001129.JPEG
```

构建示例：

```
rknn.build(do_quantization=True, dataset='./dataset.txt')
```

> **说明**：若 `do_quantization=False`，则跳过量化，模型以浮点（w16a16）形式构建，无需提供 dataset。

2.1.4 RKNN模型导出

构建完成后，通过 `rknn.export_rknn()` 导出 RKNN 模型文件，用于板端部署。`export_path` 参数指定导出文件路径，Toolkit 会以该路径为基准，生成 `.rknn`（模型结构）和 `.weight`（权重数据）两个文件：

```
rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```

上述示例会在当前目录生成 `mobilenet_v1.rknn` 和 `mobilenet_v1.weight`。转换流程完成后，调用 `rknn.release()` 释放资源：

```
rknn.release()
```

2.1.5 转换结果验证

导出后可通过 `rknn.inference()` 对 RKNN 模型进行推理验证，对比输出与原 ONNX 模型是否一致，确认转换结果正确。

> **说明**：精度分析、性能评估等更详细的评估方法见 2.4 模型评估。

2.2 LLM和MLLM模型转换

本节介绍 LLM 和 MLLM 模型的转换流程。

LLM / MLLM 模型通常以 SafeTensors 格式存储（由 Hugging Face 开发的安全张量格式），通过 HuggingFace 生态加载与管理，因此转换以 HuggingFace 格式模型为输入。由于 RKNN3-Toolkit 无法直接转换 SafeTensors / HuggingFace 模型，需先将其导出为 ONNX 及配套文件，再转换为 RKNN 模型。

与通用模型的差异

- **输入格式**：通用模型以 ONNX 为输入；LLM / MLLM 以 HuggingFace 格式模型（通常为 SafeTensors）为输入，需额外增加导出环节。
- **导出环节**：LLM 需先从 HuggingFace 模型导出 4 类文件（ONNX 模型、config 配置、tokenizer 分词器、embedding 权重）。导出依赖 `transformers`（加载模型）与 `torch`（`torch.onnx.export`）；rknn3-model-zoo 的 `export_llm.py` 对这套导出逻辑做了封装，方便直接使用，用户也可参考其实现自行导出。
- **转换工具**：转换仍使用 RKNN3-Toolkit，但通过 `rknn.load_llm()` 加载、`llm_config` 配置量化/核心/attention 等参数，与通用模型的 `rknn.load_onnx()` / `rknn.config()` 不同。

MLLM 的特殊性

MLLM 由多模态编码部分（如 Vision / Audio）与 LLM 部分组成，转换时两部分分别独立进行：

- **LLM 部分**：导出与转换方法与 LLM 模型一致（见 2.2.1~2.2.3）。
- **多模态编码部分**：本质是通用模型，导出为独立 ONNX 后按 2.1 通用模型转换方法转换（详见 2.2.2 多模态编码部分导出）。
- **联合使用**：两部分在板端推理时联合使用，LLM 接收多模态编码输出的嵌入。

LLM 模型转换的整体流程如下图：



图2-2 LLM模型转换流程

2.2.1 Hugging Face模型准备

LLM / MLLM 模型转换以 HuggingFace 格式模型为输入。用户需先准备 HuggingFace 格式的模型文件（含模型权重、config.json、tokenizer.json 等），可通过 `transformers` 库的 `AutoModelForCausalLM` / `AutoConfig` 加载：

```
from transformers import AutoModelForCausalLM, AutoConfig
config = AutoConfig.from_pretrained(args.model_path, **kwargs)
model = AutoModelForCausalLM.from_pretrained(args.model_path, **kwargs)
```

> **说明**：MLLM 模型由多模态编码部分（如 Vision / Audio）与 LLM 部分组成，需分别准备并分别转换，多模态编码部分导出见 2.2.2、转换见 2.1。

2.2.2 ONNX、配置、Tokenizer和Embedding导出

LLM 模型转换前需从 HuggingFace 模型导出 4 类文件：ONNX 模型、config 配置、tokenizer 分词器、embedding 权重。rknn3-model-zoo 将这些过程封装在 `export_llm.py` 中，对应接口为 `causal_llm_to_onnx`、`export_llm_config`、`export_tokenizer`、`export_embed_weight`。

1. 导出 ONNX 模型

将 LLM Torch 模型转为 ONNX 模型。导出时需注意 inputs 需与原始 torch 定义一致，部分模型可能不按 `input_ids` / `attention_mask` / `position_ids` 的顺序：

```
dummy_input = torch.zeros((1, in_len), dtype=torch.long)
attention_mask = torch.ones((1, in_len), dtype=torch.float)
position_ids = torch.arange(0, in_len, dtype=torch.long).unsqueeze(0)
inputs = (dummy_input, attention_mask, position_ids)
input_names = ["input_ids", "attention_mask", "position_ids"]

torch.onnx.export(
    model, inputs, args.export_llm_path,
    export_params=True, opset_version=19,
    do_constant_folding=True,
    input_names=input_names, output_names=output_names,
    dynamic_axes=dynamic_axes)
```

> **说明**：ONNX 模型的输入中 `attention_mask` 为必须包含的输入，其余输入（`input_ids` / `position_ids` / `num_logits_to_keep` / `per_layer_inputs` 等）根据模型实际情况而定。RKNN3-Toolkit 会根据 `attention_mask` 的连接关系将 attention op 分为 N 种并分别配置。ONNX 模型不能有 KVCache 输入输出（通常将 config 中的 `use_cache` 设为 False 导出）。

对于 MLLM 模型，除上述 LLM 部分的 ONNX 导出外，还需将多模态编码部分（如 Vision / Audio）单独导出为独立的 ONNX 模型。rknn3-model-zoo 将此过程封装在 `export_vision.py` 等脚本中，导出方法与通用模型 ONNX 导出类似，通过 `torch.onnx.export` 完成。多模态编码部分的转换与通用模型一致，详见 2.1 通用模型转换。

2. 导出 config 文件

config 文件是 RKNN3-Toolkit 转换 RKNN 模型需要的配置文件，包含 chat template、vocab size、hidden size 等模型规格参数及量化相关配置信息：

```
export_llm_config(args.model_path, os.path.splitext(args.export_llm_path)[0] +
'.config.pkl', chat_context, prompt)
```

3. 导出 tokenizer 文件

tokenizer 用于文本与 token id 之间的转换。板端推理时需要 `tokenizer.gguf` 文件及 `libtokenizer.a` 库：

- `tokenizer.gguf`：通过 `export_tokenizer` 导出，含分词和词表信息，对应 LLM 模型的 `tokenizer.json`。
- `libtokenizer.a`：解析 `tokenizer.gguf` 词表信息，提供文本转 token id、token id 转文本等接口。rknn3-model-zoo 使用 llama.cpp 的 tokenizer，源码路径为 `rknn3_model_zoo/tokenizer`，编译产物 `libtokenizer.a` 位于 `rknn3_model_zoo/3rdparty/tokenizer`，如需调整可修改 `tokenizer/src` 代码重新编译。

```
export_tokenizer(args.model_path, os.path.splitext(args.export_llm_path)[0] +
'.tokenizer.gguf')
```

4. 导出 embedding 文件

embedding 文件包含 LLM 模型的 embedding 数据（Float16 类型），即词表向量，按 Torch 模型中 embedding 权重的原始排布存储，shape 为 (vocab_size, embedding_dim)：

```
export_embed_weight(model.model.embed_tokens.weight,
os.path.splitext(args.export_llm_path)[0] + '.embed.bin')
```

> **说明**：embedding 层操作集中在 CPU 侧，CPU 侧基于 token id 查询对应的 embedding 数据并传输给 NPU。当模型 Embedding 层操作较复杂时，可不生成 embed.bin，由用户在应用逻辑中自行管理 Embedding。Tie Word 模式下也不需要 embed.bin。

2.2.3 LLM和MLLM模型转换

2.2.3.1 LLM 模型转换

LLM 模型转换流程为：配置 `llm_config` -> `rknn.load_llm()` 加载 -> `rknn.build()` 构建 -> `rknn.export_rknn()` 导出。

1. 配置 llm_config

针对 LLM 模型，`rknn.config()` 新增 `llm_config` 参数（Python 字典类型）。用户可先载入默认配置 `DEFAULT_RKNN_LLM_CONFIG`，再根据实际需求修改：

```
from rknn.api import RKNN, DEFAULT_RKNN_LLM_CONFIG
rknn = RKNN(verbose=True)
my_config = DEFAULT_RKNN_LLM_CONFIG.copy()
rknn.config(target_platform='rk1820', llm_config=my_config)
```

`llm_config` 主要有 4 个字典子项：

- **attention_config**：配置 Attention op 的推理机制与 KVCache。模型必须带有 attention 结构的 mask 输入，Toolkit 根据 mask 连接关系将 attention op 分为 N 种分别配置。常用字段：
- `mask_name`：mask 输入名称（不可缺省，如 `attention_mask`）。
- `kvcache_buffer_len`：KVCache 缓存长度，超长时按 FIFO 循环覆写，需 32 对齐。
- `tokens_per_page`：Paged Attention 每页 token 数；设为 `None` 使用原有连续 KVCache（见 4.2.6）。默认 `None`；配置为正整数后启用分页 KVCache，建议为 64 的整数倍；页越大推理性能越好，但提升幅度不会太大。非正数或 SlidingAttention 类型会自动回退为 `kvcache_buffer_len`。
- `request_num`：分页 Attention 的请求数，当前版本仅支持 1。
- `kvcache_dtype`：KVCache 数据类型，支持 `Float16` / `Int8_to_F16` / `Int4_to_F16`，内存占用依次降低。
- `attention_type`：Attention 类型，支持 `FullAttention` / `SlidingAttention`。
- `position_name`：位置编码输入名称，用于提取 RoPE，无位置编码时可设为 None。
- `max_position_embeddings`：位置编码最大长度，需 32 对齐。
- `position_embeddings_host_storage`：位置编码（RoPE）存储位置，`False` 存设备内存，`True` 存 Host 端内存。

- `mrope_type`：mrope 类型，支持 `Qwen2.5-VL` / `Qwen3-VL` / `Qwen3.5`，开启时须配置 `position_name`。
- `mrope_section`：mrope section 信息，对于 Qwen2.5-VL / Qwen3-VL / Qwen3.5 模型可在 config.json 中找到对应配置值。
- `mrope_new_id_name`：mrope 新生成的 position 名称，开启 mrope 后会生成 [seq_len, 3] 维度的新输入。
- **llm_head**：配置 LLM Head 层（通常为较大的 Linear 层）的优化行为。常用字段：
- `device`：计算设备，支持 `rk1820` / `rk3588` / `rk3576`。
- `quantized_dtype`：量化类型，支持 `w16a16` / `w4a16` / `w6a16` / `w8a8`，当平台为 rk3576/rk3588 时强制为 `w8a8`。
- `quantized_method`：量化方法，支持 `layer` / `channel` / `group{SIZE}`，当平台为 rk3576/rk3588 时强制为 `channel`。
- **vocab**：配置字典层优化。`tie_embeddings` 为 True 时 vocab 与 llm_head 共享权重（Tie Word 模式），无需 embed.bin。
- **keep_one_logit**：配置对输出指定轴取单值（插入 gather op），减少 Prefill 阶段 lm_head 计算量。`idx_name` 指定 gather indices 输入名称。

> **说明**：`llm_config` 各子项的完整字段与默认值见 `DEFAULT_RKNN_LLM_CONFIG`，更详细的参数说明请参考《RKNN3_Toolkit_Python_API_参考》。

2. 加载 LLM 模型

通过 `rknn.load_llm()` 加载 ONNX 模型，参数包括 `model`（ONNX 路径）、`config`（config.pkl 路径）、`seq_lens`（单次推理支持的输入 token 数列表）：

```
rknn.load_llm(model='./Qwen2.5-3B.onnx',
              config='./Qwen2.5-3B.config.pkl',
              seq_lens=[1, 128])
```

> **说明**：`seq_lens` 用于配置多种 shape 输入（如 `[1, 64]` 生成 prefill 长度 64 与 decoder 长度 1 的模型），仅适用于标准输入（input_ids / attention_mask / position_ids / num_logits_to_keep）的模型；非标准输入模型请用 `rknn.config()` 的 `dynamic_input` 参数配置。

3. 构建并导出

通过 `rknn.build()` 构建模型（可开启量化），再通过 `rknn.export_rknn()` 导出部署产物：

```
rknn.build(do_quantization=True)
rknn.export_rknn(args.rknn_path)
```

> **说明**：LLM 模型转换不需要提供 `dataset`（校准数据集），`dataset.txt` 仅在通用模型转换量化（见 2.1 / 2.3.3）时使用。

导出产物包括 `.rknn`、`.weight`、`.embed.bin`、`.tokenizer.gguf` 等（见 1.3.5 模型转换和部署产物说明）。LoRA 权重的加载见 4.2.8 LoRA。

2.2.3.2 MLLM 模型转换

MLLM 模型由多模态编码部分（如 Vision / Audio）与 LLM 部分组成，转换时两部分分别独立进行：多模态编码部分本质是通用模型，通过 `rknn.load_onnx()` 加载并构建导出（转换方法见 2.1）；LLM 部分按上述 LLM 模型转换流程（配置 `llm_config` -> `rknn.load_llm()` -> `rknn.build()` -> `rknn.export_rknn()`）转换。两部分在板端推理时联合使用。

以 Qwen2.5-VL-3B 模型为例，rknn3-model-zoo 会导出 `Qwen2.5-VL-3B-vision.onnx`（图像特征提取的 vision 编码器）和 `Qwen2.5-VL-3B-llm.onnx`（文本生成的 LLM 部分）两个文件，分别转换如下：

```
# 构建视觉模型
rknn.config(target_platform='rk1820',
              quantized_dtype='w4a16', quantized_algorithm='normal',
              quantized_method='group32', core_num=8)
rknn.load_onnx(model='../../model/vision/Qwen2.5-VL-3B-vision.onnx')
rknn.build(do_quantization=True, dataset=args.dataset_path)
rknn.export_rknn('../../model/vision/Qwen2.5-VL-3B-vision.rknn')

# 构建llm模型
rknn.config(target_platform='rk1820',
              quantized_dtype='w4a16', quantized_algorithm='normal',
              quantized_method='group32')
rknn.load_llm(model='../../model/llm/Qwen2.5-VL-3B-llm.onnx',
            config='../../model/llm/Qwen2.5-VL-3B-llm.config.pkl')
rknn.build(do_quantization=True)
rknn.export_rknn('../../model/llm/Qwen2.5-VL-3B-llm.rknn')
```

> **说明**：视觉模型在板端推理时使用通用 RKNN3 API（`rknn3_run()` 等），推理得到图像 embedding（img_embeds）；LLM 推理时将 prompt 与 img_embeds 同时传入 LLM 模型进行多模态推理。多模态输入的部署详见 3.3.6 多模态输入。

2.2.4 模型和Tokenizer适配

本节介绍 LLM / MLLM 模型在板端部署时如何使用 tokenizer 完成文本与 token id 之间的转换。tokenizer 将文本转为 token id 供模型推理，并将模型输出的 token id 转回文本。

Tokenizer 文件与库

板端推理时需要以下两个文件：

- `tokenizer.gguf`：分词器文件，含分词规则和词表信息，对应 LLM 模型的 `tokenizer.json`，在模型转换阶段通过 `export_tokenizer` 导出（见 2.2.2）。
- `libtokenizer.a`：分词器库，负责解析 `tokenizer.gguf` 并提供文本转 token id、token id 转文本等接口。rknn3-model-zoo 使用 llama.cpp 的 tokenizer，位于 `rknn3_model_zoo/3rdparty/tokenizer`，如需调整可修改源码重新编译。

Tokenizer 类

rknn3-model-zoo 封装了 `Tokenizer` 类（头文件 `Tokenizer.h`），主要接口如下：

- `Tokenizer(type, tokenizer_path)`：构造函数，加载 `tokenizer.gguf` 并初始化分词器，`type` 取 `TOKENIZER_BACKEND_LLAMA`。

- `GetVocabInfo(&info)`：获取词表信息（`vocab_size`、`special_bos_id`、`special_eos_id`、`linefeed_id` 等），用于 LLM 会话初始化。
- `Tokenize(text, text_len, tokens, n_tokens_max)`：将文本分词为 token id 序列，返回 token 数量。
- `TokenToPiece(token)`：将单个 token 转为字符串片段，用于流式输出。
- `Decode(tokens, n_tokens)`：将 token 序列还原为完整字符串，用于非流式输出。

与模型结合使用

板端部署时，tokenizer 通过 `RKLLMCallback` 的 `tokenizer_callback` 注册到 LLM 会话，由 Runtime 在推理时调用完成文本到 token id 的转换。使用流程如下：

1. 创建 `Tokenizer` 对象并获取词表信息：

```
#include "Tokenizer.h"

// tokenizer_path 为 tokenizer.gguf 文件路径
Tokenizer* tokenizer = new Tokenizer(TOKENIZER_BACKEND_LLAMA, tokenizer_path);
VocabInfo vocab_info;
tokenizer->GetVocabInfo(&vocab_info);
```

1. 实现 `tokenizer_callback`，在回调中调用 `Tokenize` 将文本转为 token id：

```
int tokenizer_callback(void *userdata, const char *text, int32_t text_len,
                       int32_t *tokens, int32_t n_tokens_max)
{
    Tokenizer *tokenizer = (Tokenizer *)userdata;
    int n_tokens = tokenizer->Tokenize(text, text_len, tokens, n_tokens_max);
    return n_tokens;
}
```

1. 将 tokenizer 注册到会话回调，推理时 Runtime 自动调用 `tokenizer_callback` 完成分词：

```
RKLLMCallback callback;
memset(&callback, 0, sizeof(RKLLMCallback));
callback.tokenizer_callback = tokenizer_callback;
callback.tokenizer_userdata  = tokenizer;
rknn3_session_set_callback(session, &callback);
```

1. 在结果回调中用 `TokenToPiece` 或 `Decode` 将输出 token 转回文本：

```
// 流式输出：逐 token 转字符串
std::string piece = tokenizer->TokenToPiece(result->token_ids[0]);
// 非流式输出：批量转字符串
std::string text = tokenizer->Decode(result->token_ids, result->num_tokens);
```

> **说明**：`tokenizer_callback` 是可选回调，仅在需要自定义分词器时使用。若不注册，Runtime 使用默认分词逻辑。tokenizer 相关接口的详细说明见《RKNN3_Runtime_C_API_参考》中的 `LLMTokenizerCallback`。

2.2.5 转换结果验证

LLM / MLLM 模型转换完成后，可通过 rknn3-toolkit-lite（板端 Python 推理库）进行验证。LLM 模型为自回归生成，需在初始化时设置 `llm_mode=True`，加载模型后进行推理验证，检查模型能否正常生成文本（或图像+文本结果）：

```
from rknn3lite.api import RKNN3Lite

rknn_lite = RKNN3Lite(llm_mode=True)
rknn_lite.load_rknn(model_path='./Qwen2.5-3B-Instruct.rknn',
                    weight_path='./Qwen2.5-3B-Instruct.weight')
# 进行推理验证
...
rknn_lite.release()
```

> **说明**：rknn3-toolkit-lite 的详细使用见 3.4 Toolkit Lite Python部署。精度分析、性能评估等更详细的评估方法见 2.4 模型评估。

2.2.6 模型分段导出

本节重点介绍 LLM 按 Transformer block 连续切分并导出为多个 ONNX / RKNN 分段的方法。模型是否采用多卡流水线与模型本身是否为 MoE 是两个独立选择；MoE 模型的转换说明见 2.2.7。

模型分段原理

分段边界位于 Transformer block 之间，不改变 block 内部的计算顺序。最后一个分段除了承担末尾 block，还固定包含 final norm 和 lm_head：

```
stage0: block 0 ... block k0
stage1: block k0+1 ... block k1
...
stageN-1: block kN-2 ... block L-1 + norm + lm_head
```

分段数量由模型规模、目标设备内存和部署方式决定。用于多卡流水线时，一个分段通常对应一张 RK1820 / RK1828 卡；分段时应同时考虑每段的 block 权重、最后一段的 norm/lm_head、KV cache 和内部 scratch，不能简单按 block 数平均切分。

分段参数

- multi_segment：开启多分段 ONNX / RKNN 导出。
- num_segments N：指定分段数量；未指定或设置为 0 时，脚本按模型层数和内存规划策略推导分段数量。
- segment_layer_counts c0 c1 ...：手动指定每个分段的连续 block 数量。列表长度就是分段数量，所有值必须为正数且总和等于模型的 block 总数；该参数优先于 num_segments。

- max_context_len：设置每个分段的 KV cache 和最大上下文长度，应结合目标卡内存选择。

不指定 segment_layer_counts 时，导出脚本会根据权重大小和目标卡容量自动寻找较均衡的连续切分，并为最后一段额外预留 norm/lm_head。脚本生成的 segment_plan.json 会给出实际分段、建议上下文长度和每卡内存估算，最终仍应以每个分段的 model_report.html 和板端结果为准。

分段导出流程

以 Qwen3.5-9B 切分为 2 段为例，先导出 ONNX 分段，再逐段构建 RKNN：

```
cd rknn3-model-zoo/examples/multicard/python/qwen3_5
python export_llm_segment.py --model_path /path/to/Qwen3.5-9B --multi_segment --
num_segments 2
python export_rknn_segment.py --multi_segment --num_segments 2
```

手动指定分段时，例如将 32 个 block 切成 8、8、16 三段：

```
python export_llm_segment.py --model_path /path/to/model --multi_segment --
segment_layer_counts 8 8 16
python export_rknn_segment.py --multi_segment --num_segments 3
```

导出后应检查以下产物：

产物	用途
model_seg0.onnx ... model_segN.onnx	各连续 block 分段的 ONNX 图
model_seg0.rknn、model_seg0.weight ... model_segN.rknn、model_segN.weight	各分段的 RKNN 图和外部权重
model.tokenizer.gguf、model.embed.bin	Host 侧 tokenizer 和非 Tie 模式下的 embedding
model_segN.safetensors	使用外置 rope cache 的模型所需的 cache 文件
model.segment_plan.json	自动切分、上下文长度和每卡内存的参考规划
tmp_segN/model_report.html	各分段 RKNN 构建后的实际内存报告

如果只调整 KV cache 长度，可以在已有 tmp_segN 的基础上使用 rebuild；如果修改了 ONNX、分段数量或 block 切分方式，必须重新执行完整的 ONNX 和 RKNN 导出。

2.2.7 MoE模型转换

MoE 模型转换与多卡部署是两个独立问题。MoE 模型可以作为单个 LLM 模型在一张卡上转换和运行；只有当模型规模、权重或运行时内存超过单卡容量时，才需要结合 2.2.6 的分段方法部署到多张卡。

MoE图转换

MoE 模型包含 router、routed experts 和 shared expert。导出时不能让普通 ONNX trace 只保留 dummy input 命中的专家。examples/multicard 中的 MoE 导出脚本会将依赖数据的专家分发逻辑转换为 `rknn::exUnifiedMoE` 图节点，并保留完整的专家权重。

当前版本的 `exUnifiedMoE` 由匹配的 RKNN3 Toolkit / Runtime 直接支持，MoE 模型转换和运行无需额外注册 plugin。具体模型的导出参数和输入输出约定以对应脚本为准。

MoE与模型分段

如果 MoE 模型需要分段，仍按 Transformer block 连续切分，不执行 expert parallelism。同一个 MoE layer 的 router、全部 routed experts 和 shared expert 必须放在同一个分段中；分段 planner 也必须把完整专家权重和 MoE workspace 计入，不能只按实际激活的 top-k 专家估算。MoE 不需要分段时，不应为了使用 MoE 而开启 `multi_segment`。

相关模型路径包括：

模型	导出脚本
Qwen3-MoE（如 Qwen3-30B-A3B）	examples/multicard/python/qwen3/
Qwen3.5-MoE	examples/multicard/python/qwen3_5/
MiniCPM-V-4.7	examples/multicard/python/minicpm4_7/（包含 llm/ 和 vision/）
MiniCPM5-15B-A2.5B	examples/multicard/python/minicpm5/

2.3 模型量化

2.3.1 量化基本概念

2.3.1.1 量化定义

模型量化是指将深度学习模型中的浮点参数或者激活值转换为定点表示，如将 FP32 转换为 INT8 等。量化能够降低内存占用，实现模型压缩和推理加速，但通常会带来一定程度的精度损失。

根据浮点参数与定点数之间映射函数的形式，量化可分为线性量化与非线性量化。常见的非线性量化包括指数量化、码本量化、分段线性量化等。RKNN3-Toolkit 仅支持线性量化。在线性量化中，定点数之间的间隔是均匀的，例如 INT8 线性量化将量化范围均匀划分为 256 个数，因此也称为均匀量化。根据量化零点是否为 0，线性量化可分为对称量化与非对称量化。对称量化是非对称量化的简化形式，非对称量化通常能更好地处理数据分布不均匀的情况，因此实际使用中多采用非对称量化方案。

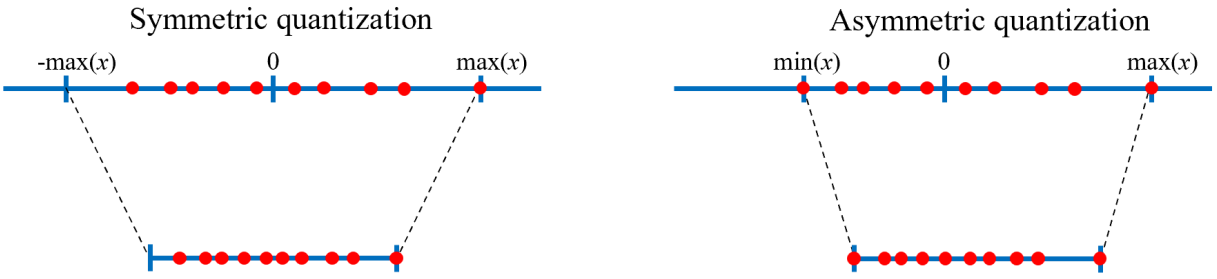


图2-3 线性对称量化和线性非对称量化

2.3.1.2 量化计算

以线性非对称量化为例，浮点数量化为有符号定点数的计算如下：

$$x_{int} = clamp(\lfloor \frac{x}{s} \rceil + z; -2^{b-1}, 2^{b-1} - 1) \tag{2-1}$$

其中$x为浮点数，$x_{int}$为量化定点数，$\lfloor \cdot \rceil$为四舍五入运算，$s$为量化比例因子，$z$为量化零点，$b$为量化位宽，如$INT8$数据类型中$b$为$8$；$clamp$为截断运算，具体定义如下：

$$clamp(x;a,c)=\begin{cases}a, & x<a,\\ x, & a\le x\le c,\\ c, & x>c,\end{cases} \tag{2-2}$$

从定点数转换为浮点数称为反量化过程，具体定义如下：

$$x\approx\widehat{x}=s(x_{int}-z) \tag{2-3}$$

其中 $\widehat{x}$ 也被称为伪量化浮点数。易知，由于量化计算中的取整操作与截断操作，$\widehat{x}$ 与 x 并不相等，两者间的差异也被称为量化误差。

设量化范围为$\$(q_{\min},q_{\max})$，截断范围为$(c_{\min},c_{\max})$，量化参数$s$和$z$\$的计算公式如下：

$$s=\frac{q_{\max}-q_{\min}}{c_{\max}-c_{\min}}=\frac{q_{\max}-q_{\min}}{2^b-1} \tag{2-4}$$

$$z=c_{\max}-\lfloor\frac{q_{\max}}{s}\rceil \quad 或\ z=c_{\min}-\lfloor\frac{q_{\min}}{s}\rceil \tag{2-5}$$

其中截断范围是根据量化的数据类型决定，例如INT8的截断范围为（-128, 127）；量化范围根据不同的量化算法确定。令上述定义公式中的$z=0$，即可得到线性对称量化的计算定义。

2.3.1.3 量化粒度

量化粒度表示模型浮点张量中，哪些参数作为一个整体进行量化。RKNN3-Toolkit中支持Per-Layer / Per-Channel / Group 三种量化粒度。Per-Layer量化是将网络层权重或者激活的所有通道作为一个整体进行量化，所有通道共享相同的量化参数。Per-Channel量化是将网络层的权重的各个输出通道独立进行量化，每个通道有自己的量化参数。Group量化仅针对线性层权重，是在Per-Channel量化的基础上，对每个输出通道的元素进行分组，每个分组独立进行量化。

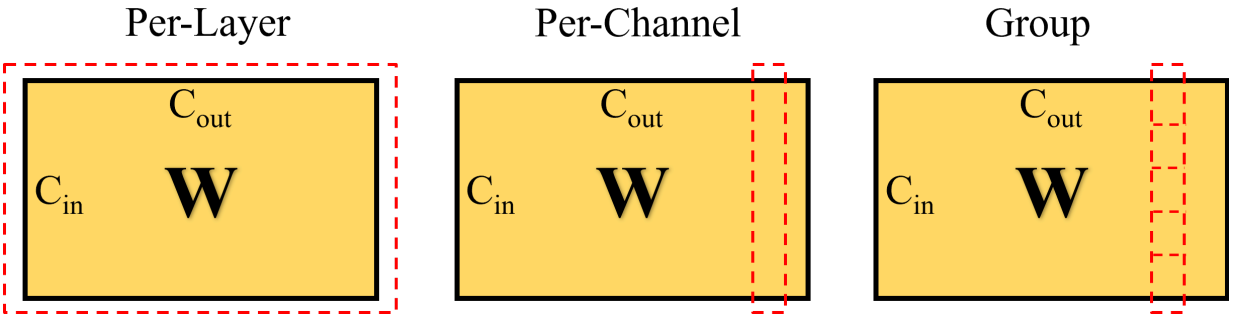


图2-4 Per-Layer量化、Per-Channel量化和Group量化

> **说明**：RKNN3-Toolkit 中的 Group 量化和 Per-Channel 量化仅针对权重；激活值和中间值仍采用 Per-Layer 量化；Group 量化仅在 a16 条件下对线性层有效。

2.3.1.4 量化算法

量化算法是指在给定量化形式下（如线性量化），求解浮点 Tensor 到低比特定点数表示及其量化参数的方案。按实现方式，可分为量化感知训练与训练后量化两类。量化感知训练通过在训练过程中微调模型参数和量化参数以提升精度，通常需要深入理解训练流程并依赖大量训练数据，因此很少在模型部署工具链中直接使用。训练后

量化则通常只需少量校准数据，甚至无需校准数据，通过较低成本的计算得到量化参数。在 RKNN3-Toolkit 中支持四种训练后量化算法：Normal / KL-Divergence / MMSE 和 GRQ。

2.3.2 Toolkit量化流程概览

RKNN3-Toolkit 提供内部量化和RKQuantizer量化两种流程。内部量化流程先将浮点模型导出为 浮点ONNX 模型，然后通过 `rknn.config()` ⟶ `rknn.load_llm()/rknn.load_onnx()` ⟶ `rknn.build()` ⟶ `rknn.export_rknn()` 导出 RKNN 模型。该流程兼容性强，支持 CNN / VIT / LLM / MLLM 等模型，可使用Normal / KL-Divergence / MMSE三种量化算法，但不支持 GRQ 量化算法。

RKQuantizer量化流程主要面向 VIT / LLM / MLLM 等标准架构模型。该流程使用 Toolkit 中的量化组件 RKQuantizer 执行 GRQ 等量化算法，将模型中的浮点权重替换为伪量化权重并导出伪量化 ONNX 模型；随后同样通过 `rknn.config()` ⟶ `rknn.load_llm()/rknn.load_onnx()` ⟶ `rknn.build()` ⟶ `rknn.export_rknn()` 导出 RKNN 模型。重要的是，在 ONNX 转换为 RKNN 时，`rknn.config()` 必须配置 `quantized_algorithm='normal'`，不可使用 `MMSE` 或 `KL-Divergence`，并且设置正确的 `quantized_dtype`（`w4a16`、`w6a16` 或 `w8a16`）和 `quantized_method='group32'`。本章节对RKQuantizer使用做简要介绍。更详细的接口和使用说明（含混合精度量化、校准数据准备、显存占用等）请参考《RKNN3_Toolkit_Python_API_参考》文档。

内部量化流程：

```
浮点 PyTorch 模型 ⟶ 导出 浮点ONNX ⟶ rknn.config() ⟶
rknn.load_llm()/rknn.load_onnx() ⟶ rknn.build() ⟶ rknn.export_rknn()
```

RKQuantizer量化流程：

```
浮点 PyTorch 模型 ⟶ RKQuantizer ⟶ fake-quant PyTorch 模型  ⟶ 导出 fake-quant
ONNX ⟶ rknn.config() ⟶ rknn.load_llm()/rknn.load_onnx() ⟶ rknn.build() ⟶
rknn.export_rknn()
```

2.3.3 内部量化流程

2.3.3.1 数据类型

内部量化支持的量化数据类型有`w4a16`、`w6a16`、`w8a16`、`w16a16`、`w8a8`，默认值为`w16a16`。其中w4/w6/w8/w16代表权重量化为对应位宽（int4/int6/int8/int16或float16），a8/a16代表激活量化为int8/float16类型。

2.3.3.2 内部量化算法

内部量化共支持 Normal / KL-Divergence / MMSE 三种量化算法：

- Normal：通过计算浮点值的最大值和最小值直接确定量化范围。该算法不会产生截断误差，但对异常值敏感，运行速度快，适用于一般场景。
- KL-Divergence：通过比较浮点值和定点值的分布，调整阈值并最小化 KL 散度来确定量化范围。该算法更适应非均匀数据分布，能够缓解少数异常值的影响，运行速度略慢于 Normal。
- MMSE：通过最小化浮点值与反量化后浮点值之间的均方误差来确定量化范围。该算法通常精度更高，但需要较多计算资源，运行速度较慢、内存开销较大。相比 KL-Divergence，它在异常值场景下往往有更好的精度表现。

建议用户优先使用 Normal 算法，当遇到量化精度问题时可尝试 KL-Divergence 或 MMSE。

2.3.3.3 校准数据准备

对于内部量化流程，校准数据集用于计算激活值的量化范围。选择时应覆盖模型实际应用场景的不同数据分布，例如分类模型的校准数据应包含实际场景中的不同类别图片。一般推荐校准数据集数量为 20~200 张，可根据量化算法的运行时间适当调整。需要注意的是，在 a16 条件下，Normal 量化不需要设置校准数据集；对其它量化算法来说，增加校准数据集数量会增加运行时间，但不一定能提高精度。

> **说明**：`dataset.txt` 校准数据集仅用于通用模型转换量化，LLM 模型转换不需要提供。

内部量化校准数据目前支持文本文件格式，用户可以将用于校准的图片（jpg 或 png 格式）或 npy 文件路径写入 `.txt` 文件。文本文件中每一行为一条路径信息，如：

```
a.jpg
b.jpg
```

如有多个输入，则每个输入对应的文件用空格隔开，如：

```
a0.jpg a1.jpg
b0.jpg b1.jpg
```

2.3.3.4 具体示例

示例1：ResNet50 内部量化流程示例：

```
from rknn.api import RKNN

model_path = "resnet50-v2-7.onnx" ## 提前导出的浮点ONNX模型
output_path = 'resnet50-v2-7.rknn'
DATASET_PATH =
'../../../datasets/imagenet/ILSVRC2012_img_val_samples/dataset_20.txt'

# Create RKNN object
rknn = RKNN(verbose=False)

# Pre-process config
print('--> Config model')
rknn.config(mean_values=[[255*0.485, 255*0.456, 255*0.406]],
            std_values=[[255*0.229, 255*0.224, 255*0.225]],
            target_platform="rk1828",
            input_attrs={'input': {'dtype': 'uint8', 'layout': 'NHWC'}},
            quantized_dtype='w8a8',           ## "w8a16", "w4a16" 等可选
            quantized_algorithm='normal',     ## "MMSE", "KL-Divergence" 可选
            quantized_method='channel',)
print('done')

# Load model
print('--> Loading model')
ret = rknn.load_onnx(model=model_path, inputs=['data'], input_size_list=[[1, 3, 224,
224]])

if ret != 0:
    print('Load model failed!')
    exit(ret)
print('done')

# Build model
print('--> Building model')
ret = rknn.build(do_quantization=True, dataset=DATASET_PATH)
if ret != 0:
    print('Build model failed!')
    exit(ret)
print('done')

# Export rknn model
print('--> Export rknn model')
ret = rknn.export_rknn(output_path)
if ret != 0:
    print('Export rknn model failed!')
    exit(ret)
print('done')
```

2.3.4 RKQuantizer量化流程

2.3.4.1 数据类型

RKQuantizer支持的量化数据类型有`w4a16`、`w6a16`、`w8a16`，权重量化方法(`quantized_method`)仅支持group32。

2.3.4.2 RKQUANTIZER量化算法

RKQuantizer量化支持 GRQ 和 Normal 两种算法。GRQ 基于反向传播，最小化网络中每个 block（如 LLM 中的 DecodeLayer）在量化前后输出的均方误差，求解 block 内各线性层的量化参数。与 KL-Divergence 和 MMSE 这类最小化张量量化误差的算法不同，GRQ 的目标是使量化后模型的输出更接近浮点模型的输出，因此通常具有更高的量化精度，是首选算法。由于 GRQ 使用反向传播，建议在 GPU / CUDA 环境下运行。

2.3.4.3 校准数据准备

Normal量化算法无需校准数据集。GRQ算法则需要使用校准数据集来统计各层激活分布并对权重进行微调。校准数据集以一个 **JSON 文件** 的形式提供，文件内容为一个列表，列表中每个元素为一条样本。量化组件 RKQuantizer 会逐条读取样本并构造模型输入，目前支持以下两种样本格式，用户可根据模型类型自行选择。

方式一：hook 捕获输入（通用，推荐）

通过 PyTorch 的 `register_forward_pre_hook` 在浮点模型前向时捕获待量化子模块的真实输入，pickle 序列化保存后构建 JSON 索引文件。适用于所有模型，包括 MLLM、语音模型等非纯文本输入的模型。

JSON 文件中每条样本为一个字典，仅包含一个 `sample` 字段，其值为 pickle 文件的相对路径（相对于该 JSON 文件所在目录）。pickle 文件内容为一个字典，包含 `args`（位置参数）与 `kwargs`（关键字参数）两个键，分别对应量化模型前向时的位置输入与关键字输入。格式如下：

```
[
  {"sample": "model_inputs/sample_0"},
  {"sample": "model_inputs/sample_1"}
]
```

获取该数据的标准做法是：在待量化的浮点模型上，对量化模型注册 `register_forward_pre_hook`（需开启 `with_kwargs=True` 以同时捕获关键字参数），在 hook 中将输入存入列表并抛出异常以提前终止前向（避免执行完整推理），随后将捕获的输入 pickle 落盘。rknn3-model-zoo 将该机制封装为公共工具 `py_utils/cali_data.py`，提供 `capture_module_input(hook_module, run, *args, **kwargs)` 函数：它把"注册 forward pre hook → 调用模型 → 捕获 `(args, kwargs)` → 抛 `StopForward` 短路前向 → 移除 hook"打包成一行，调用方无需感知 hook 细节。其内部 hook 会 `raise StopForward` 终止被捕获子模块的前向，因此即便传入 `model.generate` 也只会跑一次前向即中断，不会真正生成 token。举例如下：

```python
import os
import json
import pickle
import torch
from transformers import AutoProcessor
# 引入公共 hook 捕获工具（路径视工程结构调整）
from py_utils.cali_data import capture_module_input

model_path = 'Qwen/Qwen2.5-VL-3B-Instruct'
model = ...                          # 加载浮点模型，torch_dtype 与量化时保持一致
processor = AutoProcessor.from_pretrained(model_path)

# 逐条样本前向，一行捕获进入待量化子模块的输入
info = []
datasets = json.load(open('./dataset.json', 'r'))    # 原始标注数据（含图像/问题等）
for idx, data in enumerate(datasets):
    inputs = processor(...)                  # 按模型要求构造完整输入
    inputs = inputs.to(model.device)
    with torch.inference_mode():
        # hook_module 传量化模型本身（即 load_model 传入的子模块，这里量化 vision 部分即传
model.visual），
        # 捕获的是该子模块最外层入口的输入，而非其内部第一个 transformer block 的输入。
        # run 传要触发的模型调用（lambda 包裹，可含 prepare_inputs_embeds 等前置步骤）
        temp = capture_module_input(
            model.visual,
            lambda: model(**inputs)
        )
        # temp 即 {"args": (...), "kwargs": {...}}；若本次输入未流经该子模块则为 None

    # 将捕获的输入 pickle 落盘，并记录到索引 JSON
    sample_name = 'sample_{}'.format(idx)
    os.makedirs('./model_inputs', exist_ok=True)
    with open(os.path.join('./model_inputs', sample_name), 'wb') as f:
        pickle.dump(temp, f)
    info.append({"sample": "model_inputs/{}".format(sample_name)})

with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(info, f, indent=2, ensure_ascii=False)
```

方式二：纯文本 JSON（仅适用于纯文本 LLM）

对于输入仅为文本的 LLM 模型，可采用一种更简单的方式：JSON 文件中每条样本为一个包含 `input` 与 `target` 两个键的字典，RKQuantizer 内部会自动使用 `tokenizer(input + target)` 将其编码为模型输入。该方式无需加载浮点模型、无需 hook，准备成本最低。格式如下：

```
[
  {
    "input": "半胱氨酸和酪氨酸在人体内可分别由什么转变而来。\nA. 胱氨酸和蛋氨酸\nB. 蛋氨酸和苯丙
氨酸",
    "target": "\n答案:\n B"
  },
  {
    "input": "Natalia sold clips to 48 of her friends in April, and then she sold
half as many in May. How many altogether?",
    "target": ""
  }
]
```

其中 `input` 为提示文本，`target` 为期望的输出文本（可为空字符串）；二者拼接后整体送入 tokenizer 编码，`target` 中可包含答案或思维链等补充上下文，使校准分布更贴近真实生成场景。举例如下：

```
import json

# 直接手写或从已有数据集整理，保存为 JSON 即可
samples = [
    {"input": "请简述牛顿第一定律的内容。", "target": "\n答案:\n 牛顿第一定律指出..."},
    {"input": "1+1等于几? ", "target": "\n答案:\n 2"},
]
with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(samples, f, ensure_ascii=False, indent=2)
```

> **说明**：该方式仅适用于纯文本输入的 LLM。对于多模态（MLLM）、语音等含非文本输入的模型，必须使用方式一。

校准数据的质量极大影响 GRQ 量化后模型的精度，选择时应遵循核心原则：**使用真实应用场景下的多样化数据**，使校准数据分布尽量贴近模型部署后的实际输入分布。具体建议如下：

- **覆盖真实场景**：校准样本应来源于模型实际部署场景的真实输入，例如实际的 system prompt、function-tool 等。对于 agent 任务，这类输入尤为重要，相关输入可以通过 `load_model` 接口加载。
- **保持多样性**：样本应具有多样性，避免内容高度同质化，以保证量化参数能够适应不同输入分布。一般建议准备 20~200 个样本。

2.3.4.4 混合量化设置

RKQuantizer 提供两种混合精度量化方式：手动按层指定（`layer_quant_config`）与自动按比例挑选（`auto_hybrid_rate`）。

方式一：手动按层指定（layer_quant_config）

由用户显式指定哪些层使用 w6a16 或者 w8a16。`layer_quant_config` 可传入字典或 JSON 文件路径，格式为 `{layer_pattern: "w8a16"}`，其中 `layer_pattern` 为用于匹配 Linear 层名称的 shell-style 通配模式，支持 `*` 通配符（用于跨所有 transformer block 匹配同名子层）。匹配到的层将使用配置中指定的量化类型，未匹配到的层按 `quantized_dtype` 量化。当前 `layer_quant_config` 仅支持 "w8a16" 或者 "w6a16"，且不可与 `auto_hybrid_rate` 同时使用。

以 Qwen3.5 模型为例，其 `layer_quant_config.json` 内容如下，将线性注意力相关的 qkv/b/a/out_proj 层以及 v_proj、down_proj 层指定为 w8a16 (或者 w6a16)：

```
{
  "model.layers.*.linear_attn.in_proj_qkv": "w8a16",
  "model.layers.*.linear_attn.in_proj_b": "w8a16",
  "model.layers.*.linear_attn.in_proj_a": "w8a16",
  "model.layers.*.linear_attn.out_proj": "w8a16",
  "model.layers.*.self_attn.v_proj": "w8a16",
  "model.layers.*.mlp.down_proj": "w8a16"
}
```

对应量化调用如下：

```python
from rknn.quantization.api import RKQuantizer

QuantTool = RKQuantizer(verbose=True)

QuantTool.load_model(...)

quant_model = QuantTool.quantize(quantized_dtype="w4a16",
                                 quantized_method="group32",
                                 quantized_algorithm='grq',
                                 dataset='./dataset.json',
                                 awq_config=None,
                                 layer_quant_config="layer_quant_config.json",
                                 auto_hybrid_rate=0.)
```

方式二：自动按比例挑选（auto_hybrid_rate）

无需逐层指定，由 RKQuantizer 根据校准数据自动评估各层对量化的敏感度，并按指定比例将最敏感的若干 Linear 层保持 w8a16。`auto_hybrid_rate` 取值为 [0, 1) 的浮点数，表示被设为 w8a16 的 Linear 层占比，例如 0.2 表示约 20% 的 Linear 层会被设为 w8a16，其余按 `quantized_dtype` 量化。该参数不可与 `layer_quant_config` 同时使用。举例如下：

```python
from rknn.quantization.api import RKQuantizer

QuantTool = RKQuantizer(verbose=True)

QuantTool.load_model(...)

## auto_hybrid_rate=0.2 表示 20% 的 Linear 层会被设置为 w8a16 类型
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm="grq",
                           dataset='./dataset.json',
                           auto_hybrid_rate=0.2,
                           awq_config=None)
```

导出各层量化类型（export_op_quantized_dtype）

无论采用上述哪种混合精度方式，量化完成后都需要调用 `export_op_quantized_dtype` 将各层实际使用的量化类型导出为 JSON 文件，供后续 `rknn.build()` 阶段对齐使用。该接口需要传入导出后的 ONNX 模型路径，用于建立 torch 层名与 ONNX 算子之间的映射；导出的 JSON 为 `{onnx_op_name: dtype}` 格式，key 为 ONNX 模型中 MatMul/Gemm 等算子的名称，value 为该算子的量化类型。以 Qwen3.5 为例，导出的 `layer_bit.json` 内容如下：

```json
{
  "onnx::MatMul_8968": "w8a16",
  "onnx::MatMul_8973": "w8a16",
  "onnx::MatMul_9657": "w8a16",
  ...
}
```

举例如下：

```
# 在导出 ONNX 之后调用，传入 ONNX 模型路径
QuantTool.export_op_quantized_dtype(onnx_path='./Qwen3.5-0.8B-llm.onnx',
                                    op_dtype_path='layer_bit.json')
```

导出 RKNN 模型时的对应修改

由于混合精度量化已在 RKQuantizer 阶段完成（ONNX 权重已按各层类型完成 fake-quant），在通过 `rknn.config()` + `rknn.load_llm()` + `rknn.build()` 导出 RKNN 模型时，需要通过 `rknn.config()` 的 `op_quantized_dtype` 参数将 `layer_bit.json` 传入，使 RKNN 转换阶段按各层指定的量化类型还原，避免转换阶段对敏感层重新做低位量化导致精度回退。`op_quantized_dtype` 接受字典或 JSON 文件路径，目前仅对 MatMul / Gemm / Linear 算子生效，支持的取值为 w6a16 或者 w8a16。

```
rknn.config(
    target_platform='rk1820',
    quantized_dtype='w4a16',
    quantized_algorithm='normal',
    quantized_method='group32',
    llm_config=llm_config,
    op_quantized_dtype='./layer_bit.json',   # 传入混合精度各层量化类型
)
```

> **说明**：
>
> 1. `layer_quant_config` 与 `auto_hybrid_rate` 不可同时使用。
> 2. `export_op_quantized_dtype` 必须在 ONNX 导出完成之后调用，且传入的 `onnx_path` 应与后续 `rknn.load_llm()` 加载的 ONNX 一致，否则 torch 层名与 ONNX 算子的映射会失败。
> 3. 导出 RKNN 时 `op_quantized_dtype` 指定的 JSON 必须与量化阶段 `export_op_quantized_dtype` 导出的文件对应，二者缺一会导致混合精度配置不生效或转换报错。
> 4. 未开启混合精度量化时，无需调用 `export_op_quantized_dtype`，也无需在 `rknn.config()` 中配置 `op_quantized_dtype`。

2.3.4.5 具体示例

示例1：qwen3 RKQuantizer量化流程示例。 伪量化ONNX模型导出：

```
from transformers import AutoModelForCausalLM, AutoTokenizer, AutoConfig
from rknn.quantization.api import RKQuantizer
import torch

model_path = "Qwen3-1.7B"

## 模型加载
kwargs = {
    'trust_remote_code': True,
    'torch_dtype': torch.bfloat16
}
config = AutoConfig.from_pretrained(model_path, **kwargs)
update_config(config, ['use_cache'], False)
kwargs['config'] = config
model = AutoModelForCausalLM.from_pretrained(model_path, **kwargs)
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)

## 初始化量化工具
QuantTool = RKQuantizer(verbose=True)

## 量化工具加载模型
ret = QuantTool.load_model(model=model, tokenizer=tokenizer, device='cuda',
                           system_prompt=None, tools=None, system_role='system',
user_role='user')
if ret != 0:
    print('Load model failed!')
    exit(ret)

## 执行量化算法
dataset = args.cali_dataset
model = QuantTool.quantize(quantized_dtype="w4a16", quantized_method="group32",
quantized_algorithm='grq', dataset=dataset)
## 注意QuantTool.quantize函数返回的model中的权重已被替换为伪量化权重
model = model.cpu()

'''
  后续执行onnx导出，即可得到伪量化的onnx模型
'''
```

伪量化ONNX模型转换为rknn：

```python
import numpy as np
from rknn.api import RKNN
from rknn.api import RKNN,DEFAULT_RKNN_LLM_CONFIG

ONNX_MODEL = 'Qwen3-1.7B.onnx'
LLM_CONFIG = 'Qwen3-1.7B.config.pkl'
RKNN_MODEL = 'Qwen3-1.7B.rknn'

# Create RKNN object
rknn = RKNN(verbose=True)

llm_config = DEFAULT_RKNN_LLM_CONFIG.copy()
llm_config['attention_config'][0]['kvcache_buffer_len'] = 4*1024
llm_config['attention_config'][0]['max_position_embeddings'] = 4*1024
# llm_config['attention_config'][0]['kvcache_dtype'] = 'Int4_to_F16'

# pre-process config
print('--> config model')
## 注意此处，必须设置quantized_algorithm='normal'，quantized_dtype与quantized_method需要
与QuantTool.quantize中的完全一致。
rknn.config(target_platform="rk1820",
            quantized_dtype='w4a16', quantized_algorithm='normal',
quantized_method='group32', llm_config=llm_config
            )
print('done')

# Load model
print('--> Loading model')
ret = rknn.load_llm(model=ONNX_MODEL, config=LLM_CONFIG)
if ret != 0:
    print('Load model failed!')
    exit(ret)
print('done')

# Build model
print('--> Building model')
ret = rknn.build(do_quantization=True)
if ret != 0:
    print('Build model failed!')
    exit(ret)
print('done')

#Export rknn model
print('--> Export RKNN model')
ret = rknn.export_rknn(RKNN_MODEL)
if ret != 0:
    print('Export rknn failed!')
    exit(ret)
print('done')

rknn.release()
```

2.3.5 其他量化模型转换

对于LLM / MLLM类模型，用户使用其他量化方案（如 GPTQ / QAT 等）得到满足场景需求的量化模型时，可以先将其保存为 Huggingface 格式的伪量化模型，然后直接导出 ONNX，无需再使用 RKQuantizer。需要强调的是，使用其他量化方案时，量化类型仍支持 `w4a16`、`w6a16`、`w8a16`，且要求 `group_size=32`。同样地，在后续 ONNX 转换为 RKNN 的过程中，`rknn.config()` 必须配置

`quantized_algorithm='normal'`，并正确设置 `quantized_dtype`（`w4a16`、`w6a16` 或 `w8a16`）和 `quantized_method='group32'`。

其他量化模型转换流程：

```
浮点 PyTorch 模型 ⟶ AutoGPTQ/AutoAWQ/LLM-Compressor ⟶ fake-quant PyTorch 模型 ⟶
导出 fake-quant ONNX ⟶ rknn.config() ⟶ rknn.load_llm()/rknn.load_onnx() ⟶
rknn.build() ⟶ rknn.export_rknn()
```

2.4 模型评估

模型转换完成后，需通过一系列评估验证其在目标平台上的有效性与性能表现。本章节介绍基于 RKNN3-Toolkit Python API 的评估方法，涵盖以下四个核心维度：

- **推理验证**：验证模型在目标设备上的推理功能是否正常，确保输出结果与预期一致。
- **精度评估**：对比模型在 RKNN 后端与原始框架下的输出差异，量化精度损失。
- **性能评估**：测量模型在 NPU 上的推理耗时、吞吐量等关键性能指标。
- **内存评估**：统计模型运行期间的内存占用情况，为资源规划提供依据。

上述评估任务支持以下两种执行方式，开发者可根据阶段需求灵活选择：

执行方式	适用场景	支持能力
模拟器（Simulator）	无需接入硬件，适用于开发初期或 CI 流水线中的快速验证	推理验证、精度评估
连板（Board Connection）	连接真实硬件，适用于发布前的最终验收与性能调优	推理验证、精度评估、性能评估、内存评估

> **说明**：
>
> 1. 性能评估与内存评估依赖真实 NPU 环境，仅在连板方式下可用。
> 2. 连板精度分析与性能评估需在 `rknn.config()` 中配置 `profile_mode=True` 后构建模型，否则无法获取相应数据。

2.4.1 模型推理验证

模型转换完成后，可通过 `rknn.inference()` 进行推理验证，检查模型转换结果与功能正确性。推理验证支持模拟器和连板两种方式，区别仅在 `init_runtime()` 的参数配置，推理接口一致：

- **模拟器推理**：不指定 `target`，在 PC 上模拟 NPU 运行，无需连接开发板。
- **连板推理**：指定 `target`（目标平台）、`device_id`（设备 ID）、`core_mask`（NPU 核心掩码），在真实 NPU 上执行。

建议优先进行模拟器推理验证（无需硬件，快速确认转换结果），再进行连板推理验证（在真实 NPU 上获取实际推理结果）：

```
# 1. 模拟器推理（不指定 target，无需连接开发板）
rknn.init_runtime()
outputs = rknn.inference(inputs=[img])

# 2. 连板推理（指定 target 和 device_id，在真实 NPU 上推理）
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
outputs = rknn.inference(inputs=[img])
```

> **说明**：模拟器推理无需硬件，适合开发初期快速验证转换结果；连板推理在真实 NPU 上执行，可获取实际推理结果与性能。模拟器性能与板端实际性能有差异，性能评估需连板进行（见 2.4.3）。

2.4.2 精度评估

精度评估通过 `rknn.inference(accuracy_analysis=True)` 开启，对比量化模型与浮点模型推理结果之间的差异，分析量化误差来源。与推理验证一样，精度分析也分模拟器和连板两种：

误差类型	含义	子项
`simulator_error`	模拟器误差（与 golden 对比）	`entire`（累积误差）、`single`（单层独立误差）
`runtime_error`	板端 Runtime 误差	`entire`、`single_sim`（Runtime 与模拟器单层对比）

均报告余弦距离与欧氏距离，偏低/偏高显黄红色。

模拟器精度评估：

```
# 模拟器精度分析（不指定 target）
rknn.init_runtime()
outputs = rknn.inference(inputs=[img], accuracy_analysis=True)
```

分析结果如下：

```
I GraphPreparing : 100%|████████████████████████████████████| 187/187
[00:00<00:00, 6481.46it/s]
I AccuracyAnalysing : 100%|████████████████████████████████████| 187/187
[00:05<00:00, 34.62it/s]

# simulator_error: calculate the output error of each layer of the simulator
(compared to the 'golden' value).
#              entire: output error of each layer between 'golden' and 'simulator',
these errors will accumulate layer by layer.
#              single: single-layer output error between 'golden' and 'simulator',
can better reflect the single-layer accuracy of the simulator.

layer_name                                                  simulator_error
                                                         entire              single
                                                      cos      euc        cos
euc
--------------------------------------------------------------------------------
-------
[Input] images                                     1.00000 | 0.09873   1.00000 |
0.09873
[Conv] /model.0/conv/Conv_output_0                 0.99998 | 4.97621   0.99998 |
4.97621
[exSwish] /model.0/act/Mul_output_0                1.00000 | 3.92585   1.00000 |
1.27413
[Conv] /model.1/conv/Conv_output_0                 1.00000 | 5.41606   1.00000 |
2.07959
...
```

连板精度评估：

```python
# 连板精度分析（指定 target / device_id / core_mask）
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
outputs = rknn.inference(inputs=[img], accuracy_analysis=True)
```

分析结果如下：

```
I GraphPreparing : 100%|████████████████████████████████████| 187/187
[00:00<00:00, 5128.55it/s]
I AccuracyAnalysing : 100%|████████████████████████████████████| 187/187
[00:25<00:00,  7.33it/s]

# simulator_error: calculate the output error of each layer of the simulator
(compared to the 'golden' value).
#              entire: output error of each layer between 'golden' and 'simulator',
these errors will accumulate layer by layer.
#              single: single-layer output error between 'golden' and 'simulator',
can better reflect the single-layer accuracy of the simulator.
# runtime_error: calculate the output error of each layer of the runtime.
#              entire: output error of each layer between 'golden' and 'runtime',
these errors will accumulate layer by layer.
#              single_sim: single-layer output error between 'simulator' and
'runtime', can better reflect the single-layer accuracy of runtime.

layer_name                                                    simulator_error
runtime_error
                                                          entire              single
entire           single_sim
                                                        cos      euc        cos
euc        cos      euc        cos      euc
-------------------------------------------------------------------------------------
-----------------------------------------
[Input] images                                      1.00000 | 0.09873   1.00000 |
0.09873   1.00000 | 504.714   1.00000 | 0.70923
[Conv] /model.0/conv/Conv_output_0                  0.99998 | 4.97621   0.99998 |
4.97621   1.00000 | 5.45544   1.00000 | 1.46544
[exSwish] /model.0/act/Mul_output_0                 1.00000 | 3.92585   1.00000 |
1.27413   0.99998 | 7.01712   0.99998 | 5.80349
...
```

2.4.3 性能评估

通过 `rknn.eval_perf()` 评估模型在 NPU 上的性能，输出各算子的耗时、周期和带宽。

```
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
rknn.eval_perf()
```

执行后输出性能评估结果如下：

```
------------------------------------------------------------------------------------
--
Op Time Ranking Table (Core 0)
------------------------------------------------------------------------------------
--
OpType             Calls   CPU(us)      NPU(us)      Total(us)    Ratio(%)
------------------------------------------------------------------------------------
--
ConvExSwish        56      0            20385        20385          47.49%
Reshape            3       0            9245         9245           21.54%
InputProc          1       0            4293         4293           10.00%
Split              6       0            2174         2174            5.06%
Concat             21      0            1878         1878            4.37%
ConvSigmoid        3       0            905          905             2.11%
exYoloPostProcess  1       784          0            784             1.83%
Mul                10      0            572          572             1.33%
Add                13      0            562          562             1.31%
MaxPool            3       0            465          465             1.08%
ConvAdd            1       0            426          426             0.99%
exSwish            1       0            416          416             0.97%
Resize             2       0            412          412             0.96%
Conv               1       0            252          252             0.59%
Sub                3       0            80           80              0.19%
Pow                3       0            75           75              0.17%
InputOperator      1       0            3            3               0.01%
OutputProc         1       0            0            0               0.00%
OutputOperator     1       0            0            0               0.00%
------------------------------------------------------------------------------------
--
Total                      784          42143        42927         100.00%
------------------------------------------------------------------------------------
--
```

性能评估结果输出各算子的耗时排名表（Op Time Ranking Table），各字段说明如下：

字段	说明
OpType	算子类型
Calls	单帧内该算子运行次数
CPU(us)	单帧内该算子在 CPU 上的总耗时
NPU(us)	单帧内该算子在 NPU 上的总耗时
Total(us)	单帧内该算子的总耗时
Ratio(%)	单帧内该算子总耗时与单帧总耗时的比值

2.4.4 内存评估

通过 `rknn.eval_memory()` 获取模型的内存消耗评估结果，包括设备内存（Device Memory）和各核心内存分配（Per-Core Memory Allocation）：

```
rknn.init_runtime(target=platform, device_id='515e9b401c060c0b', core_mask=0xff)
memory_detail = rknn.eval_memory()
```

执行后输出内存评估结果如下：

```
========================== Memory Usage Information ==========================
Device Memory:
  System  :      18.98 MB total,      18.83 MB free,       0.16 MB used (  0.8%)
  Node 0  :     639.50 MB total,     590.57 MB free,      48.93 MB used (  7.7%)
  Node 1  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)
  Node 2  :     619.90 MB total,     619.81 MB free,       0.08 MB used (  0.0%)
  Node 3  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)
  Node 4  :     619.90 MB total,     619.81 MB free,       0.08 MB used (  0.0%)
  Node 5  :     619.90 MB total,     619.81 MB free,       0.08 MB used (  0.0%)
  Node 6  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)
  Node 7  :     639.50 MB total,     639.42 MB free,       0.08 MB used (  0.0%)

Per-Core Memory Allocation (MB):
  Core     Command      Weight       Internal     KVCache      Total
  -------- ------------ ------------ ------------ ------------ ------------
  0        0.22         15.42        33.20        0.00         48.84
  -------- ------------ ------------ ------------ ------------ ------------
  Total    0.22         15.42        33.20        0.00         48.84
================================================================================
```

各核心内存分配的字段说明如下：

字段	说明
Core	使用的核心
Command	寄存器配置及相关控制信息的内存占用（MB）
Weight	模型权重 Tensor 的内存占用（MB）
Internal	模型中间 Tensor 的内存占用（MB）
KVCache	模型 KVCache 的内存占用（MB）
Total	模型整体内存总占用（MB）

3. 板端部署

3.1 Runtime部署基础

本节阐述 RKNN3 Runtime 的基础概念，是进行板端部署（使用 C API 或 Toolkit Lite）的前提。内容涵盖模型与上下文生命周期、张量及数据排布格式、输入输出内存管理、同步与异步推理机制，以及错误处理与资源释放规范。相关机制同时适用于通用模型和 LLM / MLLM 模型。

3.1.1 模型和上下文生命周期

`rknn3_context` 是 RKNN3 Runtime 的核心句柄（handle），代表一个运行时上下文实例。所有模型加载、推理配置和资源管理操作均基于该句柄进行。板端部署时，应严格遵循“创建 → 使用 → 销毁”的生命周期管理规范，以避免资源泄漏或运行时异常。

通用模型（CNN / ViT 等）

不涉及会话层，直接通过上下文句柄调用 `rknn3_run()` 执行推理。上下文初始化完成后，可多次调用 `rknn3_run()`，句柄保持不变，直至上下文销毁。

LLM / MLLM 模型

在通用模型的基础上引入了**会话（Session）**层，用于管理文本生成任务的上下文状态。通过 `rknn3_session_init()` 创建会话并配置 LLM 相关参数（如词汇表、采样策略、上下文窗口长度等），再调用 `rknn3_session_run()` 执行推理。一个上下文可创建多个会话，各会话共享模型权重和内部内存，但 KVCache 与 LoRA 启用状态相互独立。多个会话不能并发执行，同一时刻仅允许一个会话运行。会话使用完毕后，需先调用 `rknn3_session_destroy()` 销毁各会话，最后再调用 `rknn3_destroy()` 销毁上下文。

> **多卡说明**：多卡流水线为每张 RK182X 卡创建独立的 context 和 session；不同卡上的 stage session 可以由主控侧并发运行，但每个 context/session 内部仍遵守“同一时刻一个 session 运行”的约束。多卡部署的 stage 间同步、张量转发和采样控制由应用负责，Runtime 不会自动把多个 context 组合成一个多卡模型。

3.1.2 张量和数据排列格式

RKNN3 模型的输入输出通过**张量（Tensor）**管理。本节介绍张量结构体系、数据类型与布局，以及 NPU 专用的 `NC1HWC2` 数据排列格式。

张量结构

通用模型和 LLM / MLLM 模型使用不同的张量结构体系：

通用模型（CNN / ViT 等）使用 `rknn3_tensor` 作为推理时的输入输出张量，它绑定两个子结构：

- **`rknn3_tensor`**：推理时使用的张量结构，包含 `mem`（指向 `rknn3_tensor_mem`）和 `attr`（指向 `rknn3_tensor_attr`）两个核心成员。用户在推理前需为每个输入输出张量分配内存并绑定属性，Runtime 通过该结构读取输入、写入输出。
- **`rknn3_tensor_attr`**：张量属性结构，通过 `rknn3_query()` 查询获取，包含张量的名称、shape、数据类型（dtype）、数据布局（layout）、量化信息（qnt_type/qnt_info）、核心 ID 等。板端部署时需先查询属性，再据此分配内存和组织数据。
- **`rknn3_tensor_mem`**：张量数据内存结构，通过 `rknn3_create_mem()` 等接口分配，代表张量在设备上的实际数据存储。用户需通过 `rknn3_mem_sync()` 在 CPU 与设备之间同步数据。

LLM / MLLM 模型使用 `rknn3_llm_input` 作为 Session 推理时的输入，它通过 `input_type` 字段区分不同的输入类型，并使用对应的子结构：

- **`rknn3_llm_input`**：LLM 推理输入结构，包含 `input_type`（输入类型）和对应类型的输入数据。主要成员：
- `role`：消息角色，如 `"user"`（用户输入）、`"tool"`（函数调用结果）。
- `input_type`：输入类型，可选 `RKNN3_LLM_INPUT_PROMPT`（文本）、`RKNN3_LLM_INPUT_TOKEN`（token id）、`RKNN3_LLM_INPUT_EMBED`（嵌入向量）、`RKNN3_LLM_INPUT_MULTIMODAL`（多模态）、`RKNN3_LLM_INPUT_AUX`（辅助输入）。
- `llm_input`：当输入类型为 PROMPT/TOKEN/EMBED 时，为 `rknn3_llm_tensor` 结构。
- `multimodal_input`：当输入类型为 MULTIMODAL 时，为 `rknn3_llm_multimodal_tensor` 结构（成员见下文）。
- `aux_input`：当输入类型为 AUX 时，为辅助输入 tensor（目前 `rknn3_aux_tensor` 与 `rknn3_tensor` 同类型）。
- **`rknn3_llm_tensor`**：`rknn3_llm_input` 的子结构，表示 LLM 的输入内容。主要成员：
- `name`：张量名称。
- `prompt`：文本提示输入，当输入类型为 `RKNN3_LLM_INPUT_PROMPT` 时使用。
- `embed`：嵌入向量指针，当输入类型为 `RKNN3_LLM_INPUT_EMBED` 时使用（大小为 `n_tokens * hidden_size`）。
- `tokens`：token id 数组，当输入类型为 `RKNN3_LLM_INPUT_TOKEN` 时使用。
- `n_tokens`：token 数量。
- `enable_thinking`：是否为本次输入启用思考模式。
- **`rknn3_llm_multimodal_tensor`**：`rknn3_llm_input` 的子结构，当输入类型为 `RKNN3_LLM_INPUT_MULTIMODAL` 时使用，表示多模态输入（文本、图像、音频、视频）。顶层成员包含 `name`、`prompt` / `tokens`（文本输入，二者二选一，若同时提供默认按 prompt 推理）、`n_tokens`、`enable_thinking`，以及三个模态子结构：
- **`image` 子结构**：图像输入。`image_embed`（FP16 嵌入指针，大小为 `n_image × n_image_tokens × embedding_dim`）、`n_image_tokens`（每张图的 token 数）、`n_image`（图像数量）、`image_start` / `image_end` / `image_content`（图像在文本中的起始/结束/占位标签）、`image_width` / `image_height`（图像宽高）。
- **`audio` 子结构**：音频输入。`audio_embed`（FP16 嵌入指针，大小为 `n_audio × n_audio_tokens × embedding_dim`）、`n_audio_tokens`（每段音频 token 数）、`n_audio`（音频数量）、`audio_start` / `audio_end` / `audio_content`（音频标签）。
- **`video` 子结构**：视频输入。`video_embed`（FP16 嵌入指针，大小为 `n_video × n_frame_per_video × n_frame_tokens × embedding_dim`）、`n_frame_tokens`（每帧 token 数）、`n_frame_per_video`（每个视频帧数）、`n_video`（视频数量）、`video_start` / `video_end` / `video_content`（视频标签）。

各模态的 `*_embed` 由对应的 vision/audio/video 编码模型推理输出，通过该结构传入 LLM；`*_start` / `*_end` / `*_content` 标签须与模型 tokenizer 的特殊 token 一致。具体填充与使用方式见 3.3.6 多模态输入。

数据类型与布局

张量的数据类型（dtype）和数据布局（layout）可通过 `rknn3_tensor_attr` 的 `dtype` 和 `layout` 字段查询。

- **数据类型（dtype）**：包括 FP32 / FP16 / INT8 / UINT8 等，由模型转换时的量化配置决定。非量化模型权重和激活均为 FP16，量化模型权重可为 INT8 等。
- **数据布局（layout）**：主要有 `NCHW`、`NHWC` 和 `NC1HWC2` 三种。`NCHW` 和 `NHWC` 是深度学习常见的数据排列方式，用户输入通常使用这两种格式，Runtime 会在内部处理转换。`NC1HWC2` 是 NPU 硬件优化的内存布局格式，若需直接操作 NPU 内存（如零拷贝场景），则需按 `NC1HWC2` 布局组织数据。

NC1HWC2 数据排列格式

`NC1HWC2` 将通道维度分块存储，其中 C1 为通道分块数（C/C2 向上取整），C2 为子通道数，由平台和数据类型决定：

	RK1820/RK3572
INT8	32
FP16	16

`NC1HWC2` 的数据排布与存储如下图所示，数字 0 代表一笔数据，即一次存放 C2 个数据，存放顺序与图中数值增长顺序一致（先存 0-15，再存 16-31）。下图以 C2 为 8 为例（示意排列原理，当前平台 FP16 的 C2=16、INT8 的 C2=32，排布方式相同），当 feature 为 (1, 13, 4, 4) 时，对应的 `NC1HWC2` 为 (1, 2, 4, 4, 8)，每个 C2 数据块只有前 5 个数据有效，其余为对齐填充数据。

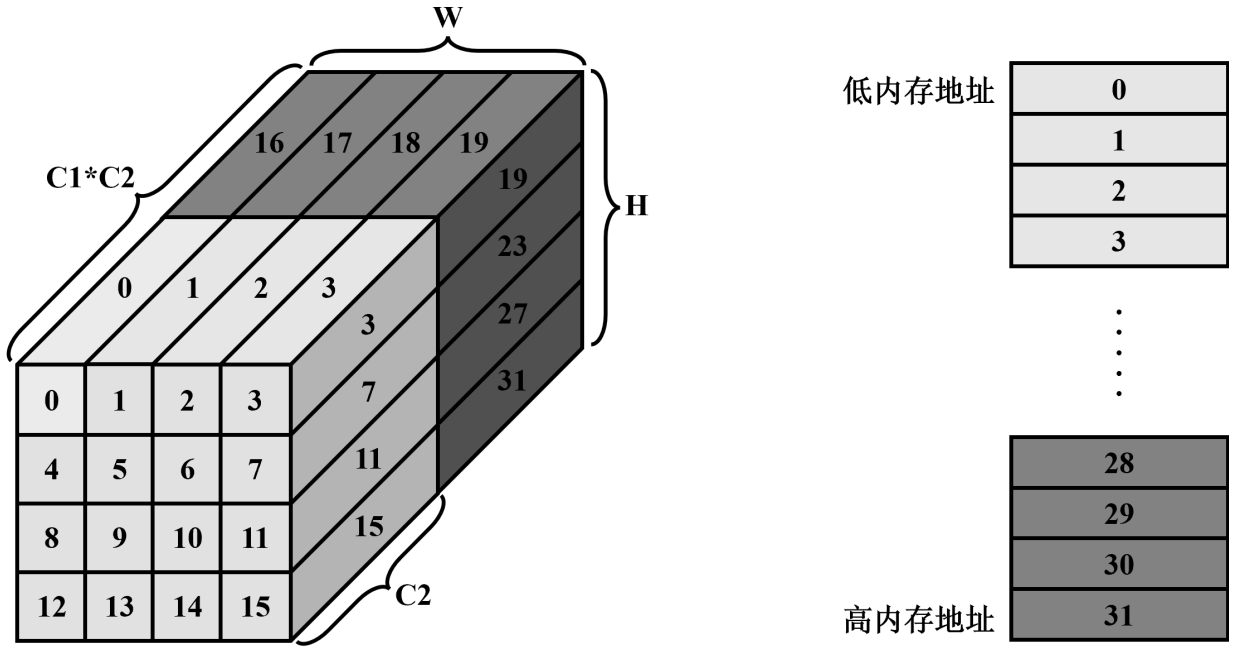
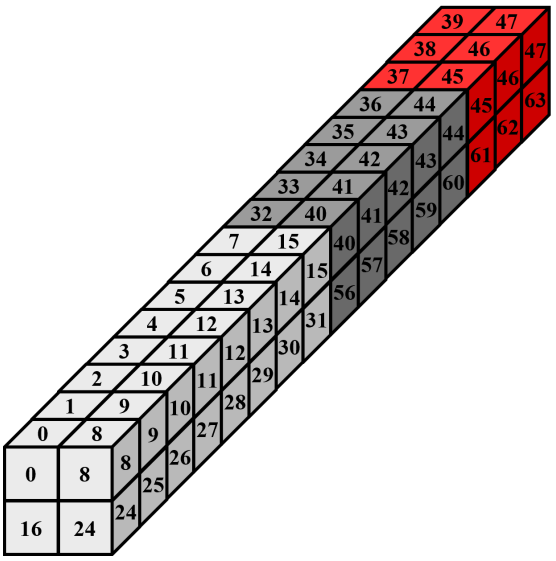


图3-1 RKNPU NC1HWC2 数据排布与存储

以 feature (1, 13, 2, 2)、C2 为 8 为例，`NC1HWC2` 为 (1, 2, 2, 2, 8)，其存储展开如下，红色部分为对齐填充的无效数据：



低地址								
	0	1	2	3	4	5	6	7
	8	9	10	11	12	13	14	15
	16	17	18	19	20	21	22	23
	24	25	26	27	28	29	30	31
	32	33	34	35	36	37	38	39
	40	41	42	43	44	45	46	47
	48	49	50	51	52	53	54	55
高地址	56	57	58	59	60	61	62	63

图3-2 NC1HWC2 数据排布展开

移除无效数据后转为 `NCHW` (1, 13, 2, 2):

低地址				
	0	8	16	24
	1	9	17	25
	2	10	18	26
	3	11	19	27
	4	12	20	28
	5	13	21	29
	6	14	22	30
	7	15	23	31
	32	40	48	56
	33	41	49	57
	34	42	50	58
	35	43	51	59
高地址	36	44	52	60

图3-3 NCHW 数据排布

移除无效数据后转为 `NHWC` (1, 2, 2, 13):

低地址	0	1	2	3	4	5	6	7	32	33	34	35	36
	8	9	10	11	12	13	14	15	40	41	42	43	44
	16	17	18	19	20	21	22	23	48	49	50	51	52
高地址	24	25	26	27	28	29	30	31	56	57	58	59	60

图3-4 NHWC 数据排布

布局转换

当需要直接操作 NPU 内存（如零拷贝场景）时，需在 `NC1HWC2` 与 `NCHW` / `NHWC` 之间转换数据。以下为常用转换示例：

`NC1HWC2` 转 `NCHW`（INT8）：

```
#define ALIGNED_SIZE(x, bytes) (((x) + ((bytes) - 1)) & (~((bytes) - 1)))
int NC1HWC2_to_NCHW(const int8_t* src, int8_t* dst, int* dims, int channel)
{
  int batch        = dims[0];
  int height       = dims[2];
  int width        = dims[3];
  int cn           = dims[4];
  int align_surf   = width * height == 1 ? 1 : ALIGNED_SIZE(width * height, 4);

  int src_batch_elems = dims[1] * align_surf * dims[4];
  int dst_batch_elems = channel * width * height;

  for (int i = 0; i < batch; ++i) {
    int idx = 0;
    for (int c = 0; c < channel; ++c) {
      int           plane  = c / cn;
      const int8_t* src_c = plane * align_surf * cn + src;
      int           offset = c % cn;

      for (int cur_h = 0; cur_h < height; ++cur_h) {
        for (int cur_w = 0; cur_w < width; ++cur_w) {
          int cur_hw = cur_h * width + cur_w;
          dst[idx++] = src_c[cn * cur_hw + offset];
        }
      }
    }

    src += src_batch_elems;
    dst += dst_batch_elems;
  }
  return 0;
}
```

`NCHW` 转 `NC1HWC2`（INT8）：

```c
#define ALIGNED_SIZE(x, bytes) (((x) + ((bytes) - 1)) & (~((bytes) - 1)))
int NCHW_to_NC1HWC2(const int8_t* src, int8_t* dst, int* dims, int channel)
{
  int batch        = dims[0];
  int height       = dims[2];
  int width        = dims[3];
  int cn           = dims[4];
  int align_surf   = width * height == 1 ? 1 : ALIGNED_SIZE(width * height, 4);
  int hw = height * width;

  int src_batch_elems = channel * width * height;
  int dst_batch_elems = dims[1] * align_surf * cn;

  for (int i = 0; i < batch; ++i) {
    for (int c = 0; c < channel; ++c) {
      int plane     = c / cn;
      int offset    = c % cn;
      int8_t* dst_c = dst + (size_t)plane * align_surf * cn;

      for (int cur_h = 0; cur_h < height; ++cur_h) {
        for (int cur_w = 0; cur_w < width; ++cur_w) {
          int cur_hw                  = cur_h * width + cur_w;
          dst_c[cn * cur_hw + offset] = src[c * hw + cur_h * width + cur_w];
        }
      }
    }

    src += src_batch_elems;
    dst += dst_batch_elems;
  }

  return 0;
}
```

> **说明**：完整的转换代码（含 `NC1HWC2` 转 `NHWC` 等）可参考 rknn3-model-zoo 中的工具函数。

3.1.3 输入输出内存

RKNN3 Runtime 中，模型的输入输出数据通过 `rknn3_tensor_mem` 内存对象管理。板端部署时，需为每个输入和输出张量分配内存，绑定到张量属性，在 CPU 与 NPU 之间同步数据，最后释放内存。本节介绍内存的分配方式、同步机制与释放规范。

内存分配方式

RKNN3 提供多种内存分配方式，适用于不同场景：

- **`rknn3_create_mem()`**：由 Runtime 分配指定大小的设备内存，是最常用的分配方式。分配时需指定内存大小、核心 ID（多核场景下指定内存所属核心）和内存分配标志（`rknn3_mem_alloc_flags`）。标志控制内存的 cache 属性：`RKNN3_FLAG_MEMORY_CACHEABLE`（默认，创建 cacheable 内存，性能更优但需配合 `rknn3_mem_sync()` 同步）和 `RKNN3_FLAG_MEMORY_NON_CACHEABLE`（创建 non-cacheable 内存，无需同步但访问性能较低）。分配后的内存对象通过 `rknn3_tensor.mem` 字段绑定到张量，Runtime 推理时通过该绑定关系读取输入、写入输出。适用于大多数常规部署场景。
- **`rknn3_create_mem_from_fd()`**：从已有的文件描述符（fd）创建内存对象，复用外部已分配的内存（如 DMA-BUF）。适用于需要与其他硬件模块（如摄像头、显示）共享内存的零拷贝场景，可避免数据拷

贝开销，但要求外部内存的 fd 有效且大小匹配。

- **`rknn3_create_mem_from_phys()`**：从物理地址创建内存对象（当前版本暂未实现）。设计用于直接操作物理内存的高级场景。

选择建议：常规部署优先使用 `rknn3_create_mem()`，简单可靠；若需与摄像头、RGA 等模块零拷贝对接，使用 `rknn3_create_mem_from_fd()`。

分配示例：

```
// 为输入张量分配 cacheable 内存
inputs[i].mem = rknn3_create_mem(ctx, input_attrs[i].aligned_size,
                                 input_attrs[i].core_id,
                                 RKNN3_FLAG_MEMORY_CACHEABLE);
```

内存同步

当内存以 `RKNN3_FLAG_MEMORY_CACHEABLE`（或默认的 `RKNN3_FLAG_MEMORY_FLAGS_DEFAULT`，两者行为相同）标志分配时，CPU 和设备各自维护 cache，直接写入 CPU 端内存后设备未必能立即看到数据，反之亦然。因此推理前后需通过 `rknn3_mem_sync()` 显式同步 CPU 和设备间的内存数据，确保数据一致性。若内存以 `RKNN3_FLAG_MEMORY_NON_CACHEABLE` 标志分配，则无需同步，但访问性能较低。

同步方向由 `rknn3_mem_sync_mode` 指定：

- **`RKNN3_MEMORY_SYNC_TO_DEVICE`**：将 CPU 端数据同步到设备。用于输入数据准备完成后、推理执行前，确保设备读取到最新的输入数据。
- **`RKNN3_MEMORY_SYNC_FROM_DEVICE`**：将设备端数据同步到 CPU。用于推理完成后、读取输出结果前，确保 CPU 读取到设备写入的最终结果。
- **`RKNN3_MEMORY_SYNC_BIDIRECTIONAL`**：双向同步，适用于 CPU 与设备双向读写的场景。

同步示例：

```
// 推理前：输入数据同步到设备
for (int i = 0; i < n_inputs; i++) {
    rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
}

// 推理后：输出数据同步到 CPU
for (int i = 0; i < n_outputs; i++) {
    rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
}
```

注意事项：

1. 同步是阻塞操作，调用后会等待数据传输完成。输入同步应在所有输入数据写入完成后统一执行，输出同步应在推理完成后执行。
2. 每个输入张量需单独同步到设备，每个输出张量需单独同步回 CPU。
3. 若 cacheable 内存跳过同步直接读取输出，可能读到未更新的 cache 数据，导致结果错误。

内存释放

内存使用完毕后，通过 `rknn3_destroy_mem()` 释放。释放需遵循以下生命周期规范：

1. 内存释放必须在上下文销毁（`rknn3_destroy()`）之前完成，否则可能导致资源泄漏。
2. 释放顺序应先释放输入输出内存，再销毁上下文。
3. 已释放的内存对象不可再使用，否则会导致未定义行为。

释放示例：

```
// 先释放输入输出内存
for (int i = 0; i < n_inputs; i++) {
    rknn3_destroy_mem(ctx, inputs[i].mem);
}
for (int i = 0; i < n_outputs; i++) {
    rknn3_destroy_mem(ctx, outputs[i].mem);
}
// 再销毁上下文
rknn3_destroy(ctx);
```

3.1.4 同步与异步推理

RKNN3 Runtime 提供同步和异步两种推理执行方式：

- **同步推理**（`rknn3_run()`）：调用后阻塞当前线程，直到推理完成后返回。使用简单，适用于大多数场景。调用前需确保输入数据已同步到设备（`RKNN3_MEMORY_SYNC_TO_DEVICE`），推理完成后需将输出数据同步回 CPU（`RKNN3_MEMORY_SYNC_FROM_DEVICE`）。

```
// 1. 输入数据同步到设备
for (int i = 0; i < n_inputs; i++) {
    rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
}

// 2. 执行同步推理（阻塞直到完成）
int ret = rknn3_run(ctx, inputs, n_inputs, outputs, n_outputs);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_run failed! ret=%d\n", ret);
    return -1;
}

// 3. 输出数据同步到 CPU
for (int i = 0; i < n_outputs; i++) {
    rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
}
```

- **异步推理**（`rknn3_run_async()` + `rknn3_wait()`）：先通过 `rknn3_run_async()` 发起非阻塞推理，主线程可继续执行其他任务，再通过 `rknn3_wait()` 等待推理完成。适用于需要在推理期间并行处理其他工作（如数据预处理）的场景，可提升整体吞吐。**注意：`rknn3_run_async()` 和 `rknn3_wait()` 当前版本暂未实现。**

选择建议：当前版本仅支持同步推理（`rknn3_run()`），异步推理接口暂未实现。后续版本支持后，若推理期间有并行任务需求可使用异步方式。

3.1.5 错误处理和资源释放

错误处理

RKNN3 C API 的接口返回值为 `int` 类型状态码，`RKNN3_SUCCESS`（0）表示成功，负值表示错误。板端部署时应对每个接口调用检查返回值，出错时不应继续执行后续操作，需跳转到资源清理流程后退出，避免在异常状态下执行导致未定义行为。

常见状态码及含义：

状态码	值	描述
`RKNN3_SUCCESS`	0	执行成功
`RKNN3_ERR_FAIL`	-1	执行失败
`RKNN3_ERR_ARGUMENT_INVALID`	-2	参数无效
`RKNN3_ERR_MODEL_INVALID`	-3	模型无效
`RKNN3_ERR_CTX_INVALID`	-4	上下文无效
`RKNN3_ERR_OUT_OF_MEMORY`	-6	内存不足
`RKNN3_ERR_TIMEOUT`	-7	执行超时
`RKNN3_ERR_DEVICE_UNAVAILABLE`	-10	设备不可用
`RKNN3_ERR_COMMUNICATION`	-13	通信错误
`RKNN3_ERR_INCOMPATIBLE_VERSION`	-18	组件版本不兼容

完整的错误码列表见《RKNN3_Runtime_C_API_参考》。

初始化阶段的错误处理示例（出错时跳转到清理流程）：

```
int ret = rknn3_init(&ctx, NULL);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_init failed! ret=%d\n", ret);
    goto out;
}
```

资源释放

板端部署中的各类资源需按正确顺序释放，避免资源泄漏。推荐使用 `goto` 统一跳转到清理流程，确保正常路径和错误路径都能正确释放资源。

- **通用模型**：释放顺序为释放输入输出内存 -> 销毁上下文。

```
out:
    // 释放输入输出内存
    for (int i = 0; i < n_inputs; i++) {
        if (inputs[i].mem) rknn3_destroy_mem(ctx, inputs[i].mem);
    }
    for (int i = 0; i < n_outputs; i++) {
        if (outputs[i].mem) rknn3_destroy_mem(ctx, outputs[i].mem);
    }
    // 销毁上下文
    if (ctx) rknn3_destroy(ctx);
```

- **LLM / MLLM 模型**：释放顺序为销毁会话 -> 释放输入输出内存 -> 销毁上下文。

```
out:
    // 销毁会话
    if (session) rknn3_session_destroy(session);
    // 释放输入输出内存
    for (int i = 0; i < n_inputs; i++) {
        if (inputs[i].mem) rknn3_destroy_mem(ctx, inputs[i].mem);
    }
    for (int i = 0; i < n_outputs; i++) {
        if (outputs[i].mem) rknn3_destroy_mem(ctx, outputs[i].mem);
    }
    // 销毁上下文
    if (ctx) rknn3_destroy(ctx);
```

> **说明**：上下文销毁后，基于该上下文创建的所有资源（内存、会话等）均不可再访问。确保所有资源在上下文销毁前已释放完毕。

3.2 通用模型C API部署

本节介绍如何使用 RKNN3 Runtime C API 部署通用模型（CNN / ViT 等），涵盖从模型初始化到推理执行再到资源释放的完整流程。适用于生产部署、性能敏感场景。

3.2.1 完整调用流程

通用模型 C API 部署的典型调用顺序为：初始化上下文 -> 加载模型 -> 模型初始化 -> 查询属性 -> 创建内存 -> 准备输入 -> 推理执行 -> 处理输出 -> 释放资源。整体流程如下图：

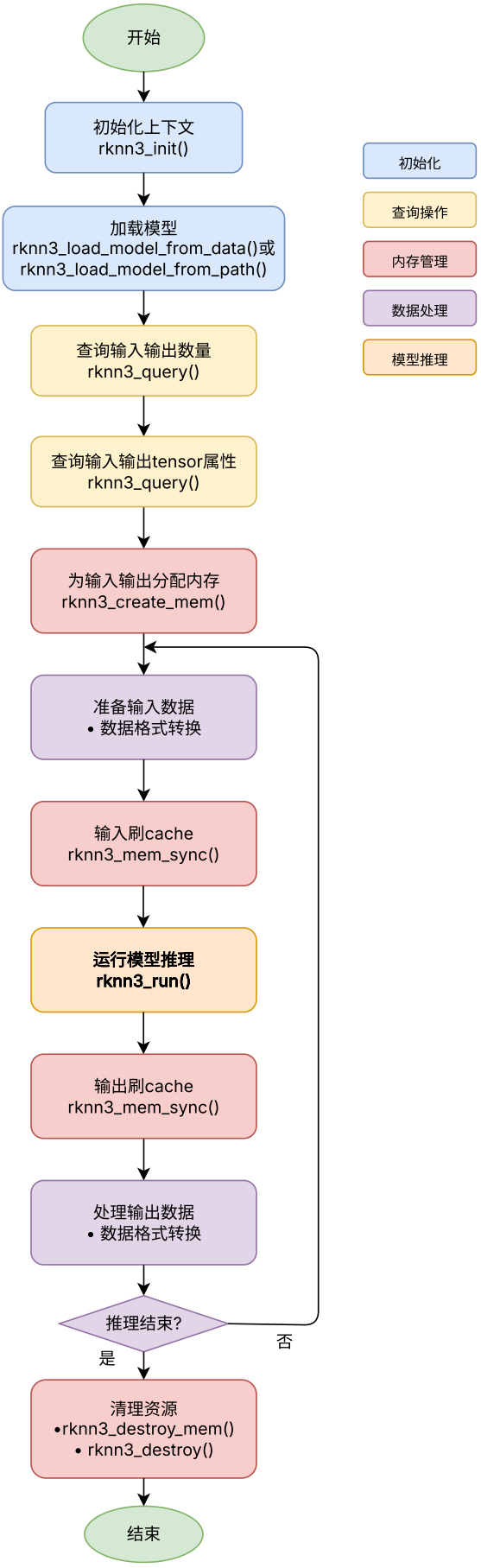


图3-5 RKNN3 C API推理流程

后续小节将按此流程逐步介绍各环节的操作方式与注意事项。

3.2.2 模型初始化

模型初始化包含三个步骤：创建上下文、加载模型、配置运行参数。

1. 创建上下文

通过 `rknn3_init()` 创建运行时上下文，是所有后续操作的前提。多设备场景下可通过 `init_extend` 指定设备 ID，单设备场景传入 `NULL` 即可：

```
rknn3_context ctx;
int ret = rknn3_init(&ctx, NULL);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_init failed! ret=%d\n", ret);
    goto out;
}
```

2. 加载模型

通过 `rknn3_load_model_from_path()` 从文件加载 `.rknn` 模型和 `.weight` 权重，或通过 `rknn3_load_model_from_data()` 从内存加载（适用于模型已在内存中的场景，如加密模型解密后加载）：

```
// 从文件加载
ret = rknn3_load_model_from_path(ctx, model_path, weight_path);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_load_model_from_path failed! ret=%d\n", ret);
    goto out;
}
```

> **说明**：`weight_path` 传 `NULL` 可进入流式权重加载模式，需在 `rknn3_model_init()` 后通过 `rknn3_load_weight_chunk()` 分段上传权重，适用于内存受限场景。

3. 配置运行参数

通过 `rknn3_model_init()` 配置模型运行参数并完成初始化。`rknn3_config` 结构体包含核心掩码（`run_core_mask`）、运行优先级（`priority`）、超时时间（`run_timeout`）等配置。核心掩码控制模型运行在哪些 NPU 核心上，需与模型转换时配置的 `core_mask` 一致，否则会报错：

```
rknn3_config config;
memset(&config, 0, sizeof(rknn3_config));
config.run_core_mask = 0xff;  // 使用所有 NPU 核心
ret = rknn3_model_init(ctx, &config);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_model_init failed! ret=%d\n", ret);
    goto out;
}
```

3.2.3 输入输出准备

模型初始化完成后，需查询模型输入输出属性，据此分配内存并准备输入数据。

1. 查询输入输出属性

通过 `rknn3_query()` 查询模型的输入输出张量个数和属性。首先查询个数（`RKNN3_QUERY_IN_OUT_NUM`），再逐个查询每个张量的属性（`RKNN3_QUERY_INPUT_ATTR` / `RKNN3_QUERY_OUTPUT_ATTR`），获取 shape / dtype / layout、量化参数、`aligned_size`（对齐后内存大小）和 `core_id`（所属核心）等信息：

```
// 查询输入输出个数
rknn3_input_output_num io_num;
ret = rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
if (ret != RKNN3_SUCCESS) {
    printf("query io_num failed! ret=%d\n", ret);
    goto out;
}

// 查询各输入张量属性
rknn3_tensor_attr input_attrs[io_num.n_input];
for (uint32_t i = 0; i < io_num.n_input; i++) {
    input_attrs[i].index = i;
    ret = rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, &input_attrs[i],
sizeof(rknn3_tensor_attr));
    if (ret != RKNN3_SUCCESS) {
        printf("query input_attr %d failed! ret=%d\n", i, ret);
        goto out;
    }
}

// 查询各输出张量属性
rknn3_tensor_attr output_attrs[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_output; i++) {
    output_attrs[i].index = i;
    ret = rknn3_query(ctx, RKNN3_QUERY_OUTPUT_ATTR, &output_attrs[i],
sizeof(rknn3_tensor_attr));
    if (ret != RKNN3_SUCCESS) {
        printf("query output_attr %d failed! ret=%d\n", i, ret);
        goto out;
    }
}
```

2. 分配输入输出内存

根据查询到的 `aligned_size` 和 `core_id`，通过 `rknn3_create_mem()` 为每个输入输出张量分配内存，并绑定到 `rknn3_tensor` 的 `mem` 字段：

```
rknn3_tensor inputs[io_num.n_input];

// 为输入张量创建内存
for (uint32_t i = 0; i < io_num.n_input; i++) {
    inputs[i].mem = rknn3_create_mem(ctx, input_attrs[i].aligned_size,
                                     input_attrs[i].core_id,
                                     RKNN3_FLAG_MEMORY_CACHEABLE);
    if (inputs[i].mem == NULL) {
        printf("create input mem %d failed!\n", i);
        goto out;
    }
}

// 为输出张量创建内存
rknn3_tensor outputs[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_output; i++) {
    outputs[i].mem = rknn3_create_mem(ctx, output_attrs[i].aligned_size,
                                      output_attrs[i].core_id,
                                      RKNN3_FLAG_MEMORY_CACHEABLE);
    if (outputs[i].mem == NULL) {
        printf("create output mem %d failed!\n", i);
        goto out;
    }
}
```

3. 准备输入数据

推理前，需将经过前处理（如 resize、颜色空间转换等）后的输入数据写入对应的输入张量内存（`inputs[i].mem`）。写入完成后，调用 `rknn3_mem_sync()` 将数据同步至 NPU 设备端。

> **说明**：
>
> - **数据属性一致性**：输入数据的 `dtype`（数据类型）与 `shape`（形状）必须与查询到的张量属性严格匹配，否则会导致推理异常。
> - **均值归一化处理**：若模型转换时已在 `rknn.config()` 中配置了 `mean_values` 和 `std_values`，则 C API 推理阶段**无需手动进行均值归一化**，可有效减少部署阶段的耗时。
> - **数据布局要求**：输入数据的布局（layout）需与张量属性中的 `layout` 字段保持一致。RKNN3 Runtime 内部支持常见的 `NHWC` 与 `NCHW` 布局，并在推理过程中自动处理必要的布局转换。

```
// 将预处理数据写入输入内存
// memcpy(inputs[0].mem->virt_addr, input_data, input_attrs[0].aligned_size);

// 输入数据同步到设备
for (uint32_t i = 0; i < io_num.n_input; i++) {
    ret = rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
    if (ret != RKNN3_SUCCESS) {
        printf("mem_sync input %d failed! ret=%d\n", i, ret);
        goto out;
    }
}
```

3.2.4 推理执行

输入数据同步到设备后，通过 `rknn3_run()` 执行同步推理，传入上下文、输入张量数组及数量、输出张量数组及数量。该接口会阻塞直到推理完成：

```
ret = rknn3_run(ctx, inputs, io_num.n_input, outputs, io_num.n_output);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_run failed! ret=%d\n", ret);
    goto out;
}
```

推理完成后，输出数据已写入输出张量的内存，但仍在设备侧，需通过 `rknn3_mem_sync()` 同步回 CPU 后才能读取（见 3.2.5 输出处理）。

3.2.5 输出处理

推理完成后，输出数据在设备侧，需先同步回 CPU，再根据输出张量属性读取和处理结果。

1. 同步输出数据

逐个将输出张量的内存从设备同步回 CPU：

```
for (uint32_t i = 0; i < io_num.n_output; i++) {
    ret = rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
    if (ret != RKNN3_SUCCESS) {
        printf("mem_sync output %d failed! ret=%d\n", i, ret);
        goto out;
    }
}
```

2. 读取输出数据

同步完成后，通过 `outputs[i].mem` 的虚拟地址读取输出数据。读取时需注意：

- **数据类型**：根据 `output_attrs[i].dtype` 确定读取的数据类型（如 FP16 / INT8 等）。
- **数据布局**：输出布局可能为 `NC1HWC2`，需转换为 `NCHW` 或 `NHWC` 后再进行后处理（转换方法见 3.1.2）。
- **量化参数**：若输出为量化类型（INT8 等），需使用 `output_attrs[i].qnt_info` 中的 `scale` 和 `zero_point` 进行反量化，还原为浮点值。

```
// 读取输出数据（以 FP16 为例）
float16* output_data = (float16*)outputs[0].mem->virt_addr;
// 根据 output_attrs[0].shape 和 layout 进行后处理
// ...
```

3. 后处理

后处理取决于具体模型和应用场景，例如目标检测模型的 NMS、分类模型的 argmax 等，可参考 rknn3-model-zoo 中对应模型的示例代码。

3.2.6 完整示例

以下是一个通用模型 C API 推理的完整示例，整合了模型初始化、输入输出准备、推理执行、输出处理和资源释放各步骤：

```c
#include "rknn3_api.h"

int main() {
    int ret;
    rknn3_context ctx = NULL;

    // 1. 初始化上下文
    ret = rknn3_init(&ctx, NULL);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_init failed! ret=%d\n", ret);
        goto out;
    }

    // 2. 加载模型
    ret = rknn3_load_model_from_path(ctx, "model.rknn", "model.weight");
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_load_model_from_path failed! ret=%d\n", ret);
        goto out;
    }

    // 3. 模型初始化
    rknn3_config config = {0};
    config.run_core_mask = 0xff;
    ret = rknn3_model_init(ctx, &config);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_model_init failed! ret=%d\n", ret);
        goto out;
    }

    // 4. 查询输入输出个数
    rknn3_input_output_num io_num;
    ret = rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
    if (ret != RKNN3_SUCCESS) {
        printf("query io_num failed! ret=%d\n", ret);
        goto out;
    }

    // 5. 查询属性并分配内存
    rknn3_tensor_attr input_attrs[io_num.n_input];
    rknn3_tensor_attr output_attrs[io_num.n_output];
    rknn3_tensor inputs[io_num.n_input];
    rknn3_tensor outputs[io_num.n_output];

    for (uint32_t i = 0; i < io_num.n_input; i++) {
        input_attrs[i].index = i;
        ret = rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, &input_attrs[i],
sizeof(rknn3_tensor_attr));
        if (ret != RKNN3_SUCCESS) {
            printf("query input_attr %d failed! ret=%d\n", i, ret);
            goto out;
        }
        inputs[i].mem = rknn3_create_mem(ctx, input_attrs[i].aligned_size,
                                         input_attrs[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
        if (inputs[i].mem == NULL) {
            printf("create input mem %d failed!\n", i);
            goto out;
        }
    }
    for (uint32_t i = 0; i < io_num.n_output; i++) {
        output_attrs[i].index = i;
```

```
        ret = rknn3_query(ctx, RKNN3_QUERY_OUTPUT_ATTR, &output_attrs[i],
sizeof(rknn3_tensor_attr));
        if (ret != RKNN3_SUCCESS) {
            printf("query output_attr %d failed! ret=%d\n", i, ret);
            goto out;
        }
        outputs[i].mem = rknn3_create_mem(ctx, output_attrs[i].aligned_size,
                                          output_attrs[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
        if (outputs[i].mem == NULL) {
            printf("create output mem %d failed!\n", i);
            goto out;
        }
    }

    // 6. 准备输入数据并同步到设备
    // ... 填充 inputs[0].mem->virt_addr ...
    for (uint32_t i = 0; i < io_num.n_input; i++) {
        ret = rknn3_mem_sync(ctx, inputs[i].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
        if (ret != RKNN3_SUCCESS) {
            printf("mem_sync input %d failed! ret=%d\n", i, ret);
            goto out;
        }
    }

    // 7. 执行推理
    ret = rknn3_run(ctx, inputs, io_num.n_input, outputs, io_num.n_output);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_run failed! ret=%d\n", ret);
        goto out;
    }

    // 8. 同步输出并处理
    for (uint32_t i = 0; i < io_num.n_output; i++) {
        ret = rknn3_mem_sync(ctx, outputs[i].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
        if (ret != RKNN3_SUCCESS) {
            printf("mem_sync output %d failed! ret=%d\n", i, ret);
            goto out;
        }
    }
    // ... 读取 outputs[i].mem->virt_addr 并进行后处理 ...

out:
    // 9. 释放资源
    for (uint32_t i = 0; i < io_num.n_input; i++) {
        if (inputs[i].mem) rknn3_destroy_mem(ctx, inputs[i].mem);
    }
    for (uint32_t i = 0; i < io_num.n_output; i++) {
        if (outputs[i].mem) rknn3_destroy_mem(ctx, outputs[i].mem);
    }
    if (ctx) rknn3_destroy(ctx);
    return 0;
}
```

> **说明**：更多模型的完整示例代码可参考 rknn3-model-zoo（如 `examples/yolov6/cpp/`）。

3.2.7 调试方法

板端部署过程中若遇到精度或性能问题，可使用以下调试工具辅助排查：

导出中间特征

通过 `rknn3_dump_features()` 逐层导出模型中间特征到指定目录，保存为 `.npy` 文件，便于与浮点模型逐层对比分析精度问题。导出时可指定核心掩码（`core_mask`）和导出范围（`dump_selector`，如 `"*"` 或 `"all"` 表示导出所有算子）：

```
ret = rknn3_dump_features(ctx, inputs, n_inputs, NULL, 0,
                          "./dump/", 0, "all");
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_dump_features failed! ret=%d\n", ret);
}
```

逐层算子性能分析

通过 `rknn3_profile_ops()` 打印逐层算子的性能信息，包含耗时、Cycle 和带宽等统计，并输出汇总表。`log_level` 控制输出详细程度：0 仅打印耗时汇总，1 增加逐算子耗时表，2 进一步增加 Cycle 和带宽信息：

```
ret = rknn3_profile_ops(ctx, inputs, n_inputs, NULL, 0, 2);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_profile_ops failed! ret=%d\n", ret);
}
```

常见问题排查

1. **内存对齐**：分配内存时使用 `aligned_size` 而非 `n_elems * sizeof(dtype)`，否则可能导致内存越界。
2. **核心 ID**：创建内存时需使用 `attr->core_id`，确保内存分配在正确的核心上。
3. **缓存同步**：使用 cacheable 内存时必须正确调用 `rknn3_mem_sync()`，否则可能导致数据不一致。
4. **布局转换**：`NC1HWC2` 布局需特殊处理，注意 FP16/INT8 的子通道数不同（见 3.1.2）。
5. **量化参数**：INT8 模型需使用 `qnt_info.scale` 和 `zero_point` 进行量化/反量化。
6. **core_mask 一致性**：运行时的 `core_mask` 需与模型转换时配置的一致，否则会报错。

3.3 LLM和MLLM Session部署

本节介绍如何使用 RKNN3 Runtime 的 Session API 部署 LLM / MLLM 模型，涵盖会话生命周期、输入与回调、推理执行、Chat Template、多模态输入等内容。Session API 适用于需要会话管理、KVCache、采样控制等高级功能的 LLM / MLLM 部署场景。

3.3.1 完整调用流程

LLM / MLLM Session 部署的典型调用顺序为：初始化上下文 -> 加载模型 -> 模型初始化 -> Session初始化 -> 设置回调 -> 准备输入 -> 执行推理 -> 处理结果 -> 销毁会话 -> 销毁上下文。整体流程如下图：

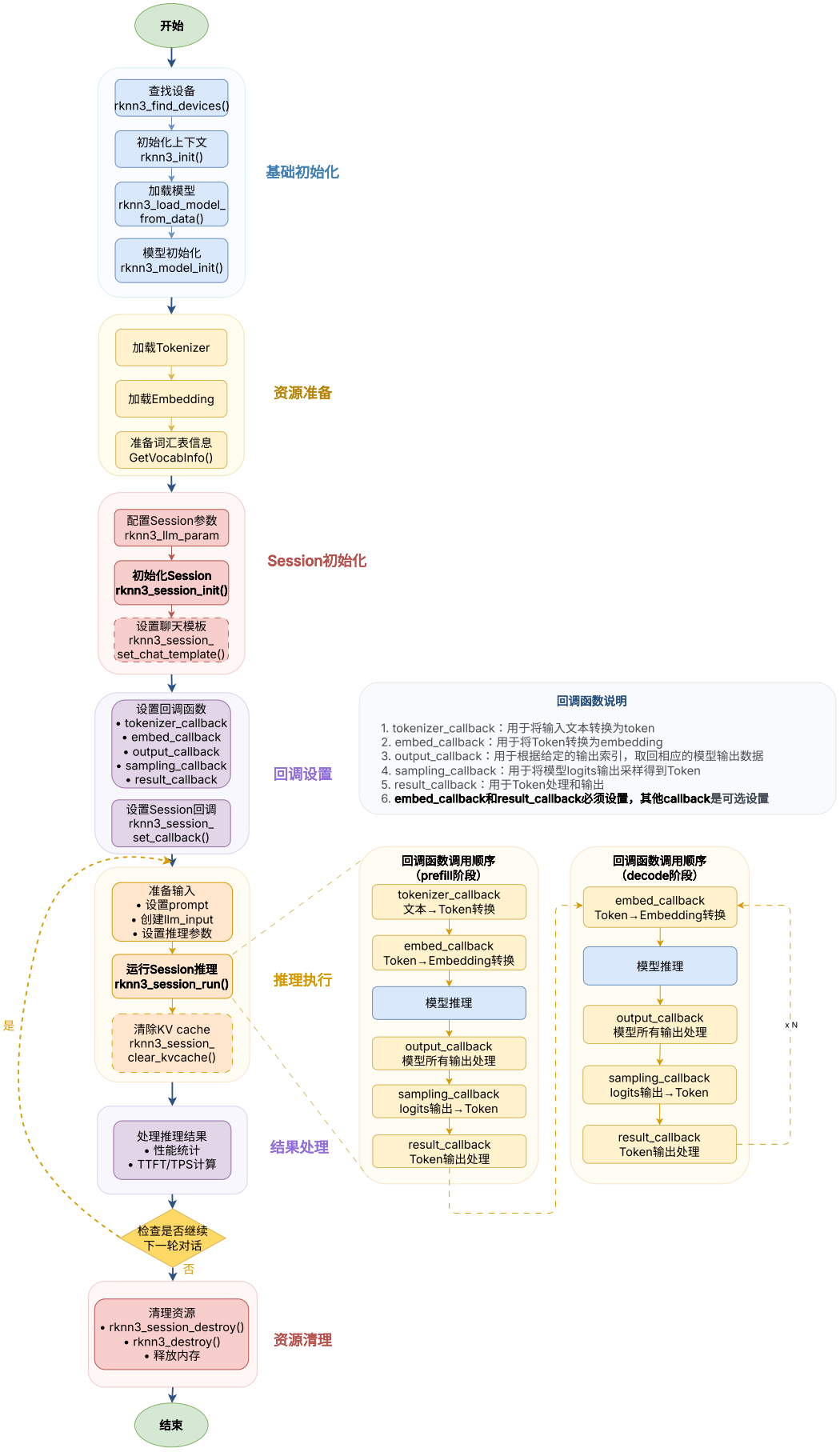


图3-6 基于 Session 管理的 LLM 模型部署流程

后续小节将按此流程逐步介绍各环节的操作方式与注意事项。

3.3.2 Session生命周期

模型加载和初始化完成后（`rknn3_init()` -> `rknn3_load_model_from_path()` -> `rknn3_model_init()`），通过 `rknn3_session_init()` 创建会话。创建时需配置 `rknn3_llm_param` 结构体，包含以下关键参数：

- `logits_name`：推理输出节点名称。
- `max_context_len`：最大上下文长度（token 数）。
- `sampling_param`：采样参数，控制生成的随机性与质量，包括：
- `temperature`：采样温度，值越大生成越随机（多样性高），值越小生成越确定（偏向高概率 token）。
- `top_k`：Top-K 采样，仅从概率最高的 K 个 token 中采样，值越小生成越保守。
- `top_p`：Top-P 采样，从累计概率达到 P 的最小 token 集合中采样，动态调整候选范围。
- `repeat_penalty`：重复惩罚，对已生成的 token 施加惩罚，降低重复内容的生成概率。
- `frequency_penalty`：频率惩罚，对出现频率高的 token 施加额外惩罚。
- `presence_penalty`：存在惩罚，对已出现过的 token 施加惩罚，鼓励生成新内容。
- `vocab_info`：词表信息，通常从 tokenizer 的 `GetVocabInfo()` 获取，包括：
- `vocab_size`：词汇表大小。
- `special_bos_id`：特殊序列开始（BOS）token 的 ID 列表，`n_special_bos_id` 为其数量。
- `special_eos_id`：特殊序列结束（EOS）token 的 ID 列表，`n_special_eos_id` 为其数量，控制推理何时结束。
- `linefeed_id`：换行 token 的 ID。
- `ignore_eos_token`：生成时是否忽略 EOS token，设为 True 时即使遇到结束 token 也不停止生成。

```
rknn3_llm_param params;
memset(&params, 0, sizeof(rknn3_llm_param));

params.logits_name = "logits";
params.max_context_len = 1024;
params.sampling_param.temperature = 1.0f;
params.sampling_param.top_k = 1;
params.sampling_param.top_p = 0.9;
params.sampling_param.repeat_penalty = 1.1f;
params.vocab_info.vocab_size = vocab_info.vocab_size;
params.vocab_info.n_special_eos_id = vocab_info.n_special_eos_id;
memcpy(params.vocab_info.special_eos_id, vocab_info.special_eos_id,
sizeof(vocab_info.special_eos_id));

rknn3_session *session = rknn3_session_init(ctx, &params, 1);  /* 第三参数 n_params
为参数数量 */
if (session == NULL) {
    printf("rknn3_session_init failed!\n");
    goto out;
}
```

会话使用完毕后，通过 `rknn3_session_destroy()` 销毁，释放会话相关资源（KVCache 等）。销毁后会话指针不可再使用：

```
ret = rknn3_session_destroy(session);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_destroy failed! ret=%d\n", ret);
}
```

> **说明**：一个 context 可创建多个 session，多个 session 共享模型权重和 internal memory，但 KVCache 及 LoRA 启用状态相互独立。多个 session 不能并发执行，同一时刻仅允许一个 session 运行。

3.3.3 输入和回调

LLM 推理通过回调机制处理 tokenization、embedding、采样、结果输出等环节。创建会话后，需通过 `rknn3_session_set_callback()` 注册回调函数。`RKLLMCallback` 结构体包含以下回调：

- **`result_callback`**：结果回调，每生成一个 token 时触发，用于处理模型输出（如将 token 转为文本输出）。通常搭配 tokenizer 的 `TokenToPiece()` 或 `Decode()` 使用。
- **`tokenizer_callback`**（可选）：分词回调，将输入文本转为 token id 序列。通常调用 tokenizer 的 `Tokenize()` 实现。
- **`embed_callback`**（可选）：嵌入回调，将 token id 转为 embedding 向量。通常从 `embed.bin` 文件中按 token id 查询。
- **`sampling_callback`**（可选）：采样回调，从模型输出的 logits 中采样得到 token id。若不设置，Runtime 使用默认采样策略（根据 `sampling_param` 配置）。
- **`output_callback`**（可选）：输出回调，用于获取模型输出 tensor。需配合 `output_tensors` 和 `n_output_tensors` 字段指定输出 tensor 数组，适用于需要直接处理输出 tensor 的场景（如将输出作为其他模型的输入）。
- **`input_callback`**（可选）：输入回调，用于提供自定义输入 tensor。需配合 `input_tensors_index` 字段指定输入 tensor 索引，适用于需要直接传入预处理后 tensor 的场景。

```
RKLLMCallback callback;
memset(&callback, 0, sizeof(RKLLMCallback));

callback.result_callback    = result_callback;
callback.result_userdata    = tokenizer;
callback.tokenizer_callback = tokenizer_callback;
callback.tokenizer_userdata = tokenizer;
callback.embed_callback     = embed_callback;
callback.embed_userdata     = &embedding_info;
callback.sampling_callback  = sampling_callback;
callback.sampling_userdata  = &embedding_info;

ret = rknn3_session_set_callback(session, &callback);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_set_callback failed! ret=%d\n", ret);
    goto out;
}
```

> **说明**：`tokenizer_callback` 和 `embed_callback` 的实现可参考 2.2.4 模型和Tokenizer适配。若使用 Tie Word 模式（见 4.2.9 Tie Word Embedding），则无需设置 `embed_callback`。

3.3.4 推理执行与中断

执行推理

会话创建并设置回调后，通过 `rknn3_session_run()` 执行推理。需准备 `rknn3_llm_input` 输入结构和 `rknn3_llm_infer_param` 推理参数。输入类型通过 `input_type` 指定，常用为 `RKNN3_LLM_INPUT_PROMPT`（文本输入）；推理参数包括 `keep_history`（是否保留历史上下文）和 `max_new_tokens`（最大生成 token 数）：

```
char *prompt = "Please explain the basic concept of relativity";
rknn3_llm_infer_param param = {.keep_history = 0, .max_new_tokens = 128};
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = prompt,
                                 .embed = NULL, .tokens = NULL,
                                 .n_tokens = 0, .enable_thinking = false};
rknn3_llm_input input;
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input = input_tensor;

rknn3_llm_input inputs[1] = {input};
ret = rknn3_session_run(session, inputs, 1, &param);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_run failed! ret=%d\n", ret);
    goto out;
}
```

推理过程中，模型每生成一个 token 便通过 `result_callback` 回调返回，用户在回调中处理输出（如打印文本）。推理持续直到达到 `max_new_tokens` 或遇到 EOS token。

Runtime 同时提供 `rknn3_session_run_async()` 异步推理接口，用法与同步接口一致，但不阻塞当前线程。

> **说明**：通用模型的 `rknn3_run_async()` / `rknn3_wait()` 当前版本暂未实现（见 3.1.4）；LLM Session 的 `rknn3_session_run_async()` 可用。

中断推理

推理过程中可通过 `rknn3_session_stop()` 中断当前会话推理，Runtime 会停止生成并返回：

```
ret = rknn3_session_stop(session);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_stop failed! ret=%d\n", ret);
}
```

3.3.5 Chat Template

适用场景

不同 LLM 采用不同的对话格式（如 ChatML / Llama-3 / Qwen 等），模型在训练时学习了特定的系统提示、角色标记和分隔符。若直接将用户原始文本送入模型，会因缺少格式标记导致生成质量下降或答非所问。Chat

Template 用于在用户输入外层包装模型要求的对话格式，使输入符合模型的预期。

使用方式

通过 `rknn3_session_set_chat_template()` 设置聊天模板，包含三部分：

- `system_prompt`：系统提示，定义模型的角色与行为上下文，每轮对话固定拼接在最前。
- `prompt_prefix`：用户输入前缀，标识用户消息的开始（如 `<|im_start|>user\n`）。
- `prompt_postfix`：用户输入后缀，标识用户消息的结束并引导模型生成（如 `<|im_end|>\n<|im_start|>assistant\n`）。

设置后，每次 `rknn3_session_run()` 传入 PROMPT 输入时，Runtime 会自动按 `system_prompt + prompt_prefix + 用户输入 + prompt_postfix` 拼接成完整对话送入模型。该接口应在 `rknn3_session_init()` 之后、`rknn3_session_run()` 之前调用，且通常只需设置一次。

```
const char* system_prompt  = "<|im_start|>system\nYou are a helpful assistant.
<|im_end|>\n";
const char* prompt_prefix  = "<|im_start|>user\n";
const char* prompt_postfix = "<|im_end|>\n<|im_start|>assistant\n";
ret = rknn3_session_set_chat_template(session, system_prompt, prompt_prefix,
prompt_postfix);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_set_chat_template failed! ret=%d\n", ret);
    goto out;
}
```

> **说明**：模板字符串须与模型训练时的对话格式严格一致，可从模型的 `tokenizer_config.json` 中的 `chat_template` 字段推导。模板设置错误会导致模型输出混乱或重复。若模型在转换时已通过 config 文件配置了 chat template（见 2.2.2），则此处为运行时覆盖；不设置则沿用转换时的配置。

3.3.6 多模态输入

适用场景

MLLM（多模态大模型）除文本外，还能接收图像、音频、视频等输入。典型流程为：先用对应编码模型（vision/audio/video）将原始模态数据编码为嵌入向量（embeds），再将文本 prompt 与各模态嵌入一并送入 LLM，由 LLM 融合多模态信息生成回答。RKNN3 通过 `RKNN3_LLM_INPUT_MULTIMODAL` 输入类型支持该流程。

输入结构

多模态输入使用 `rknn3_llm_multimodal_tensor` 结构（`rknn3_llm_input.multimodal_input`），成员分两部分：

- **文本部分**：`prompt`（文本提示）与 `tokens`（token id 数组）二选一，若同时提供默认按 `prompt` 推理；`enable_thinking` 控制本次输入是否启用思考模式。与 Grammar 同时使用时，完整输出约束规则见 4.2.11。
- **模态子结构**：`image`、`audio`、`video` 三个子结构，三者结构对称，均包含对应编码模型输出的 `*_embed` 嵌入指针、token 数量（`n_*_tokens`）、模态数量（`n_image` / `n_audio` / `n_video`），以及文本中的占位标签（`*_start` / `*_end` / `*_content`，须与模型 tokenizer 特殊 token 一致）。按实际输入模态填充其一或多者，完整成员定义见《RKNN3_Runtime_C_API_参考》。

使用方式（以 VLM 为例）

VLM（视觉语言模型）部署时，vision 模型与 LLM 模型是两个独立的 `.rknn` 文件，需分别初始化、分别推理。vision 模型使用通用 RKNN3 C API（见 3.2）推理得到图像嵌入；LLM 侧将输入类型设为 `RKNN3_LLM_INPUT_MULTIMODAL`，填充 `image` 子结构与文本 prompt 后调用 `rknn3_session_run()`：

```
/* img_embeds 由 vision 模型推理输出 */
rknn3_llm_multimodal_tensor tensor;
memset(&tensor, 0, sizeof(rknn3_llm_multimodal_tensor));
tensor.name   = "input_embeds";
tensor.prompt = prompt;                          // 文本提示

tensor.image.image_embed = img_embeds;           // vision 模型输出的图像嵌入
if (rknn_app_ctx.vision.embeds_ndims == 2) {
    tensor.image.n_image_tokens = rknn_app_ctx.vision.embeds_shape[0];
    tensor.image.n_image        = 1;
} else {
    tensor.image.n_image_tokens = rknn_app_ctx.vision.embeds_shape[1];
    tensor.image.n_image        = rknn_app_ctx.vision.embeds_shape[0];
}
tensor.image.image_width   = rknn_app_ctx.vision.model_width;
tensor.image.image_height  = rknn_app_ctx.vision.model_height;
tensor.image.image_start   = "<|vision_start|>";   // 须与模型 tokenizer 一致
tensor.image.image_end     = "<|vision_end|>";
tensor.image.image_content = "<|image_pad|>";
tensor.enable_thinking = false;

rknn3_llm_input inputs[1];
memset(inputs, 0, sizeof(inputs));
inputs[0].input_type       = RKNN3_LLM_INPUT_MULTIMODAL;
inputs[0].multimodal_input = tensor;

ret = rknn3_session_run(session, inputs, 1, &param);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_session_run failed! ret=%d\n", ret);
    goto out;
}
```

> **说明**：audio / video 模态的填充方式与 image 完全对称，将 `image_*` 成员替换为对应 `audio_*` / `video_*` 即可。完整的多模态部署示例（vision + LLM 联动）可参考 `rknn3-model-zoo` 中 Qwen2.5-VL / FastVLM 等示例工程。

3.3.7 完整示例

本节给出 LLM Session 部署的完整调用示例，串联前述生命周期、回调、推理各环节。完整工程代码请参考 SDK 示例目录 `rknn3-runtime/examples/rknn3_session_test_demo/`。

完整示例涵盖以下步骤：

1. **初始化上下文与加载模型**：`rknn3_init()` -> `rknn3_load_model_from_path()` -> `rknn3_model_init()`。
2. **创建会话**：配置 `rknn3_llm_param`（logits_name / max_context_len / sampling_param / vocab_info），调用 `rknn3_session_init()`。

3. **设置回调**：填充 `RKLLMCallback`（result_callback / tokenizer_callback / embed_callback / sampling_callback），调用 `rknn3_session_set_callback()`。
4. **设置聊天模板**（可选）：调用 `rknn3_session_set_chat_template()`。
5. **执行推理**：填充 `rknn3_llm_input`（input_type=PROMPT）和 `rknn3_llm_infer_param`，调用 `rknn3_session_run()`，结果通过 `result_callback` 流式返回。
6. **销毁资源**：`rknn3_session_destroy()` -> `rknn3_destroy()`。

```cpp
#include "rknn3_api.h"
#include "Tokenizer.h"

/* 结果回调：每生成一个 token 后被调用，按 LLMCallState 处理不同状态 */
static int result_callback(void* userdata, RKLLMResult* result, LLMCallState state)
{
    Tokenizer* tokenizer = (Tokenizer*)userdata;

    if (state == RKLLM_RUN_NORMAL) {
        std::string piece;
        if (result->num_tokens == 1) {
            piece = tokenizer->TokenToPiece(result->token_ids[0]);
        } else {
            piece = tokenizer->Decode(result->token_ids, result->num_tokens);
        }
        printf("%s", piece.c_str());
    } else if (state == RKLLM_RUN_FINISH) {
        printf("\n[generation finished]\n");
    } else if (state == RKLLM_RUN_ERROR) {
        printf("\n[inference error]\n");
    }
    fflush(stdout);
    return 0;
}

/* 分词回调：将文本转为 token id 序列 */
int tokenizer_callback(void *userdata, const char *text, int32_t text_len, int32_t
*tokens, int32_t n_tokens_max)
{
    int n_tokens = 0;
    Tokenizer *tokenizer = (Tokenizer *)userdata;
    n_tokens = tokenizer->Tokenize(text, text_len, tokens, n_tokens_max);

    if (n_tokens <= 0)
    {
        printf("tokenizer failed for %s\n", text);
        return n_tokens;
    }
    return n_tokens;
}

/* 嵌入回调：按 token id 从 embed.bin 查询 embedding */
int embed_callback(void* userdata, int32_t* tokens, uint64_t num_tokens, void*
embed, uint64_t len)
{
    struct embedding_info* embed_info = (struct embedding_info*)userdata;

    if (len != num_tokens * embed_info->embedding_dim * sizeof(float16)) {
        printf("invalid embed buffer\n");
        return -1;
    }

    for (int n = 0; n < num_tokens; n++) {
        memcpy((unsigned char*)embed +
               n * embed_info->embedding_dim * sizeof(float16),
               embed_info->embedding_data + tokens[n] * embed_info->embedding_dim,
               embed_info->embedding_dim * sizeof(float16));
    }
    return 0;
}
```

```c
int main(int argc, char** argv)
{
    int ret = 0;
    rknn3_context ctx = NULL;
    rknn3_session* session = NULL;

    /* tokenizer 与 embedding_info 的创建/加载省略，完整实现见 SDK 示例 */
    Tokenizer* tokenizer = NULL;
    struct embedding_info embedding_info;

    /* 1. 初始化上下文与加载模型 */
    ret = rknn3_init(&ctx, NULL);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_init failed! ret=%d\n", ret);
        goto out;
    }
    ret = rknn3_load_model_from_path(ctx, "model.rknn", NULL);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_load_model_from_path failed! ret=%d\n", ret);
        goto out;
    }
    rknn3_config config = {0};
    config.run_core_mask = 0xff;  /* 按实际核心数配置 */
    ret = rknn3_model_init(ctx, &config);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_model_init failed! ret=%d\n", ret);
        goto out;
    }

    /* 2. 创建会话 */
    rknn3_llm_param params;
    memset(&params, 0, sizeof(rknn3_llm_param));
    params.logits_name    = "logits";
    params.max_context_len = 1024;
    params.sampling_param.temperature   = 1.0f;
    params.sampling_param.top_k         = 1;
    params.sampling_param.top_p         = 0.9;
    params.sampling_param.repeat_penalty = 1.1f;
    /* vocab_info 由 tokenizer->GetVocabInfo() 填充，此处省略 */

    session = rknn3_session_init(ctx, &params, 1);
    if (session == NULL) {
        printf("rknn3_session_init failed!\n");
        goto out;
    }

    /* 3. 设置回调 */
    RKLLMCallback callback;
    memset(&callback, 0, sizeof(RKLLMCallback));
    callback.result_callback    = result_callback;
    callback.result_userdata    = tokenizer;
    callback.tokenizer_callback = tokenizer_callback;
    callback.tokenizer_userdata = tokenizer;
    callback.embed_callback     = embed_callback;
    callback.embed_userdata     = &embedding_info;
    ret = rknn3_session_set_callback(session, &callback);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_session_set_callback failed! ret=%d\n", ret);
        goto out;
    }

    /* 4. 设置聊天模板 */
```

```
    const char* system_prompt  = "<|im_start|>system\nYou are a helpful assistant.
<|im_end|>\n";
    const char* prompt_prefix  = "<|im_start|>user\n";
    const char* prompt_postfix = "<|im_end|>\n<|im_start|>assistant\n";
    ret = rknn3_session_set_chat_template(session, system_prompt, prompt_prefix,
prompt_postfix);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_session_set_chat_template failed! ret=%d\n", ret);
        goto out;
    }

    /* 5. 执行推理 */
    char* prompt = "Please explain the basic concept of relativity";
    rknn3_llm_infer_param param = {.keep_history = 0, .max_new_tokens = 128};
    rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = prompt,
                                     .embed = NULL, .tokens = NULL,
                                     .n_tokens = 0, .enable_thinking = false};
    rknn3_llm_input inputs[1];
    inputs[0].input_type = RKNN3_LLM_INPUT_PROMPT;
    inputs[0].llm_input  = input_tensor;

    ret = rknn3_session_run(session, inputs, 1, &param);
    if (ret != RKNN3_SUCCESS) {
        printf("rknn3_session_run failed! ret=%d\n", ret);
        goto out;
    }

out:
    /* 6. 销毁资源 */
    if (session) {
        rknn3_session_destroy(session);
    }
    rknn3_destroy(ctx);
    return ret;
}
```

> **说明**：上述示例中 tokenizer 的初始化、`GetVocabInfo()` 调用、`embed.bin` 加载等准备步骤省略，完整实现请参考 SDK 示例 `rknn3-runtime/examples/rknn3_session_test_demo/`。
> `result_callback` 的 `state` 参数为 `LLMCallState` 枚举，常见取值包括 `RKLLM_RUN_NORMAL`（正常生成）、`RKLLM_RUN_FINISH`（推理完成）、`RKLLM_RUN_ERROR`（出错）、`RKLLM_RUN_MAX_NEW_TOKEN_REACHED`（达到最大生成数）等，完整状态定义见《RKNN3_Runtime_C_API_参考》。

3.4 Toolkit Lite Python部署

前置条件：使用 Toolkit Lite 前，需先通过 PC 端 RKNN3-Toolkit 将模型转换为 `.rknn` + `.weight`。

定位与适用场景

RKNN3-Toolkit Lite 是板端 Python 推理接口，对 RKNN3 Runtime（C API）做 Python 封装，提供与传统 `RKNN-Toolkit Lite` 一致的高层推理体验，同时支持 LLM / MLLM 的 Session、Callback、KVCache 等能力。与 C API 相比：

对比项	Toolkit Lite（Python）	Runtime（C API）
开发效率	高，Python 快速迭代	低，需编译部署

对比项	Toolkit Lite（Python）	Runtime（C API）
运行性能	略低（封装开销）	最优
典型场景	原型验证、算法调试、应用开发	量产部署、极致性能
支持平台	RK1820 / RK1828 / RK3572	同左

Toolkit Lite 适用于板端 Python 快速验证与上层应用开发；当性能与资源敏感、准备量产时，建议迁移到 C API。

3.4.1 Python运行环境

Toolkit Lite 运行于板端，当前支持的平台如下：

项目	支持范围
硬件平台	RK1820 / RK1828 / RK3572
操作系统	Debian 10 / 11 / 12（aarch64）
Python	3.9 / 3.10 / 3.11 / 3.12
主要 Python 依赖	`numpy`、`transformers`、`jinja2`

具体的版本矩阵、安装步骤、导入方式见《RKNN3_Toolkit_Lite_Python_API_参考》第 1 章。

3.4.2 RKNN3Lite对象初始化与释放

`RKNN3Lite` 是 Toolkit Lite 的 Python 主入口，所有操作围绕该对象展开。使用前需构造对象，使用完毕后需释放，整个生命周期遵循"创建对象 -> 加载模型 -> 初始化运行时 -> 推理 -> 释放"的顺序。

构造对象

`RKNN3Lite()` 构造时仅建立 Python 侧封装，不会立即创建 Runtime 或 Session，底层资源由后续 `init_runtime()` / `init_llm_session()` 创建。主要参数：

- `llm_mode`：是否为 LLM / MLLM 模式，默认 `False`。LLM / MLLM 需设为 `True`，用于初始化 Session 相关代码；通用模型不设置。
- `verbose`：是否在终端打印详细日志，默认 `False`。
- `verbose_file`：日志文件路径，指定后日志同时写入该文件，默认 `None`。

```python
from rknn3lite.api import RKNN3Lite

# 通用模型
rknn_lite = RKNN3Lite(verbose=True)
# LLM / MLLM 模型
rknn_lite = RKNN3Lite(llm_mode=True, verbose=True, verbose_file='./inference.log')
```

释放对象

`release()` 释放资源。LLM / MLLM 多 Session 场景下，可指定 `session_index` 仅释放某个 Session、保留共享 Runtime；不指定则释放全部 Session 与 Runtime：

```
# 释放全部资源
rknn_lite.release()
# 仅释放指定 LLM Session（保留 Runtime 供其他 Session 使用）
rknn_lite.release(session_index=0)
```

> **说明**：`release()` 应在所有推理结束后调用，且每个 `RKNN3Lite` 对象须配对调用一次，避免资源泄漏。

3.4.3 加载和初始化模型

加载模型

`load_rknn()` 加载已转换的 `.rknn` 与 `.weight` 文件，两者路径须配套，否则加载失败：

```
ret = rknn_lite.load_rknn(model_path='mobilenetv2-12.rknn',
                          weight_path='mobilenetv2-12.weight')
if ret != 0:
    raise RuntimeError(f"load_rknn failed, ret={ret}")
```

初始化运行时

`init_runtime()` 初始化运行时环境。通用模型与 LLM / MLLM 模型参数不同：

- **通用模型**：指定 `target`、`core_mask`、`device_id`（多设备时通过 `get_devices_id()` 获取）。
- **LLM / MLLM**：额外通过 `llm_args` 传入采样等参数，`llm_callback` 传入回调对象。

```
# 通用模型
device_id = rknn_lite.get_devices_id()
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
device_id=device_id[0])

# LLM / MLLM
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0xff,
                             llm_args=ARGS, llm_callback=callback)
```

初始化要点

- `core_mask` 必须与模型转换时 `rknn.config(core_num=...)` 配置的核心数一致，否则报错。RK1820 / RK1828 为 0x1~0xff（1~8 核），RK3572 仅 0x1（单核）。
- LLM / MLLM 推荐采用"先 `init_runtime`、再 `rknn3_query(RKNN3_QUERY_LLM_CONFIG)` 查询模型配置、最后 `init_llm_session()`"的分步流程，可在创建 Session 前读取模型实际的 `vocab_size`、`embedding_dim`、`max_ctx_len`、KVCache 配置等，避免硬编码。若应用已确定 `llm_args` 与 Callback，也可在 `init_runtime()` 时直接传入，由初始化流程创建首个 Session。
- 一个 LLM 对象可在同一 Runtime 上创建多个 Session（以 `session_index` 管理），各 Session 独立维护对话状态、KVCache 及按 Session 生效的配置，但同一时刻仅允许一个 Session 运行。
- `llm_args` 的采样参数（`top_k` / `top_p` / `temperature` / `repeat_penalty` / `max_new_tokens` 等）与 C API 的 `rknn3_sampling_params` / `rknn3_llm_param` 含义对应，见 3.3.2。

3.4.4 张量和内存管理

Toolkit Lite 在 Python 层封装了张量与内存管理，按所有权分为三类：

- **高层输入输出**：传统模型的 numpy ndarray 输入输出，由高层接口自动完成数据准备与布局转换，用户无需关心底层内存。
- **Runtime 创建的 Tensor Memory**：通过 `create_mem()` 等接口分配的设备内存，由 Runtime 管理生命周期，用户通过 `mem_sync()` 在 CPU 与设备间同步数据。
- **用户提供的模型运行内存**：通过 `set_internal_mem()` / `set_weight_mem()` 将用户预分配的内存绑定给模型，适用于内存复用、跨进程共享等高级场景。

多模态嵌入组织

MLLM 部署时，各模态编码模型（vision/audio/video）的输出需转为 Toolkit Lite 期望的 FP16 格式，再通过对应的专用类型传入 LLM。三种模态类型结构对称，均包含编码模型输出的嵌入指针、token 数量、模态数量及文本占位标记：

模态	类型	嵌入指针	token 数	模态数量	占位标记
图像	`RKNN3Image`	`image_embed`	`n_image_tokens`	`n_image`	`image_start` / `image_end` / `image_content`
音频	`RKNN3Audio`	`audio_embed`	`n_audio_tokens`	`n_audio`	`audio_start` / `audio_end` / `audio_content`
视频	`RKNN3Video`	`video_embed`	`n_frame_tokens` / `n_frame_per_video`	`n_video`	`video_start` / `video_end` / `video_content`

以图像为例，vision 模型输出转 FP16 后填充 `RKNN3Image` 传入 LLM：

```
import numpy as np
import ctypes
from rknn3lite.api import RKNN3Image, Float16

# vision 模型输出转 FP16
outputs = np.float16(np.expand_dims(rknn_vision.inference(inputs=[feature])[0], 0))

llm_input = RKNN3Image()
llm_input.image_embed   = outputs.ctypes.data_as(ctypes.POINTER(Float16))
llm_input.n_image_tokens = outputs.shape[1]
llm_input.n_image        = outputs.shape[0]
llm_input.image_width    = 392
llm_input.image_height   = 392
llm_input.image_start    = "<|vision_start|>".encode('utf-8')
llm_input.image_end      = "<|vision_end|>".encode('utf-8')
llm_input.image_content  = "<|image_pad|>".encode('utf-8')

ret, perf = rknn_llm.session_run(inputs=[llm_input], prompt="<image>请描述图片内容")
```

音频、视频的填充方式与图像完全对称：将编码模型输出转 FP16，按对应类型的成员（`audio_embed` / `video_embed` 等）填充即可。多种模态可同时传入，按实际输入构造对应的 `RKNN3Image` / `RKNN3Audio` / `RKNN3Video` 列表交给 `session_run(inputs=...)`。

> **说明**：各模态的占位标记须与模型 tokenizer 特殊 token 一致。底层结构与 C API 的 `rknn3_llm_multimodal_tensor` 对应（见 3.3.6），完整成员定义见《RKNN3_Runtime_C_API_参考》5.5.25。

3.4.5 执行推理

通用模型

通过 `inference()` 同步推理，输入输出均为 numpy ndarray：

```
outputs = rknn_lite.inference(inputs=[img])
if outputs is None:
    raise RuntimeError("inference failed")
```

也支持异步 `inference_async()` -> `wait()` -> `get_outputs()`，适用于需要在等待结果期间执行其他任务的场景。

LLM / MLLM

通过 `session_run()` 推理，支持流式输出，结果通过 `result_callback` 异步返回：

```
prompt = "介绍一下LLM模型的工作原理。"
ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] = \
    rknn_llm.session_run(prompt=prompt)
if ret != 0:
    raise RuntimeError(f"session_run failed, ret={ret}")
```

- `prompt`（文本）与 `embeds`（嵌入向量，3D：batch × seq × hidden）互斥，后者跳过 tokenization。
- 多模态输入通过 `inputs` 传入 `RKNN3Image` / `RKNN3Audio` / `RKNN3Video` / `RKNN3AuxTensor` 等专用类型，填充方式与 C API 的 `rknn3_llm_multimodal_tensor` 对应。
- 返回值含性能统计 `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`，可用于性能分析。
- 异步推理使用 `session_run_async()`，配合 `session_stop()` / `session_pause()` / `session_resume()` 实现中断、暂停、恢复。

3.4.6 完整示例

本节给出两个最小可运行示例，分别对应通用模型与 LLM 两类流程，完整代码见 SDK 的 `rknn3-toolkit-lite/examples` 目录。

通用模型示例：

以 MobileNetV2 图像分类为例，演示创建对象 -> 加载模型 -> 初始化运行时 -> 查询张量属性 -> 推理 -> 后处理 -> 释放的完整流程：

```python
from rknn3lite.api import RKNN3Lite

rknn_lite = RKNN3Lite()

# 1. 加载模型
ret = rknn_lite.load_rknn(model_path='mobilenetv2-12.rknn',
                          weight_path='mobilenetv2-12.weight')

# 2. 初始化运行时（core_mask 须与转换时一致）
device_id = rknn_lite.get_devices_id()
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
device_id=device_id[0])

# 3. 查询输入输出张量属性（shape/dtype/layout 等）
for attr in rknn_lite.get_inputs_tensor_attr():
    print("input:", attr)
for attr in rknn_lite.get_outputs_tensor_attr():
    print("output:", attr)

# 4. 推理（输入为预处理后的 numpy ndarray）
outputs = rknn_lite.inference(inputs=[img])

# 5. 后处理（Top5 分类）与释放
show_top5(outputs[0])
rknn_lite.release()
```

LLM 示例：

以 Qwen2.5-0.5B Instruct 为例，演示 LLM 模式下创建对象 -> 加载模型 -> 构造回调 -> 初始化运行时 -> 设置聊天模板 -> 多轮推理 -> 释放的完整流程：

```
import ctypes, numpy as np
from transformers import AutoTokenizer
from rknn3lite.api import (RKNN3Lite, RKLLMCallback,
                           LLMResultCallback, LLMGetEmbedCallback,
LLMTokenizerCallback)

# 预备：加载 tokenizer 与 embed.bin（词表向量，shape = vocab_size × embedding_dim）
tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_PATH)
embeds_data = np.fromfile(EMBED_PATH, dtype=np.float16).reshape(VOCAB_SIZE, -1)

# 1. 创建 LLM 模式对象并加载模型
rknn = RKNN3Lite(llm_mode=True, verbose=True)
rknn.load_rknn(RKNN_MODEL, WEIGHT_MODEL)

# 2. 构造回调：result（流式输出）、tokenizer（文本转 token）、embed（按 token 查词表向量）
callback = RKLLMCallback()
callback.result_callback = LLMResultCallback(result_callback)
callback.tokenizer_callback = LLMTokenizerCallback(tokenizer_callback)
callback.embed_callback = LLMGetEmbedCallback(embed_callback)
# userdata 通过 ctypes.py_object + ctypes.cast 传递 Python 对象指针
userdata = ctypes.py_object(tokenizer)
callback.tokenizer_userdata = ctypes.cast(ctypes.pointer(userdata), ctypes.c_void_p)
userdata = ctypes.py_object(embeds_data)
callback.embed_userdata = ctypes.cast(ctypes.pointer(userdata), ctypes.c_void_p)

# 3. 初始化运行时（llm_args 含采样参数，llm_callback 传入回调对象）
ARGS = [{"top_k": 1,
         "top_p": 0.8,
         "temperature": 0.8,
         "repeat_penalty": 1.1,
         "vocab_size": VOCAB_SIZE,
         "special_eos_id": 151645,
         "max_context_len": 1024,
         "keep_history": 0,
         "max_new_tokens": 1024}]
rknn.init_runtime(target='rk1820', core_mask=0xff, llm_args=ARGS,
llm_callback=callback)

# 4. 设置聊天模板
rknn.set_chat_template(system_prompt, prompt_prefix, prompt_postfix)

# 5. 多轮推理，返回值含性能统计
for prompt in prompts:
    ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] = \
        rknn.session_run(prompt=prompt)
    printf_perf(first_token, n_decode_tokens, n_prefill_tokens, llm_start_time,
llm_end_time)

rknn.release()
```

示例要点说明

- **回调实现**：`result_callback` 按 `LLMCallState` 状态分支处理（`RKLLM_RUN_NORMAL` 正常生成、`RKLLM_RUN_FINISH` 完成、`RKLLM_RUN_ERROR` 出错、`RKLLM_RUN_WAITING` 等待 UTF-8 字符、`RKLLM_RUN_STOP` 停止、`RKLLM_RUN_MAX_NEW_TOKEN_REACHED` 达最大 token、`RKLLM_RUN_PAUSE` / `RKLLM_RUN_RESUME` 暂停/恢复），正常状态下累积 token 并增量解码打印，解决 UTF-8 字符跨 token 截断问题；`tokenizer_callback` 调用 HuggingFace tokenizer 分词；`embed_callback` 按 token id 从 `embeds_data` 查询词表向量拷贝到 buffer。

- **userdata 传递**：Python 对象通过 `ctypes.py_object` 包装后 `cast` 为 `c_void_p` 传入回调，回调内再还原使用，避免被 Python GC 回收。
- **性能统计**：`session_run` 返回的 `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]` 配合首 token 时间，可分别计算 prefill 与 generate 阶段的每 token 耗时和吞吐（tokens/s）。
- **多轮对话**：`keep_history=1` 时 Session 自动保留历史 KVCache，实现多轮上下文；`keep_history=0` 时每轮独立。

> **说明**：指定的 RKNN 模型路径须与配套权重文件路径一致。由于模型版本、输入图像或运行环境差异，实际输出可能与示例略有不同。完整示例代码见 SDK 的 `rknn3-toolkit-lite/examples` 目录。

3.5 RKLLM3 Server集成

rkllm3-server 是基于 RKNPU3 实现的轻量级 LLM HTTP Server，提供与 OpenAI API 兼容的推理服务，底层推理加速由 RKLLM3 Runtime（封装 RKNN3 Runtime）实现。本节介绍其适用场景、部署方式与客户端调用。

3.5.1 Server适用场景

rkllm3-server 适用于以下场景：

- **服务化部署**：将 LLM / MLLM 模型封装为 HTTP 服务，供多客户端通过标准 OpenAI API 调用，与现有应用生态对接。
- **多模态推理**：支持文本、图像、音频输入（暂不支持视频输入），覆盖 LLM、VLM、语音模型、多模态融合模型等多种模型类型。
- **快速验证与集成**：无需开发 C/Python 推理代码，通过 `curl`、OpenAI SDK 或任意 HTTP 客户端即可调用，适合模型效果验证、原型集成。
- **多模型与多会话**：支持单进程加载多个模型、多会话并发（最大 16），适用于多模型路由、多用户服务等场景。

与原生接口的定位差异：rkllm3-server 牺牲部分极致性能与灵活性（受限于 HTTP 与 OpenAI 协议），换取易用性与生态兼容；需要极致性能、细粒度控制（如 KVCache/LoRA/Callback）或嵌入式集成时，应使用 C API 或 Toolkit Lite。

3.5.2 服务部署概览

rkllm3-server 以板端可执行程序形式部署，通过命令行参数指定模型、词表、采样配置等。典型部署流程为：准备模型产物（`.rknn` / `.weight` / `tokenizer.gguf` / `embed.bin` 等）-> 启动服务进程 -> 客户端访问。

启动示例

```
# LLM 模型（标准模式，需 embed 文件）
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf \
  --embed qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 \
  -c 768 --n_predict 512 --top-k 1 --top-p 0.8 --temp 0.8

# 多模态模型（LLM + Vision）
./rkllm3-server -m Qwen2.5-VL-3B-llm.rknn --model2 Qwen2.5-VL-3B-vision.rknn \
  --vocab Qwen2.5-VL-3B-llm.tokenizer.gguf --embed Qwen2.5-VL-3B-llm.embed.bin \
  --host 0.0.0.0 --port 8080 -c 768 --n_predict 512 \
  --img-start "<|vision_start|>" --img-end "<|vision_end|>" --img-content "
<|image_pad|>" \
  --img-width 392 --img-height 392

# 多模态模型（LLM + Vision + Audio）
./rkllm3-server -m Qwen2.5-Omni-3B-llm.rknn \
  --model2 Qwen2.5-Omni-3B-vision.rknn --model3 Qwen2.5-Omni-3B-audio.rknn \
  --vocab Qwen2.5-Omni-3B-llm.tokenizer.gguf --embed Qwen2.5-Omni-3B-llm.embed.bin \
  --mel-filter mel_128_filters.txt --host 0.0.0.0 --port 8080 \
  --img-start "<|vision_bos|>" --img-end "<|vision_eos|>" --img-content "<|IMAGE|>"
\
  --audio-start "<|audio_bos|>" --audio-end "<|audio_eos|>" --audio-content "
<|AUDIO|>"
```

关键参数分类

类别	常用参数	说明
模型路径	`-m` / `--weight` / `--model2` / `--model3`	LLM/vision/audio 模型与权重路径，多模态时用 `--model2` / `--model3` 指定 vision/audio 模型
词表与嵌入	`--vocab` / `--embed` / `--mel-filter`	tokenizer（`.gguf`）、词表嵌入（`.bin`）、音频 mel 滤波器
上下文与生成长度	`-c` / `--n_predict`	上下文长度（0=从模型加载）、生成 token 数（-1=上下文大小）
采样参数	`--temp` / `--top-k` / `--top-p` / `--repeat-penalty` / `--presence-penalty` / `--frequency-penalty`	与 C API 采样参数含义一致，见 3.3.2
多模态占位标记	`--img-start` / `--img-end` / `--img-content`（及 audio/video 对应项）	须与模型 tokenizer 特殊 token 一致
服务配置	`--host` / `--port` / `--timeout` / `--n-session`	监听地址端口、超时、并发会话数（最大 16）
NPU 核心	`--core-mask` / `--core-mask2` / `--core-mask3`	LLM/vision/audio 模型的 NPU 核心掩码，RK182X 为 0x1~0xff，RK3572 仅 0x1
高级功能	`--lora-weight` / `--tie-word-embedding` / `--checkpoint-*` / `--embedding`	LoRA、Tie Word、KVCache Checkpoint、词嵌入模型（`--embedding` 标识词嵌入模型，与 `--embed` 指定 embed.bin 路径不同）

> **说明**：多模态占位标记（`--img-start` 等）必须与模型 tokenizer 特殊 token 严格一致，否则模型无法正确识别模态输入位置。`--core-mask` 须与模型转换时 `core_mask` 一致。完整参数列表与多模型 `--params-file` JSON 配置格式见《RKLLM3_Server使用指南与API_参考》。

库形式集成

除可执行程序外，rkllm3-server 还提供 `librkllm3-server.so`，可将服务能力以库形式集成到自有应用中，配置通过 JSON 字符串传入，适用于不希望独立部署服务进程的嵌入式集成场景：

```
#include "rkllm3-server.h"
// 通过 JSON 配置启动 server，参数与命令行/params.json 一致
rkllm3_server_start(json_config_str);
```

3.5.3 客户端调用方式

rkllm3-server 兼容 OpenAI API，客户端可直接使用 OpenAI SDK、`curl` 或任意 HTTP 客户端调用。

OpenAI SDK 调用

将 `base_url` 指向服务地址即可复用现有 OpenAI 应用代码：

```
from openai import OpenAI

client = OpenAI(base_url="http://<board_ip>:8080/v1", api_key="rkllm3")
response = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "请解释一下相对论的基本概念"}],
    stream=True,   # 支持流式输出
)
for chunk in response:
    print(chunk.choices[0].delta.content or "", end="", flush=True)
```

多模态调用

图像输入通过 `messages` 中的 `image_url` 传递，音频输入通过对应字段传递，格式遵循 OpenAI 多模态规范：

```
response = client.chat.completions.create(
    model="Qwen2.5-VL-3B",
    messages=[{"role": "user", "content": [
        {"type": "text", "text": "请描述图片内容"},
        {"type": "image_url", "image_url": {"url": "file:///path/to/image.jpg"}},
    ]}],
)
```

curl 调用

```
curl http://<board_ip>:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"Qwen2.5-0.5B","messages":[{"role":"user","content":"你是
谁? "}],"stream":false}'
```

> **说明**：`model` 字段需与启动时 `-a/--alias` 或模型名一致。支持流式（`stream:true`）与非流式两种模式。支持的端点与完整请求/响应字段见《RKLLM3_Server使用指南与API_参考》。

3.5.4 与原生C/Python接口的选择

三种 LLM / MLLM 部署方式的对比如下，按场景选择：

对比项	rkllm3-server	C API（Runtime）	Toolkit Lite（Python）
调用方式	HTTP/OpenAI API	C 推理接口	Python 推理接口
易用性	最高，标准协议	低，需编译	高，快速迭代
性能	略低（HTTP 开销）	最优	略低（封装开销）
功能完整性	OpenAI 协议覆盖范围内	最全	略低
典型场景	服务化、多客户端、生态对接	量产部署、极致性能、细粒度控制	原型验证、算法调试

选择建议：

- **服务化/多客户端/快速集成**：选 rkllm3-server。
- **量产/极致性能/细粒度控制**：选 C API。
- **板端 Python 验证/上层应用**：选 Toolkit Lite。

3.5.5 Server文档入口

rkllm3-server 的完整内容（全量命令行参数、多模型 JSON 配置、OpenAI 兼容端点与请求响应字段、错误码、`librkllm3-server.so` 集成 API 等）见独立文档《RKLLM3_Server使用指南与API_参考》，本手册仅介绍部署与调用的核心流程。

3.6 RK182X多卡LLM部署

本节说明如何参考 rknn3-model-zoo/examples/multicard，将一个按 Transformer block 连续切分的 LLM 部署到多张 RK1820 / RK1828 加速卡上。该方案使用主控侧的多个 RKNN3 context/session，通过 Host 侧线程和队列组成 pipeline parallelism；Runtime 不会自动把多个 context 合并成一个多卡模型。

多卡部署与 NPU 多核是两个不同层次的概念：

配置项	控制范围	说明
rknn3_config.run_core_mask / core_mask	单张卡内部	选择该 stage 在一张 RK182X 卡上使用的 NPU core，必须与转换时的 core 配置匹配
stage_count	模型分段数量	一个 stage 对应一张 RK182X 卡，必须与实际导出的 seg 数量一致
device_id	物理设备选择	将某个 stage 绑定到指定的 RK182X 设备
bucket_size	Host 流水粒度	prefill 时每次从一个 stage 转发到下一个 stage 的 token 数，通常从 128 开始调试

RK3572 是单 SoC NPU，不使用本节的多张 RK182X device_id 绑定流程；在 RK3572 上应使用单模型的普通 Session 部署方式。

3.6.1 多卡拓扑和运行边界

分段边界、导出参数和产物命名统一见 2.2.6。本节只说明导出的 `model_segN.rknn` 如何映射到多张 RK182X 卡并参与推理：`stageN` 使用对应的 `segN` 模型和 `device_id[N]`，由主控侧线程、队列和回调连接成 Host pipeline。Runtime 不会自动把多个 context 合并成一个多卡模型。

逻辑数据流如下，箭头表示 Host 侧的 hidden states 转发：

```
prompt/token
    |
    v
stage0: seg0
    |  output_callback -> queue
    v
stage1: seg1
    |  output_callback -> queue
    v
...
    |
    v
stageN-1: segN-1
    |
    v  result_callback
logits -> sampling -> next token
```

多卡 pipeline 是模型并行，不是数据并行：不会在每张卡上复制完整模型，也不会把一批请求自动广播到所有卡。中间 stage 只转发 hidden states，最后 stage 负责 logits、采样和生成状态；当前 demo 通过 prefill 分桶提高流水利用率。

3.6.2 运行参数与资源约束

离线分段导出、RKNN 构建、文件命名和内存规划请按 2.2.6 执行；多卡运行时只负责把这些分段装载到指定设备。应用侧需要保持以下对应关系：

- `stage_count`、实际分段数量和参与运行的 RK182X 卡数一致。
- `device_id[i]`、模型路径和 `stage[i]` 使用相同的下标顺序；不能把设备顺序与 `segN` 顺序混用。
- `run_core_mask` 与构建该分段时的 core 配置匹配；它只决定单张卡内部的 NPU core，不会增加卡数。
- 所有 stage 使用一致的 `max_context_len`、词表信息和会话参数；`bucket_size` 只影响 prefill 的 Host 转发粒度。

Runtime 不会在启动时重新切分模型，也不会根据某一段的内存或计算负载自动迁移 stage。若分段内存不足或负载不均，应回到 2.2.6 调整分段数量、连续 block 切分或上下文长度后重新构建；`segment_plan.json` 和 `model_report.html` 用于离线规划和定位具体 stage 的资源瓶颈。

3.6.3 每个stage的Runtime初始化

每张卡都需要独立完成 context、模型、输出 tensor、session 和 callback 的初始化。推荐顺序如下：

顺序	调用或动作	关键要求
1	rknn3_find_devices() 或读取外部 device_id	设备数量不少于 stage_count；外部 ID 的顺序就是 stage0、stage1 ...
2	rknn3_init()	通过 rknn3_init_extend.device_id 将 context 绑定到目标卡
3	rknn3_load_model_from_path()	加载该 stage 的 .rknn 和 .weight
4	rknn3_model_init()	每个 stage 使用与转换匹配的 run_core_mask
5	注册可选 custom-op plugin	必须在对应 context 的 model_init 之后注册；多卡时对每个 stage context 分别注册

顺序	调用或动作	关键要求
6	查询 LLM 配置和输出属性、分配 output tensor	中间 stage 需要 output_callback 的输出缓冲区
7	rknn3_session_init()	各 stage 使用一致的 max_context_len、词表信息和 logits_name
8	rknn3_session_set_callback()	stage0 需要 tokenizer/embed；非最后 stage 需要 output_callback；使用外置 cache 时每个 stage 都需 input_callback

核心初始化代码结构如下，错误检查和输出 tensor 分配应按实际工程补齐：

```
for (size_t i = 0; i < stage_count; ++i) {
    rknn3_init_extend ext;
    memset(&ext, 0, sizeof(ext));
    ext.device_id = device_ids[i];
    rknn3_init(&ctx[i], &ext);

    rknn3_load_model_from_path(ctx[i], model_paths[i], weight_paths[i]);

    rknn3_config cfg;
    memset(&cfg, 0, sizeof(cfg));
    cfg.run_core_mask = run_core_mask;
    rknn3_model_init(ctx[i], &cfg);

    if (custom_op_path != NULL) {
        rknn3_register_custom_ops_plugins(
            ctx[i], custom_op_path, strlen(custom_op_path));
    }

    session[i] = rknn3_session_init(ctx[i], &llm_param, 1);
    rknn3_session_set_callback(session[i], &callback[i]);
}
```

应用可以只接收 stage0 的模型路径，再按 _seg0 后缀推导其余段路径，但必须在初始化前检查所有分段文件确实存在。stage_count、device_id 数量和实际 seg 数量不一致时，不应继续初始化。

3.6.4 Host侧数据转发和推理控制

多卡之间的中间数据由 Host 侧回调转发，典型实现包括：

1. stage0 以 PROMPT 或 TOKEN 输入运行；tokenizer_callback 把文本转成 token，embed_callback 从 embed.bin 查询 FP16 embedding。
2. 每个非最后 stage 的 output_callback 负责输出，将 hidden states 从 Runtime 输出缓冲区复制到带有 tensor 属性和 token 数量的队列对象。不能把 Runtime 自有输出缓冲区的指针直接交给下一个异步 worker 后再复用。
3. stage1 到 stageN-1 各有一个 worker，从上游队列取出 batch，以 RKNN3_LLM_INPUT_EMBED 作为当前 stage 的输入调用 rknn3_session_run()。
4. 每个 batch 携带实际 token 数；下游按 token 数计算 embedding 字节数，而不是假定每个 callback 都是固定 bucket_size。
5. 队列需要有 producer_done、failed 和 close 语义。任一 stage 失败时应唤醒并关闭下游队列，主线程 join 所有 worker 后再释放资源。

一次 pipeline step 建议使用以下 LLM 推理参数：

```
rknn3_llm_infer_param infer_param;
memset(&infer_param, 0, sizeof(infer_param));
infer_param.keep_history = 1;
infer_param.max_new_tokens = 1;
infer_param.prefill_only = true;
infer_param.disable_sampling = true;
```

prefill 时，stage0 的 prompt 可能被切成多个 bucket。中间 stage 只传播 hidden states，不采样；最后 stage 只有处理完本轮的最后一个 bucket 时才打开 sampling，避免每个 bucket 重复生成 token。decode 时，应用把上一步返回的一个 token 再次送入 stage0，每次 pipeline step 只生成一个 token。单卡模式没有下游 worker，stage0 同时是最后 stage，应直接关闭 disable_sampling。

每个 stage 都有自己的 KVCache，因此 prefill 和 decode 必须以相同的逻辑顺序经过所有 stage；清理上下文时也要对所有 session 调用 rknn3_session_clear_kvcache()，最后按“销毁 session -> 释放 tensor/mmap -> 销毁 context”的顺序释放资源。

3.6.5 外置输入和回调

外置 rope cache

Qwen3.5 和 Gemma4 的 multicard demo 默认可通过 safetensors 提供外置 rope cache。应用通常只需在 Host 侧 mmap 一份文件，然后对每个 stage：

1. 查询该 stage 的输入属性，找到 rope_cos_cache / rope_sin_cache 索引；若模型另有 per_layer_inputs 等外部输入，则按该模型的导出约定单独处理。
2. 注册 input_callback，并把这些索引传给 callback。
3. 在 callback 中按 LLMInputCallbackParam.pos 复制当前位置所需的 cache 片段。

不同模型的 tensor 名称和维度布局必须以导出的输入属性为准；demo 当前识别 Qwen3.5 的 rope_cos_cache / rope_sin_cache，以及 Gemma4 的编号 rope cache。未使用外置 rope 的模型不要盲目注册该 callback。

回调的职责划分

回调	多卡 demo 中的职责
tokenizer_callback	仅在需要文本输入时将 prompt 转成 token id
embed_callback	stage0 将 token id 映射为 embedding，并统计本轮 token 数
output_callback	非最后 stage 复制 hidden states 并入队；最后 stage 通常不向下游转发
result_callback	最后 stage 输出 token、更新 next_token 和生成状态
input_callback	填充外置 rope 或其它 per-layer external input

3.6.6 命令行部署示例

multicard demo 只需传入 seg0 的模型和权重路径，程序可根据 _seg0 后缀推导其它段。下面以四段 LLM 为例：

```
adb shell
cd /data/rknn_multicard_demo
export LD_LIBRARY_PATH=./lib
./rknn_multicard_demo --model /data/models/multicard/model_seg0.rknn --weight
/data/models/multicard/model_seg0.weight --vocab
/data/models/multicard/model.tokenizer.gguf --embed
/data/models/multicard/model.embed.bin --ctx-size 4096 --core-mask 0xff --stage-
count 4 --bucket-size 128 --predict 128 --device-id id0#id1#id2#id3
```

关键约束如下：

- stage_count 必须等于导出的 segment 数和使用的 RK182X 卡数；本例中需要 seg0 到 seg3 和 4 个可用设备。
- device-id 使用 # 分隔，顺序固定为 stage0 到 stage3；不指定时 demo 调用 rknn3_find_devices() 自动分配前 stage_count 个设备。
- core-mask 只控制每张卡内部的 NPU core，不表示使用多少张卡。
- 如果模型使用外置 rope，应增加 rope-tensor 参数；如果模型或图中其它节点需要运行时插件，应增加 custom-op-path 参数，且该插件会被每个 stage context 分别注册。
- performance mode 的输入长度、输出长度必须满足 input_tokens + output_tokens - 1 不超过所有 stage 的 max_context_len。

3.7 MatMul C API使用

> **注：暂不支持RK182X**

MatMul C API用于在应用程序中直接执行矩阵乘法：A为M×K矩阵，B为K×N矩阵，输出C为M×N矩阵。它适用于应用自行组织输入和权重、需要独立调用NPU矩阵乘法的场景，无需生成或加载RKNN模型，也无需创建Session。应用创建父 `rknn3_context` 和独立的 `rknn3_matmul_context`，通过 `rknn3_matmul_run()` 执行。

3.7.1 接口与运行环境

包含 `rknn3_api.h` 和 `rknn3_matmul_api.h`，使用包含MatMul接口的SDK头文件与对应库。RK3572使用native库。库的部署方法见3.1；类型、形状和资源要求应以实际接口返回值为准。

输入结构体及保留字段均需清零。一个MatMul句柄使用一个NPU核心，`core_mask` 选择单个核心；不要组合多个核心掩码。

desc.type	A	B	C	量化参数
RKNN3_MATMUL_W16A16	FP16	FP16	FP16	不需要
RKNN3_MATMUL_W4A16	FP16	INT4	FP16	仅B，沿K分组
RKNN3_MATMUL_W6A16	FP16	INT6	FP16	仅B，沿K分组
RKNN3_MATMUL_W8A16	FP16	INT8	FP16	仅B，沿K分组
RKNN3_MATMUL_W8A8_PER_CHANNEL	INT8	INT8	INT8	A/C按tensor，B按输出通道
RKNN3_MATMUL_W8A8_PER_LAYER	INT8	INT8	INT8	A/C按tensor，B按批次全层
RKNN3_MATMUL_W8A8O32	INT8	INT8	INT32	不需要，K不超过4096

W16A16同时支持普通权重B和双feature输入，双feature用法见3.7.3。

W4A16/W6A16的逻辑B采用解包存储，每个权重值占一个 `int8_t`，值域分别为 `[-8,7]` / `[-32,31]`；不能将位打包的数据直接作为逻辑B传入。分组量化类型的 `group_size` 设置为32、64或128，其余类型设0。

3.7.2 数据布局、内存与调用流程

数学矩阵维度与连续逻辑存储顺序如下。单批次时，A的索引为 `k*M+m`，B为 `k*N+n`，C为 `n*M+m`；不要直接将按 `m*K+k` 存储的普通行主序A作为逻辑输入。

Tensor	规范逻辑shape	连续NCHW索引
A	`[A_batch,K,1,M]`	`(batch*K+k)*M+m`
B	`[B_batch,K,1,N]`	`(batch*K+k)*N+n`
C	`[C_batch,N,1,M]`	`(batch*N+n)*M+m`

`A_batch` / `B_batch` 均大于0，`C_batch=max(A_batch,B_batch)`，且必须能被两者整除。下文示例使用单批次。

`rknn3_matmul_query_infos()` 返回各shape的A/B/C属性，包括布局、物理步长、字节数、对齐和核心编号。NPU端属性直接使用查询结果；逻辑端属性从零填写 `n_dims=4`、shape、dtype、NCHW布局和连续数据字节数。A/C的NPU布局为NC1HWC2；普通B在 `desc.B_layout=NCHW` 时查询得到打包权重布局O1I1HWI2O2。

MatMul C API的典型调用流程如下图，具体操作见下表。

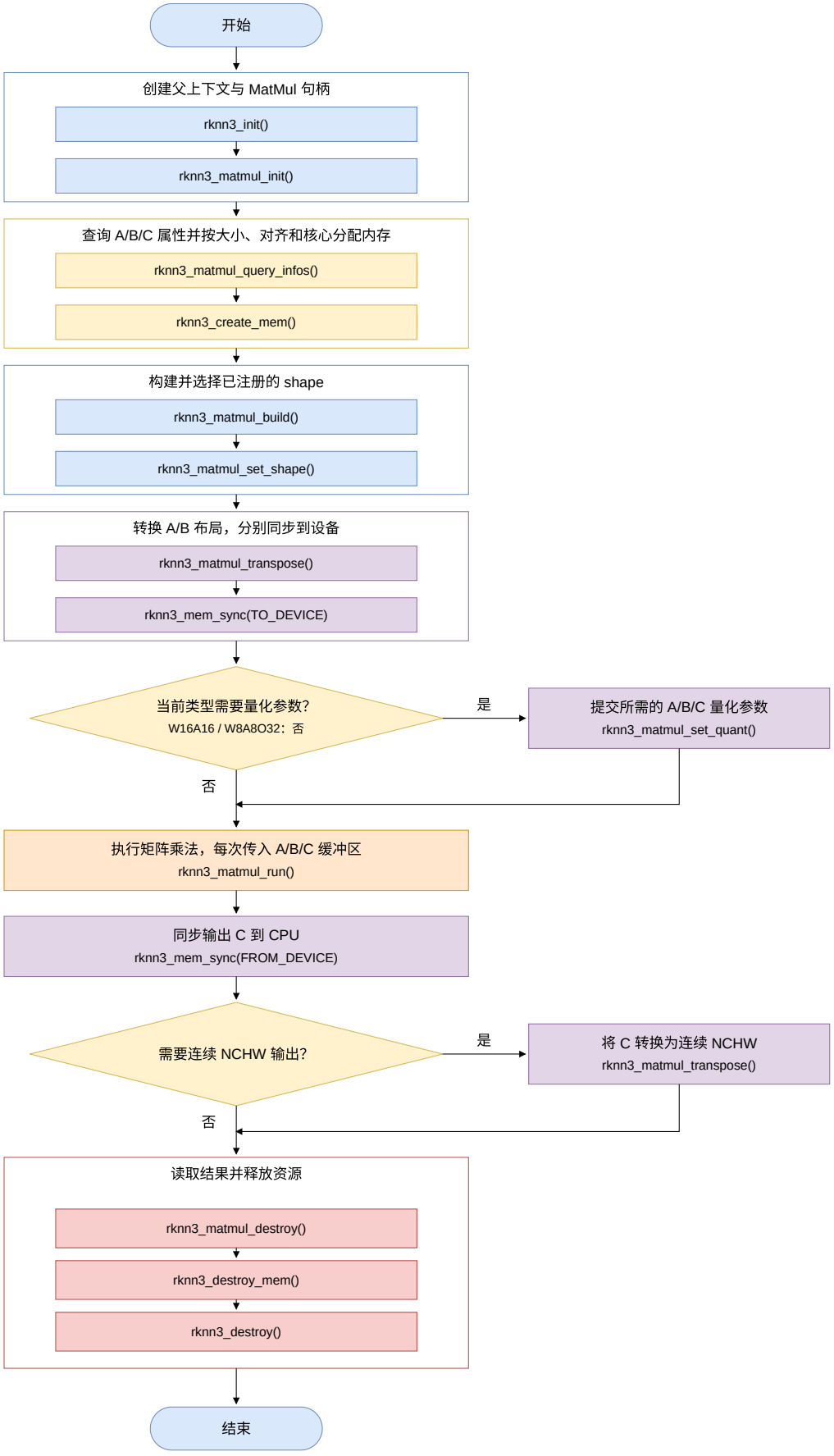


图3-7 MatMul C API调用流程

步骤	接口与操作
1. 初始化	`rknn3_init()` 创建父上下文；`rknn3_matmul_init()` 注册一个或多个shape。
2. 查询与分配	`rknn3_matmul_query_infos()`；按各属性的size、alignment、core_id调用 `rknn3_create_mem()` 分配A/B/C。
3. 构建与选形状	`rknn3_matmul_build()`；`rknn3_matmul_set_shape()` 选择注册数组中的shape索引。
4. 准备输入	`rknn3_matmul_transpose()` 转换A/B到查询布局；分别调用 `rknn3_mem_sync(..., TO_DEVICE)`。
5. 提交量化	需要量化的类型调用 `rknn3_matmul_set_quant()`，详见3.7.4。
6. 执行	`rknn3_matmul_run(matmul, a_mem, b_mem, c_mem)`，每次均传入A/B/C。
7. 读取结果	成功后同步C到CPU；需要连续NCHW输出时再调用 `rknn3_matmul_transpose()`。
8. 释放	仅在 `rknn3_matmul_destroy()` 成功后，才能释放A/B/C缓冲区，再销毁父上下文；销毁失败时应保留这些资源，详见3.7.6。

分配大小不能仅按M×K、K×N或M×N计算，NPU布局包含填充。保证 `phys_addr + offset` 按查询的alignment对齐，并且缓冲区从offset起的剩余容量不少于属性size。CPU与NPU使用同一offset。`transpose()` 负责布局转换，不代替显式内存同步；cacheable缓冲区在CPU写入后同步到设备，CPU读取输出前从设备同步。C是输出缓冲区，执行前无需填写输出数据。

3.7.3 W16A16核心调用示例

以下片段展示M=16、K=128、N=64的成功调用流程。省略了内存分配、逻辑属性与数据准备、返回值检查和失败清理，不能独立编译。实际应用应检查每次调用的返回值，仅在成功后继续下一步。`am/bm/cm` 为按查询结果分配并对齐的NPU缓冲区；`aa/ba/ca` 与 `al/bl/cl` 分别为连续NCHW逻辑属性和内存描述符，准备规则见3.7.2。W16A16无需提交量化参数。

```
rknn3_context ctx = 0;
rknn3_matmul_context mm = 0;
rknn3_matmul_desc desc = {0};
rknn3_matmul_info info = {0};

desc.M = 16;
desc.K = 128;
desc.N = 64;
desc.A_batch = 1;
desc.B_batch = 1;
desc.type = RKNN3_MATMUL_W16A16;
desc.A_layout = RKNN3_TENSOR_NCHW;
desc.B_layout = RKNN3_TENSOR_NCHW;
desc.C_layout = RKNN3_TENSOR_NCHW;
desc.core_mask = RKNN3_NPU_CORE_0;

rknn3_init(&ctx, NULL);
rknn3_matmul_init(ctx, &desc, 1, &mm);
rknn3_matmul_query_infos(mm, &info, 1);

/* Allocate aligned am/bm/cm and prepare logical aa/ba/ca and al/bl/cl. */
rknn3_matmul_build(mm, NULL);
rknn3_matmul_set_shape(mm, 0);

rknn3_matmul_transpose(mm, &aa, &al, &info.A, am);
rknn3_mem_sync(ctx, am, RKNN3_MEMORY_SYNC_TO_DEVICE);

rknn3_matmul_transpose(mm, &ba, &bl, &info.B, bm);
rknn3_mem_sync(ctx, bm, RKNN3_MEMORY_SYNC_TO_DEVICE);

/* W16A16 requires no set_quant call. */
rknn3_matmul_run(mm, am, bm, cm);

rknn3_mem_sync(ctx, cm, RKNN3_MEMORY_SYNC_FROM_DEVICE);
rknn3_matmul_transpose(mm, &info.C, cm, &ca, &cl);

/* Read logical C; destroy the MatMul handle before releasing its buffers. */
if (rknn3_matmul_destroy(mm) == RKNN3_SUCCESS) {
    rknn3_destroy_mem(ctx, am);
    rknn3_destroy_mem(ctx, bm);
    rknn3_destroy_mem(ctx, cm);
    rknn3_destroy(ctx);
}
```

应用编译时使用公开SDK中的 `rknn3_api.h` 和 `rknn3_matmul_api.h`。RK3572链接 `librknn3_api.so` 与 `librknn3_api_native.so`。程序与对应库的部署和库搜索路径设置见3.1。

W16A16双feature（A/B均为特征输入）

双feature仍使用 `RKNN3_MATMUL_W16A16`，A/B/C均为FP16，无需 `set_quant()`。初始化前将 `desc.B_layout` 设置为 `RKNN3_TENSOR_NC1HWC2`，A_layout/C_layout仍设置为NCHW，描述连续逻辑输入/输出。查询得到的A、B、C的NPU布局均为NC1HWC2；普通权重B的查询布局则为O1I1HWI2O2。

这种用法适合B也作为运行时特征输入的场景。保持前述维度、类型和核心设置，使用以下配置与输入准备流程。代码仍省略返回值检查和内存准备；输出同步、转换及资源释放与前例相同。

```
/* Set these fields before init; keep the other W16A16 fields unchanged. */
desc.A_layout = RKNN3_TENSOR_NCHW;
desc.C_layout = RKNN3_TENSOR_NCHW;
desc.B_layout = RKNN3_TENSOR_NC1HWC2;

rknn3_matmul_init(ctx, &desc, 1, &mm);
rknn3_matmul_query_infos(mm, &info, 1);

/* Allocate A/B/C using this query; build and select shape 0. */
rknn3_matmul_build(mm, NULL);
rknn3_matmul_set_shape(mm, 0);

/* Prepare logical aa/ba/al/bl; convert both inputs to feature layouts. */
rknn3_matmul_transpose(mm, &aa, &al, &info.A, am);
rknn3_mem_sync(ctx, am, RKNN3_MEMORY_SYNC_TO_DEVICE);

rknn3_matmul_transpose(mm, &ba, &bl, &info.B, bm);
rknn3_mem_sync(ctx, bm, RKNN3_MEMORY_SYNC_TO_DEVICE);

rknn3_matmul_run(mm, am, bm, cm);
```

逻辑B仍按 `[B_batch,K,1,N]` 的连续NCHW存储准备，`ba.layout` 设为NCHW，转换目标使用新查询的 `info.B`；不要沿用普通权重B的打包属性或缓冲区容量。如果A/B已在NC1HWC2中，且shape、dtype、步长、容量、对齐和核心均满足查询要求，可直接传入相应NPU缓冲区，省去逻辑布局转换。CPU写入后的cacheable输入仍须同步到设备。

双feature当前仅支持单shape W16A16。每次run都传入本次A/B/C；更新B后，完成必要的布局转换与同步再执行。该模式的B不能使用3.7.5的固定B多shape流程。

3.7.4 量化参数准备

先从 `info.A/B/C.quant_info` 复制量化要求，再填写scale/zp数组及字节数。查询只返回格式与数量，不产生实际量化值。数组分别连续存储，不交错、不按NPU布局补齐；`scales[i]` 与 `zero_points[i]` 对应同一量化单元。

令 `G=ceil(K/group_size)`，各类型的数组组织如下：

类型 / Tensor	每个数组的逻辑shape	参数索引	scale元素
W4A16/W6A16/W8A16，B	`[B_batch,G,N]`	`(batch*G+k/group_size)*N+n`	FP16位模式，2字节
W8A8 per-channel，B	`[B_batch,N]`	`batch*N+n`	FP32，4字节
W8A8 per-layer，B	`[B_batch]`	`batch`	FP32，4字节
W8A8，A/C各自	`[1]`	`0`，覆盖全部元素和批次	FP32，4字节

全部zp使用解包 `int32_t` 存储，每个4字节，即使查询的 `zero_point_dtype` 为INT4/INT8也不改变数组存储类型。`scales_size` / `zero_points_size` 是字节数：分别为 `num_quant_params * scale元素字节数` 和 `num_quant_params * sizeof(int32_t)`。scale必须为有限正数；零点为0时仍提供全0的zp数组。

例如，分组量化B的 `B_batch=1,K=64,N=64,group_size=32`，则两个数组各有128个元素，FP16 scales占256字节、int32 zp占512字节；索引0..63对应第一组的64个输出通道，64..127对应第二组。

- W4A16/W6A16/W8A16调用 `set_quant(matmul, NULL, &qb, NULL)`，仅提交B参数。
- W8A8 per-channel / per-layer调用 `set_quant(matmul, &qa, &qb, &qc)`，提交A/B/C参数。

- W16A16/W8A8O32不调用 `set_quant()`。

提交应在build和set_shape之后、run之前完成；B布局转换与量化提交的先后顺序独立。K不能整除group_size时，最后不足一组的有效K仍需一整组N个参数；数量以查询的 `num_quant_params` 为准。详细量化值约束见《RKNN3_Runtime_C_API_参考》。

3.7.5 共享固定B的多shape使用

一个句柄可注册多个shape，通过 `set_shape(matmul, shape_id)` 切换。只有M和A_batch可以不同；type、core_mask、B_batch、K、N、group_size、B_layout及查询得到的B大小和布局必须一致。使用普通权重B（`desc.B_layout=NCHW`）；feature B（NC1HWC2）仅用于单shape W16A16。

例如K=128、N=64固定，注册M=1和M=16两个shape：

1. 将两个desc传入一次init，查询 `infos[2]`，完成一次build。
2. B转换、同步一次；A/C可按所有shape的最大size预分配，但须满足每个shape的对齐和核心要求。
3. 每次set_shape后，按 `infos[shape_id].A` 准备和同步当前A，复用固定B执行run。
4. 按当前shape同步并处理C，完成输出转换或保存后再切换下一shape。

量化提交规则：A16分组量化在整个句柄上成功提交一次B参数即可；W8A8需先为每个shape分别选择形状并提交A/B/C参数，不同shape的B量化值一致，A/C量化值可不同。准备完成后，切换shape无需重新提交量化值。W16A16/W8A8O32无需量化设置。

3.7.6 返回值、资源生命周期与文档入口

每次调用检查返回值，初始化拒绝的类型/shape不能继续构建或执行。`run()` 为同步接口，成功返回后才能处理本次C；失败时不能将C作为有效结果。同一句柄的形状选择、转换、量化、执行和输出转换应串行组织。

若返回 `RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN`，保留MatMul句柄、父上下文及A/B/C，禁止修改、复用或释放这些缓冲区。仅在 `rknn3_matmul_destroy()` 成功后才能释放相关资源；destroy失败时不得继续释放A/B/C或父上下文。集成应用应持有这些资源直至句柄销毁成功。

详细说明见《RKNN3_Runtime_C_API_参考》中的 MatMul 相关章节。

4. 高级功能与扩展

本章介绍 RKNN3 的高级功能与扩展能力，包括 Runtime 层的多核、动态 Shape、内存管理，LLM / MLLM 层的多种输入、回调、Function Calling、KVCache / LoRA 等，以及后处理插件、自定义算子、模型加密等扩展与安全能力。这些功能面向进阶部署与优化场景，基础部署可跳过本章。

4.1 Runtime高级功能

4.1.1 NPU多核

适用场景

RK1820 / RK1828 协处理器 NPU 含 8 个核心（RK3572 为单核）。多核运行将计算拆分到多个核心并行执行，适用于网络层计算量较大的模型（如大参数 LLM、高分辨率视觉模型）；对小网络可能因算子在核间切换需 CPU 介入而导致性能下降，建议单核。

使用方式

多核需在模型转换和板端推理两端配合配置：

- **转换侧**：`rknn.config(core_num=N)` 指定模型占用的核心数。`core_num=0` 为自动模式（`load_llm` 加载的模型用 8 核，其余用 1 核）。
- **推理侧**：`core_mask` 必须与转换时 `core_num` 匹配，通过 `rknn3_config.run_core_mask`（C）或 `init_runtime(core_mask=)`（Python）设置，范围 0x1~0xff。

```
/* 查询核心数并构造掩码 */
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));
uint32_t core_mask = 0;
for (uint32_t i = 0; i < core_num; i++) core_mask |= (1 << i);

rknn3_config config;
memset(&config, 0, sizeof(rknn3_config));
config.run_core_mask = core_mask;
rknn3_model_init(ctx, &config);
```

> **说明**：CNN 模型建议 `core_num=1`；Transformer 类模型按需设置。可通过 `RKNN3_QUERY_CORE_MEM_SIZE` 查询每核心内存占用，辅助内存规划。RK3572 仅支持单核（`core_mask=0x1`）。
>
> **多卡区别**：core_num/core_mask 只决定一张卡内部使用多少个 NPU core；多卡模型的卡数由分段数和 stage_count 决定，卡的选择由 device_id 决定。将 core_mask 设为 0xff 不会自动启用第二张 RK182X 卡。

4.1.2 动态Shape

适用场景

动态 Shape 允许同一模型在运行时切换输入尺寸，用一份模型替代多份静态模型，节省 Flash/DDR 占用，适用于输入尺寸多变的场景（如可变分辨率检测、多档位图像分类）。

使用前提

- **模型需支持动态 shape**：若原始模型本身不是动态 shape，RKNN3-Toolkit 支持将其扩展为动态 shape 模型，但模型不能存在限制动态 shape 的算子或子图结构（如常量的形状无法改变），否则转换过程会报错。遇到不支持扩展的情况，需根据报错信息修改模型结构、重新训练。建议直接使用原始就是动态 shape 的模型。
- **shape 数量有限**：由于 NPU 硬件特性，动态 shape 模型不支持输入形状任意改变，要求用户在转换时显式枚举有限个输入形状。多输入模型每个输入的 shape 个数必须相同，并按顺序一一对应，不支持交叉组合。

使用方式

- **转换侧**：`rknn.config(dynamic_input=[...])` 配置多组输入 shape，每组以 `rknn3_shape_info` 描述。量化时用所有尺寸之和最大的那组 shape，所有 shape 共用一套量化参数。

```
dynamic_shapes = [
    [[1, 3, 224, 224]],
    [[1, 3, 192, 192]],
    [[1, 3, 160, 160]],
]
rknn.config(mean_values=[[0,0,0]], std_values=[[255,255,255]],
            dynamic_input=dynamic_shapes)
```

- **推理侧**：模型默认使用 Shape ID 0。切换时需先释放旧内存再设置新 ID，重新分配并同步：

```
rknn3_destroy_mem(ctx, old_mem);              /* 释放旧 shape 内存 */
rknn3_set_shape(ctx, shape_id);               /* 切换 shape */
rknn3_create_mem(ctx, ...);                   /* 按新 shape 分配内存 */
rknn3_mem_sync(ctx, mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
```

通过 `RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG` / `RKNN3_QUERY_DYNAMIC_SHAPE_INFO` 查询支持的 shape 列表。

> **限制**：运行时切换 shape 必须遵循"释放旧内存 -> `rknn3_set_shape()` -> 重新分配内存 -> 同步"的顺序；动态 Shape 模型不支持权重共享与上下文复用（`rknn3_dup_context()`）。极端尺寸组合下因共用一套量化参数可能影响精度。

4.1.3 Cacheable内存一致性

适用场景

CPU 与 NPU 有独立 Cache。当两者通过带 Cache 属性的共享内存交换数据时，若未同步会导致数据不一致、推理错误。Cacheable 内存一致性机制通过 `rknn3_mem_sync()` 在数据流转向时显式刷新 Cache，保证一致性。

使用方式

通过 `rknn3_create_mem()` 的 flags 指定内存属性：`RKNN3_FLAG_MEMORY_CACHEABLE`（带 Cache）或 `RKNN3_FLAG_MEMORY_NON_CACHEABLE`（不带 Cache）。带 Cache 的内存在 CPU 与 NPU 之间传递数据时须按流向同步：

数据流向	同步模式	时机
CPU 写 -> NPU 读	`RKNN3_MEMORY_SYNC_TO_DEVICE`	推理前（输入）
NPU 写 -> CPU 读	`RKNN3_MEMORY_SYNC_FROM_DEVICE`	推理后（输出）

使用建议：

1. 当确定数据流向时，推荐使用单方向同步，可减少多余 Cache 刷新操作，提高推理性能。
2. 在向设备发送输入数据时，调用 `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_TO_DEVICE)`。
3. 在从设备读取输出数据时，调用 `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_FROM_DEVICE)`。

```
rknn3_tensor_mem* mem = rknn3_create_mem(ctx, size, 0, RKNN3_FLAG_MEMORY_CACHEABLE);
/* 填充输入数据后，推理前同步到设备 */
rknn3_mem_sync(ctx, mem, RKNN3_MEMORY_SYNC_TO_DEVICE);
rknn3_run(ctx, ...);
/* 推理后从设备同步回 CPU 读取输出 */
rknn3_mem_sync(ctx, mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
```

4.1.4 Internal内存复用

适用场景

在多模态模型（如 VLM）部署时，视觉编码器与 LLM 往往串行执行。两者之间通过复用同一段 Internal 内存，可显著降低设备上的整体内存占用，有效缓解内存资源有限的瓶颈。

其核心思路为“时间分片 + 空间复用”：在视觉模型完成推理并将图像特征传递至 LLM 后，其所占用的 Internal 内存将被释放并复用给 LLM 的中间计算，避免为两个模型分别开辟独立的内存空间。

该策略特别适用于以下场景：

- **多模型流水线部署**：如 VLM、语音识别 + 大语言模型等组合式推理任务。
- **内存受限的边缘设备**：需在有限内存下运行多阶段模型。
- **多模态交互应用**：在兼顾响应速度的同时，需最大化内存利用效率。

使用方式

前提：`rknn3_config.user_mem_internal=1`，internal 内存由用户分配。流程（以串行执行的两个模型 A、B 为例）：

1. 分别用 `RKNN3_QUERY_CORE_NUMBER` 查询两模型的核心数，再用 `RKNN3_QUERY_CORE_MEM_SIZE` 查询各核心的 internal 内存大小（按核心数分配 `rknn3_core_mem_size` 数组接收）；
2. 遍历两模型的核心，按 `core_id` 匹配找出运行在相同物理核心的部分，取 `max(A_internal, B_internal)` 作为共用大小；
3. 总内存块数 = A 核心数 + B 核心数 - 重叠核心数；
4. 对复用核心，B 的 internal_mems 指针直接指向 A 已分配的内存；非复用核心单独分配（统一用 A 的 ctx 分配，便于后续统一释放、避免重复释放）；
5. 用 `rknn3_set_internal_mem()` 将内存分别设置给两模型，临时指针数组可释放，internal 内存块本身由 `rknn3_destroy_mem()` 释放。

以下是一个 VLM 实现 Internal 内存复用的示例代码：

```
...
int ret = -1;

uint32_t core_num_vision = 0;
uint32_t core_num_llm = 0;
//获取vision模型的core_num
ret = rknn3_query(app_ctx->vision.rknn_ctx, RKNN3_QUERY_CORE_NUMBER,
&core_num_vision, sizeof(core_num_vision));
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query failed! ret=%d\n", ret);
    return ret;
}
//获取llm模型的core_num
ret = rknn3_query(app_ctx->llm.rknn_ctx, RKNN3_QUERY_CORE_NUMBER, &core_num_llm,
sizeof(core_num_llm));
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query failed! ret=%d\n", ret);
    return ret;
}

rknn3_core_mem_size* core_mem_sizes_vision =
(rknn3_core_mem_size*)malloc(sizeof(rknn3_core_mem_size) * core_num_vision);
if (!core_mem_sizes_vision) {
    printf("Failed to allocate memory for core_mem_sizes_vision\n");
    return ret;
}
rknn3_core_mem_size* core_mem_sizes_llm =
(rknn3_core_mem_size*)malloc(sizeof(rknn3_core_mem_size) * core_num_llm);
if (!core_mem_sizes_llm) {
    printf("Failed to allocate memory for core_mem_sizes_llm\n");
    return ret;
}
//获取vision在各个npu核心上模型占用weight和internal内存大小
ret = rknn3_query(app_ctx->vision.rknn_ctx, RKNN3_QUERY_CORE_MEM_SIZE,
core_mem_sizes_vision, sizeof(rknn3_core_mem_size) * core_num_vision);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query core memory size failed! ret=%d\n", ret);
    return ret;
}
//获取llm在各个npu核心上模型占用weight和internal内存大小
ret = rknn3_query(app_ctx->llm.rknn_ctx, RKNN3_QUERY_CORE_MEM_SIZE,
core_mem_sizes_llm, sizeof(rknn3_core_mem_size) * core_num_llm);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_query core memory size failed! ret=%d\n", ret);
    return ret;
}

int llm_to_vision[core_num_llm];
for (int i = 0; i < core_num_llm; i++)  { llm_to_vision[i] = -1; }

// 遍历视觉模型的核心和 LLM 的核心，寻找运行在相同npu核心上的情况
int core_num_same = 0;
for (int i = 0; i < core_num_vision; i++) {
    for (int j = 0; j < core_num_llm; j++) {
        if (core_mem_sizes_vision[i].core_id == core_mem_sizes_llm[j].core_id) {
            uint64_t internal_size =
std::max(core_mem_sizes_vision[i].internal_size,
core_mem_sizes_llm[j].internal_size);
            core_mem_sizes_vision[i].internal_size = internal_size;
            core_mem_sizes_llm[j].internal_size = internal_size;
            core_num_same ++;
```

```
            llm_to_vision[j] = i;
            break;
        }
    }
}

// 为所有实际需要分配的 Internal 内存块指针申请存储空间，总共需要分配的 Internal 内存块数 = 视觉
模型数量 + LLM 数量 - 重叠的核心数（复用的部分）
app_ctx->n_internal_mems = core_num_vision + core_num_llm - core_num_same;
app_ctx->internal_mems = (rknn3_tensor_mem**)calloc(app_ctx->n_internal_mems,
sizeof(rknn3_tensor_mem*));
if (!app_ctx->internal_mems) {
    printf("Failed to allocate memory for app_ctx->internal_mems array\n");
    return -1;
}
// 为视觉模型每个核心分配指针数组（用于后续设置模型 internal 内存）
rknn3_tensor_mem** internal_mems_vision =
(rknn3_tensor_mem**)calloc(core_num_vision, sizeof(rknn3_tensor_mem*));
if (!internal_mems_vision) {
    printf("Failed to allocate memory for internal_mems_vision array\n");
    return -1;
}
// 为 LLM 每个核心分配指针数组（用于后续设置模型 internal 内存）
rknn3_tensor_mem** internal_mems_llm = (rknn3_tensor_mem**)calloc(core_num_llm,
sizeof(rknn3_tensor_mem*));
if (!internal_mems_llm) {
    printf("Failed to allocate memory for internal_mems_llm array\n");
    return -1;
}

// 分配内存并初始化视觉模型的 internal 内存块
int idx = 0;
for (uint32_t i = 0; i < core_num_vision; i++) {
    internal_mems_vision[i] = rknn3_create_mem(app_ctx->vision.rknn_ctx,
core_mem_sizes_vision[i].internal_size, core_mem_sizes_vision[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
    if (!internal_mems_vision[i]) {
        return -1;
    }
    printf("Created user internal memory for core %d: size=%lu, virt_addr=%p,
phys_addr=0x%lx\n", core_mem_sizes_vision[i].core_id,
           internal_mems_vision[i]->size, internal_mems_vision[i]->virt_addr,
internal_mems_vision[i]->phys_addr);
    app_ctx->internal_mems[idx++] = internal_mems_vision[i];
}

// 为 LLM 分配 internal 内存（复用或新分配）
for (uint32_t i = 0; i < core_num_llm; i++) {
    if (llm_to_vision[i] != -1) {
        // 复用：LLM 核心与视觉模型核心在同一物理核心上，直接指向已分配的内存
        internal_mems_llm[i] = internal_mems_vision[llm_to_vision[i]];
        continue;
    }
    //不满足则需要为该 LLM 核心单独分配内存，这里使用 vision.rknn_ctx 分配，是为了后续统一释放
（避免重复释放问题）
    internal_mems_llm[i] = rknn3_create_mem(app_ctx->vision.rknn_ctx,
core_mem_sizes_llm[i].internal_size, core_mem_sizes_llm[i].core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
    if (!internal_mems_llm[i]) {    // 都使用vision.rknn_ctx分配，方便后面释放
        return -1;
    }
    printf("Created user internal memory for core %d: size=%lu, virt_addr=%p,
```

```
phys_addr=0x%lx\n", core_mem_sizes_llm[i].core_id,
           internal_mems_llm[i]->size, internal_mems_llm[i]->virt_addr,
internal_mems_llm[i]->phys_addr);
    // 将新分配的内存块也加入全局管理数组
    app_ctx->internal_mems[idx++] = internal_mems_llm[i];
}

// 将分配好的 internal 内存设置给视觉模型
ret = rknn3_set_internal_mem(app_ctx->vision.rknn_ctx, internal_mems_vision,
core_num_vision);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_set_internal_mem failed! ret=%d\n", ret);
    return ret;
}
// 将分配好的 internal 内存（含复用部分）设置给 LLM 模型
ret = rknn3_set_internal_mem(app_ctx->llm.rknn_ctx, internal_mems_llm,
core_num_llm);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_set_internal_mem failed! ret=%d\n", ret);
    return ret;
}

// 释放临时分配的指针数组（internal 内存块本身由 rknn3_create_mem 申请，需后续
rknn3_destroy_mem 释放）
free(internal_mems_vision);
free(internal_mems_llm);
free(core_mem_sizes_vision);
free(core_mem_sizes_llm);

return ret;
```

> **说明**：复用仅在两模型串行执行（同一时刻只有一个运行）时安全；若并发执行则不可复用。

4.2 LLM和MLLM高级功能

4.2.1 Prompt、Token和Embedding输入

适用场景

LLM / MLLM 推理的文本类输入支持三种类型，通过 `rknn3_llm_input.input_type` 指定，对应不同的数据准备阶段。按实际数据形态选择可减少预处理开销：

输入类型	适用场景	数据准备阶段
`RKNN3_LLM_INPUT_PROMPT`	通用文本输入，由 SDK 完成 tokenizer 分词	自然语言文本
`RKNN3_LLM_INPUT_TOKEN`	已完成分词，跳过文本->token 转换	token id 序列
`RKNN3_LLM_INPUT_EMBED`	已完成嵌入，跳过文本->token->embedding	embedding 向量

使用方式

三种类型统一通过 `llm_input`（`rknn3_llm_tensor`）传入，填入 `rknn3_llm_input` 数组后交给 `rknn3_session_run()`，按类型填充对应字段。三者互斥，同一 `rknn3_llm_input` 只能填一种。

- **PROMPT 输入**：直接输入自然语言文本，SDK 完成 tokenizer 转换，推荐大部分通用场景。

```
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = prompt,
                                 .embed = NULL, .tokens = NULL,
                                 .n_tokens = 0, .enable_thinking = false};
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input  = input_tensor;
```

- **TOKEN 输入**：直接输入 token id 序列，适用于已完成分词、希望跳过文本->token 转换的场景（如复用历史 token、自定义分词）。

```
int32_t tokens[6] = {151628, 151629, 151630, 151631, 151632, 151633};
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = NULL,
                                 .embed = NULL, .tokens = tokens,
                                 .n_tokens = 6, .enable_thinking = false};
input.input_type = RKNN3_LLM_INPUT_TOKEN;
input.llm_input  = input_tensor;
```

- **EMBED 输入**：直接输入 embedding 向量（大小为 `n_tokens * embedding_dim * sizeof(float16)`），跳过文本->token->embedding 流程，适用于无需传统分词流程的模型或自定义嵌入。

```
float16* embed = (float16*)malloc(n_tokens * embedding_dim * sizeof(float16));
/* 填充 embed 数据 */
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = NULL,
                                 .embed = embed, .tokens = NULL,
                                 .n_tokens = n_tokens, .enable_thinking = false};
input.input_type = RKNN3_LLM_INPUT_EMBED;
input.llm_input  = input_tensor;
```

> **说明**：结构体成员详见 3.1.2。PROMPT/TOKEN/EMBED 互斥使用，同一 `rknn3_llm_input` 只能填一种。

4.2.2 Multimodal和Aux输入

适用场景

MLLM 推理除文本类输入（见 4.2.1）外，还支持多模态输入与辅助输入，适配多模态融合与特殊模型的扩展输入需求：

输入类型	适用场景	数据准备阶段
`RKNN3_LLM_INPUT_MULTIMODAL`	多模态输入，支持图像/音频/视频单独或组合	各模态编码模型输出的嵌入
`RKNN3_LLM_INPUT_AUX`	其他类型输入，配合主输入用于特殊模型（如 Qwen3-VL 的 3 个 deepstack 辅助输入）	辅助张量

使用方式

- **MULTIMODAL 输入**：使用 `rknn3_llm_multimodal_tensor` 结构（`rknn3_llm_input.multimodal_input`），按实际模态填充 `image` / `audio` / `video` 子结构，未涉及模态不设置。每个子结构包含编码模型输出的嵌入指针、token 数量、模态数量及文本占位标记。

prompt 中用 `<image>` / `<audio>` / `<video>` 标签占位，数量须与 `n_image` / `n_audio` / `n_video` 一致。占位标记（如 `<|vision_start|>` 或 `<|vision_bos|>`）因模型 tokenizer 而异，须以实际模型 tokenizer 的特殊 token 为准。

```
rknn3_llm_multimodal_tensor tensor;
memset(&tensor, 0, sizeof(tensor));
tensor.prompt = "1.<image> 2.<audio> 3.<video> 请分析以上内容";
tensor.image.image_embed   = vision_output;     /* n_image × n_image_tokens ×
embedding_dim */
tensor.image.n_image_tokens = 196;
tensor.image.n_image        = 1;
tensor.image.image_start    = "<|vision_bos|>";
tensor.image.image_end      = "<|vision_eos|>";
tensor.image.image_content  = "<|IMAGE|>";
/* audio/video 子结构按相同方式填充 */
input.input_type      = RKNN3_LLM_INPUT_MULTIMODAL;
input.multimodal_input = tensor;
```

- **AUX 输入**：其他类型输入，配合主输入提供辅助张量，用于特殊模型的扩展输入。以 Qwen3-VL-2B 为例，除 1 个 MULTIMODAL 主输入外还需 3 个 deepstack 辅助输入，共 4 个 `rknn3_llm_input`。每个 AUX 通过 `rknn3_aux_tensor.mem`（由 `rknn3_create_mem()` 分配）承载辅助数据：

```
/* 主输入：MULTIMODAL，占 llm_inputs[0] */
/* 辅助输入：3 个 AUX，占 llm_inputs[1..3] */
rknn3_aux_tensor aux_input_tensor0;
rknn3_llm_input aux_input0;

aux_input_tensor0.mem = rknn3_create_mem(ctx, n * embedding_dim * sizeof(float16),
                                         0, RKNN3_FLAG_MEMORY_CACHEABLE);
aux_input0.input_type = RKNN3_LLM_INPUT_AUX;
aux_input0.aux_input  = aux_input_tensor0;
llm_inputs[1] = aux_input0;
/* aux_input1、aux_input2 同理 */
```

> **说明**：各模态占位标记须与模型 tokenizer 特殊 token 一致。MULTIMODAL 结构成员详见 3.1.2，完整定义见《RKNN3_Runtime_C_API_参考》5.5.25，填充示例见 3.3.6。AUX 不能单独使用，须与主输入组合在同一 `llm_inputs` 数组中传入 `rknn3_session_run()`。

4.2.3 Callback机制

适用场景

LLM 推理的 tokenization、embedding 检索、采样、结果输出等环节通过回调机制交由用户实现，提供灵活性。六类回调统一通过 `RKLLMCallback` 结构体配置，用 `rknn3_session_set_callback()` 注册：

回调	作用	必需性
`result_callback`	每步生成 token 后调用，处理输出（如转文本打印）	必需
`tokenizer_callback`	文本转 token id 序列	可选（不设则用默认）
`embed_callback`	按 token id 检索 embedding 填入 buffer	可选（Tie Word 模式下不需）

回调	作用	必需性
`sampling_callback`	自定义采样策略（argmax/top-k/top-p/temperature）	可选（不设则用默认采样）
`output_callback`	取回输出 tensor（logits 等），需预配置 output_tensors	可选
`input_callback`	推理前修改/填充输入 tensor（如 Per-Layer Embeddings）	可选

每个回调配 `*_userdata` 传入上下文（如 tokenizer 实例、embedding_info）。

```
RKLLMCallback callback;
memset(&callback, 0, sizeof(RKLLMCallback));
callback.result_callback    = result_callback;
callback.tokenizer_callback = tokenizer_callback;
callback.embed_callback     = embed_callback;
callback.sampling_callback  = sampling_callback;
rknn3_session_set_callback(session, &callback);
```

`result_callback` 按 `LLMCallState` 处理不同阶段（`RKLLM_RUN_NORMAL` 正常生成、`RKLLM_RUN_FINISH` 完成、`RKLLM_RUN_ERROR` 出错、`RKLLM_RUN_PAUSE/RESUME` 暂停恢复等）。

> **说明**：`output_callback` 需额外设置 `callback.output_tensors` 与 `n_output_tensors`。完整回调签名与参数见《RKNN3_Runtime_C_API_参考》。

4.2.4 Function Calling

适用场景

Function Calling 允许推理中由模型输出触发预设函数工具，实现与外部系统数据交互（如天气查询、日程提醒、数据库检索），扩展 LLM 的能力边界。

使用方式

调用 `rknn3_session_set_function_tools()` 设置一组工具（JSON 描述函数名、参数定义），推理时模型据会话模板判断是否发起调用，并在输出中提供调用请求。调用流程为两轮推理：

1. **第一轮**（role="user"）：模型生成函数调用请求，解析输出得 tool_call_list，实现实际函数调用，结果存为 tool_prompt；
2. **第二轮**（role="tool"）：将函数调用结果反馈给模型，模型处理后返回最终结果。

```
/* 设置函数工具 */
char* function_tools = R"([{"type":"function","function":{
    "name":"get_current_temperature","description":"...",
    "parameters":{"type":"object","properties":{...},"required":[...]}}}])";
rknn3_session_set_function_tools(session, function_tools);

/* 第一轮: user 提问 */
input.role = "user";
rknn3_session_run(session, inputs, 1, &param);
/* 解析输出 -> 实际调用函数 -> 构造 tool_prompt */

/* 第二轮: tool 反馈结果 */
input.role = "tool";
input_tensor.prompt = tool_prompt.c_str();
rknn3_session_run(session, inputs, 1, &param);
```

> **说明**: 完整示例见 `rknn3-runtime/examples/rknn3_session_test_demo/src/rknn3_session_test_function_call.cpp`。

4.2.5 KVCache管理

适用场景

KVCache 缓存注意力的 key-value 状态以加速自回归生成。RKNN3 提供三种 KVCache 策略与导入导出能力，适配长对话、中断续算、跨会话上下文保存等场景。

三种策略

通过 `rknn3_session_set_kvcache_policy()` 设置（`rknn3_kvcache_policy`）:

策略	行为	适用场景
`RKNN3_KVCACHE_POLICY_RECURRENT`（默认）	KVCache 自动复写，通过 `n_keep` / `n_keep_aligned` 保留 system prompt 不被复写	多轮长对话
`RKNN3_KVCACHE_POLICY_NORMAL`	只使用 max_context_len 长度的 KVCache，超过后停止生成	固定长度上下文
`RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT`	开启 Checkpoint 保存模式（见 4.2.7）	Linear/Sliding Attention 模型加速

```
rknn3_kvcache_policy_param param;
memset(&param, 0, sizeof(param));
param.recurrent.n_keep         = 21;   /* system prompt 实际 token 数 */
param.recurrent.n_keep_aligned = 32;   /* 按 32 对齐 */
rknn3_session_set_kvcache_policy(session, RKNN3_KVCACHE_POLICY_RECURRENT, &param);
```

导入导出

`rknn3_session_save_kvcache()`（保存到文件）、`rknn3_session_load_kvcache_from_path()`（从文件加载）、`rknn3_session_load_kvcache_from_data()`（从内存加载），适用于中断续算与跨会话保存上下文。典型流程：第一轮推理后 save，第二轮推理前 load。

> **说明**：KVCache 状态与模型及运行库版本高度关联，不同模型/版本不能混用。

4.2.6 Paged Attention

适用场景

传统连续 KVCache 为每个会话预留完整 `max_context_len` 的缓存空间，多请求并发时内存开销大。Paged Attention（分页 KVCache）将 KV Cache 划分为固定 token 数的页（page），按请求的实际长度以页为粒度分配和管理缓存，显著降低多请求场景的内存占用。是否启用分页由模型转换时的 `tokens_per_page` 配置决定（见 2.2.3.1），启用后模型即为"分页模型"；页内存可由 Runtime 自动分配，也可由应用显式管理。

转换阶段

在 `rknn.config()` 的 `llm_config` 中为 attention_config 配置分页参数：

```
llm_config['attention_config'][0]['tokens_per_page'] = 1024  # 启用分页，每页 1024
token
llm_config['attention_config'][0]['request_num'] = 1         # 分页请求数，当前仅支持 1
```

- `tokens_per_page` 取正整数即启用分页；建议为 64 的整数倍。页越大推理性能越好，但提升幅度不会太大；SlidingAttention 类型或非正数会回退为连续 KVCache（`kvcache_buffer_len`）。
- `request_num` 为分页请求数，当前版本仅支持 1。

板端 Runtime 使用（Session API）

通过 Session API 部署时，KVCache 页内存由 Session 内部自动管理，应用无需手工建页表；需要精确控制初始内存占用（如先按短上下文启动、再扩容）时，可在 Runtime 层使用页级接口。要点如下：

1. **查询页规格**：对每个参与运行的物理核心，将 `rknn3_kvcache_page_info` 的 `core_id` 设为核心 ID，调用 `rknn3_query(ctx, RKNN3_QUERY_KVCACHE_PAGE_INFO, ...)`，返回每种 cache 类型的 `cache_type`、每页字节数 `page_size`、编译期最大页数 `max_page_num` 与最大并发请求数 `max_request_num`。非分页模型返回 `n_cache_types == 0`，此时应使用连续 KVCache 流程（见 4.2.5）。
2. **计算页数**：FullAttention 页数按 `ceil(max_ctx_len / tokens_per_page)` 计算，不超过 `max_page_num`；其他 cache 类型按 `max_page_num` 分配完整容量。
3. **创建页内存**：每页调用 `rknn3_create_mem(ctx, page_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` 创建独立的 `rknn3_tensor_mem` 对象；CPU 需要写入或清零页内容时，写后调用 `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_TO_DEVICE)`。
4. **绑定页表**：为每个 `(core_id, cache_type, request_index)` 组装 `rknn3_kvcache_page_binding`（`n_pages` 为页数、`pages` 指向页数组），调用 `rknn3_set_kvcache_pages(ctx, bindings, n_bindings)`。调用时 context 须处于空闲状态；接口只替换列出的绑定，未列出的绑定保持不变。
5. **解绑与释放**：传入 `n_pages = 0` 即可解除对应绑定；确认解绑成功后再调用 `rknn3_destroy_mem` 销毁页内存，不要在页仍被页表引用时销毁。

```
/* page_attr 来自 RKNN3_QUERY_KVCACHE_PAGE_INFO; page_count 按 max_ctx_len
   与 tokens_per_page 计算，且不超过 page_attr->max_page_num。 */
uint32_t page_count = (max_ctx_len + tokens_per_page - 1) / tokens_per_page;
rknn3_tensor_mem* pages[page_count];
for (uint32_t i = 0; i < page_count; ++i) {
    pages[i] = rknn3_create_mem(ctx, page_attr->page_size, core_id,
                                RKNN3_FLAG_MEMORY_CACHEABLE);
}
rknn3_kvcache_page_binding binding = {
    core_id, page_attr->cache_type, request_index,
    page_count, (const rknn3_tensor_mem* const*)pages, {0}
};
rknn3_set_kvcache_pages(ctx, &binding, 1);

/* 推理完成: 先解绑 (n_pages=0)，再销毁页内存 */
rknn3_kvcache_page_binding unbind = {core_id, page_attr->cache_type,
                                     request_index, 0, NULL, {0}};
if (rknn3_set_kvcache_pages(ctx, &unbind, 1) == RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, pages[i]);
}
```

> **说明**：页绑定后页内存仍归调用方所有，须保持有效直至解绑；校验失败时旧绑定保持不变，页表写入失败时本次绑定不会生效，应根据返回值处理本批次页的生命周期。分页模型不能同时调用 `rknn3_set_kvcache_mem()` 设置连续 KVCache。接口与结构体的完整定义见《RKNN3_Runtime_C_API_参考》4.4.4 与 6.6 节。

4.2.7 KVCache Checkpoint

适用场景

Checkpoint 策略按固定间隔保存 KV Cache 快照，后续可从某 checkpoint 恢复继续生成，避免重新 prefill 整个前缀。仅对 Linear Attention / Sliding Attention 模型生效（如 Qwen3.5、Gemma4 系列），Full Attention 走独立路径，对其他模型设置无效。

使用方式

通过 `RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT` 策略，在 `rknn3_kvcache_policy_param.save_checkpoint` 配置四个参数（均需按 128 对齐）：

参数	含义
`checkpoint_start_pos`	基准位置，本身不保存，实际保存位置 = start + k×interval（k=1..max_count）
`checkpoint_interval`	保存间隔，常见 128/256/512/1024
`max_checkpoint_count`	最大保存数，上界 = floor((max_context_len - start)/interval)
`checkpoint_tail_overwrite`	默认 false；设 true 时最后一个槽位滚动覆写，适合内存受限但需保留最近恢复点

```
rknn3_kvcache_policy_param param;
memset(&param, 0, sizeof(param));
param.save_checkpoint.checkpoint_start_pos      = 0;
param.save_checkpoint.checkpoint_interval       = 128;
param.save_checkpoint.max_checkpoint_count      = 2;
param.save_checkpoint.checkpoint_tail_overwrite = false;
rknn3_session_set_kvcache_policy(session, RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT,
&param);
```

> **说明**：仅复用固定 system prompt 时建议 `max_checkpoint_count<=2`。校验规则：`start_pos > max_context_len - interval` 直接报错；各参数未对齐强制向上对齐；`max_count` 超上界强制下调。可通过日志 `load LINEAR_ATTN_CACHE kvcache_checkpoint pos: ...` 验证生效。

4.2.8 LoRA

适用场景

LoRA（低秩适配）允许不修改基础模型权重动态加载/切换适配器，适用于多任务微调部署--一份基础模型 + 多个 LoRA 适配器，按任务切换，节省存储与内存。LoRA 的使用分两个阶段：转换阶段通过 RKNN3-Toolkit 将 LoRA 权重与基础模型一起转换为 RKNN 模型；板端部署阶段通过 Runtime 接口加载并按需启用适配器。

转换阶段

RKNN3-Toolkit 提供 `load_lora`、`enable_lora`、`disable_lora` 接口，用于在模型转换阶段加载和控制 LoRA 权重。LoRA 权重需在模型加载（`load_onnx` / `load_llm`）之后、构建模型（`build`）之前加载：

```
rknn.load_lora(lora_paths=[["./adapter_model.safetensors"]],
               lora_config_paths=[["./adapter_config.json"]],
               lora_patterns=[["lora0_pattern0"]],
               lora_distribute_strategy='best_perf')
rknn.enable_lora(enable_patterns=["lora0_pattern0"])
rknn.disable_lora(disable_patterns=["lora0_pattern0"])
```

板端部署

板端通过 Runtime 接口加载并控制 LoRA。`rknn3_lora_init()` 从权重文件路径初始化上下文级别的 LoRA 支持并加载元数据，需在 `rknn3_model_init()` 之后、会话使用 LoRA 之前调用；调用后通过 `RKNN3_QUERY_LORA_NUM` 和 `RKNN3_QUERY_LORA_INFO` 查询已加载的 LoRA 信息，再通过 `rknn3_lora_load()` 加载指定的适配器。

LoRA 权重在 context 级加载管理，同一 context 下多 session 共享；是否对某 session 生效分两级控制：

- **Context 级**（对 context 下所有推理生效，仅调用时更新一次）：

接口	功能
`rknn3_lora_init()` / `rknn3_lora_init_from_data()`	从文件/内存初始化 LoRA 元数据
`rknn3_lora_load()` / `rknn3_lora_unload()`	加载/卸载适配器权重
`rknn3_lora_enable()` / `rknn3_lora_disable()`	上下文级启用/禁用

- **Session 级**（仅控制该 session，每次 `session_run` 重复更新，不影响其他 session）：

接口	功能
`rknn3_session_enable_lora()` / `rknn3_session_disable_lora()`	会话级启用/禁用，不重复加载权重

板端调用示例：

```
rknn3_lora lora_list[RKNN3_MAX_LORA_NUM];
memset(lora_list, 0, sizeof(lora_list));
rknn3_lora_init(ctx, "lora_weight_path");            /* 初始化元数据，须在 model_init
之后 */
uint32_t n_lora = 0;
rknn3_query(ctx, RKNN3_QUERY_LORA_NUM, &n_lora, sizeof(n_lora));        /* 查询 LoRA
数量 */
rknn3_query(ctx, RKNN3_QUERY_LORA_INFO, lora_list, sizeof(lora_list));  /* 查询合法的
lora 描述符 */
rknn3_lora_load(ctx, &lora_list[0]);                 /* 加载适配器权重 */
rknn3_session_enable_lora(session, &lora_list[0]);   /* 会话级启用 */
```

> **说明**：不建议混用两级接口，会话级会覆盖上下文级导致生效顺序不确定。LoRA 权重支持 int4 / int6 / int8 / float16 量化，多核拆分策略 `best_perf` / `less_mem`。

4.2.9 Tie Word Embedding

适用场景

Tie Word Embeddings 是权重绑定技术：输入 embedding 层与输出 lm_head 层共享相同权重矩阵。启用后 Device 端复用模型内 lm_head 权重，**Host 端无需加载 `embed.bin`**，节省 Host 端内存与磁盘空间。

使用方式

- **确认支持**：检查 HuggingFace `config.json` 的 `tie_word_embeddings` 字段是否为 true（常见 Qwen 系列）。
- **转换配置**：`llm_config['vocab'][0]['tie_embeddings'] = True`，转换后用 `strings xxx.rknn | grep tie_word_embeddings` 验证。

```
llm_config = DEFAULT_RKNN_LLM_CONFIG.copy()
llm_config['vocab'][0]['tie_embeddings'] = True
rknn.config(..., llm_config=llm_config)
```

- **部署差异**：Tie 模式下不加载 `embed.bin`（不 open/mmap）、不设 `embed_callback`；非 Tie 模式两者都需要。

模式	embed.bin	embed_callback	Host 内存
Tie	不需要	不需要	较低
非 Tie	需要	需要	较高

> **说明**：即使模型含 `tie_word_embeddings:true`，仍可设环境变量 `RKNN_ENABLE_TIE_WORD_EMBEDDINGS=0` 强制以非 Tie 模式运行，但需加载 embed.bin 并设 embed_callback，失去内存优势。

4.2.10 暂停、恢复和状态查询

适用场景

推理过程中可暂停/恢复会话，用于在生成期间插入其他事务（如用户打断、优先级任务）；状态查询用于监控推理进度、性能统计与 LoRA/KVCache 状态。

使用方式

- **暂停**：`rknn3_session_pause()` 。
- **恢复**：`rknn3_session_resume()` 。

暂停时 `result_callback` 被调用并传入 `RKLLM_RUN_PAUSE`，恢复时传入 `RKLLM_RUN_RESUME`，可在回调中处理这些状态实现业务逻辑。

```
rknn3_session_pause(session);
/* 暂停期间处理其他事务 */
rknn3_session_resume(session);
```

- **状态查询**：`rknn3_session_query_state()` 查询当前状态，结果存入 `RKLLMRunState`，主要字段：

字段	含义
`n_max_tokens`	最大推理 token 数
`n_total_tokens`	已推理 token 数
`n_prefill_tokens`	prefill 阶段处理的 token 数
`n_decode_tokens`	generate 阶段生成的 token 数
`n_output_tokens`	模型输出 token 数（= n_decode_tokens + 1）
`input_tokens`	指向输入 token 数组的指针（用户不应释放）
`output_tokens`	指向输出 token 数组的指针（用户不应释放）
`kvcache_policy`	当前 KVCache 策略
`n_loras_enabled`	启用的 LoRA 数量

> **说明**：完整字段见《RKNN3_Runtime_C_API_参考》5.5.31。

4.2.11 结构化输出（Grammar）

适用场景

结构化输出（Grammar-guided generation）用于在 LLM 推理的 decode 阶段约束模型输出严格符合指定格式（JSON Schema / 正则 / EBNF / 任意合法 JSON / 结构化标签），从 token 级别屏蔽非法 token，保证输出结构正确，降低后处理容错成本。典型场景包括：

- 强制模型输出符合业务 JSON Schema 的结构化数据（如 `{"name":..., "age":...}`），免去正则提取与字段校验。
- 限定模型输出为邮箱、日期、UUID、十六进制色值等正则格式。

- 通过 EBNF 限定模型输出为枚举值（如 `yes` / `no` / `maybe`）或自定义语法。
- 用结构化标签（Structural Tag）约束 Qwen3.5 等模型的工具调用输出格式。

基本原理

SDK 集成 XGrammar 实现 token 级约束：将用户提供的 JSON Schema / Regex / EBNF 等编译为 BNF，在每步 decode 前生成 token bitmask，把非法 token 的 logit 置为 `-inf`，再交由采样器选 token，使输出严格符合约束格式。约束状态由 Runtime 在 `rknn3_session_run()` 的 decode 循环中自动维护，用户无需介入采样流程。

Runtime 始终从本次 run 的第一个生成 token 开始应用 `rknn3_session_set_grammar()` 设置的 grammar，不检测模型是否进入或退出 thinking，也不自动补充 thinking 前缀。`enable_thinking` 只影响模型/chat template：不开启 thinking 时，grammar 通常只需描述最终答案；开启 thinking 时，调用方必须让同一个 grammar 覆盖 thinking 内容、结束标记以及最终答案的完整生成流。

支持的约束类型

`rknn3_grammar_type`	**说明**	**典型用法**
`RKNN3_GRAMMAR_JSON_SCHEMA`	JSON Schema 字符串，约束输出为符合 schema 的 JSON	`{"type":"object","properties":{...}}`
`RKNN3_GRAMMAR_EBNF`	EBNF 语法字符串，支持多规则组合与枚举	`root ::= "yes" | "no" | "maybe"`
`RKNN3_GRAMMAR_BUILTIN_JSON`	内置任意合法 JSON，无需提供 schema（`grammar_str` 传 `NULL`）	约束输出为合法 JSON 但不限定字段
`RKNN3_GRAMMAR_REGEX`	正则表达式，约束输出完整匹配该正则	`[0-9]{4}-[0-9]{2}-[0-9]{2}`（日期）
`RKNN3_GRAMMAR_STRUCTURAL_TAG`	XGrammar Structural Tag JSON，可组合常量串、正则、JSON Schema，并支持 `tag` / `triggered_tags`	Thinking 与 Qwen 等模型的工具调用格式约束

使用流程

Grammar 使用分三步：上下文初始化（建词表与编译器）-> 会话级设置约束（编译 grammar）-> 会话推理（decode 循环自动 mask）；推理结束后可清除约束恢复自由生成。该版本不提供单独的 thinking grammar 设置接口，thinking 场景也只调用现有的 `rknn3_session_set_grammar()`，但传入的 grammar 必须覆盖完整生成流。整体流程如下：



`enable_thinking` 与 grammar 内容的关系如下：

`enable_thinking`	**`rknn3_session_set_grammar()` 应描述的生成流**
`false`	最终答案，从首个生成 token 直接应用约束
`true`	完整输出：thinking 内容 + thinking 结束/切换标记 + 最终答案

> **注意**：Runtime 不根据模型类型、chat template、首个 token 或已经生成的 `</think>` 推断 grammar 阶段。若开启 thinking 却只设置最终答案 JSON Schema，最终答案 grammar 会错误地从 reasoning 的第一个 token 开始生效。

接口说明

Grammar 接口分三级，按需选择：

- **Context 级**（一次性，建词表与编译器）：

接口	**功能**
`rknn3_init_grammar(ctx, &config)`	用 `encoded_vocab` 构建 Host 侧 TokenizerInfo + GrammarCompiler，须在 session 使用 grammar 之前调用；重复调用返回错误

`rknn3_grammar_config` 主要字段：

字段	**含义**
`vocab_size`	词表 token 数量
`encoded_vocab`	序列化词表 blob：对每个 token 依次写入 `int32_t len` + `len` 字节 token 字符串
`encoded_vocab_len`	blob 总字节数

字段	含义
`n_stop_tokens` / `stop_tokens`	EOS/stop token id 列表，约束生成在这些 token 处终止

- **Session 级**（由 `session_run` decode 循环自动驱动 mask，推荐）：

接口	功能
`rknn3_session_set_grammar(session, str, type, options)`	编译 grammar 并绑定到 session，run 时自动 mask；空串且 `type` 非 `BUILTIN_JSON` 时等价清除
`rknn3_session_clear_grammar(session)`	清除 grammar，恢复自由生成

`rknn3_grammar_set_options` 用于微调 JSON 输出格式，全部字段可选（用宏 `RKNN3_GRAMMAR_SET_OPTIONS_INIT` 初始化，未设字段保持默认）：

字段	含义
`any_whitespace`	JSON 元素间是否允许任意空白
`indent`	缩进格数（-1 表示不缩进）
`strict_mode`	严格模式开关
`max_whitespace_cnt`	最大空白字符数
`any_order`	JSON 对象键是否允许任意顺序
`item_separator` / `key_value_separator`	元素分隔符 / 键值分隔符
`root_rule_name`	EBNF 多规则时指定的根规则名

- **Handle 级**（用户手动编排采样循环时使用）：

接口	功能
`rknn3_grammar_create(ctx)` / `rknn3_grammar_destroy(ctx, g)`	创建 / 销毁 grammar 句柄（须先 `init_grammar`）
`rknn3_grammar_set(g, str, type, options)` / `rknn3_grammar_clear(g)`	设置 / 清除约束
`rknn3_grammar_apply_mask(g, logits, vocab_size, output_idx)`	对 logits 应用 mask（非法 logit 置 `-inf`）
`rknn3_grammar_accept_token(g, token_id, output_idx)`	通知 grammar 已采样该 token，推进状态机
`rknn3_grammar_reset(g)`	重置状态机，重新开始（不重新编译）

> **说明**：session 级与 handle 级的区别--session 级由 `session_run` 的 decode 循环自动驱动 `apply_mask` / `accept_token`；handle 级由用户自行编排采样循环（自定义采样 / 特殊推理流程），协处理器模式下纯 Host 端执行，不经过 Device。

使用示例

以 JSON Schema 约束为例（要求输出含 `name` / `age` 字段的 JSON）。tokenizer 与 `VocabInfo` 的获取见 2.2.4 模型和 Tokenizer 适配：

```
/* 1. 用 tokenizer 序列化 encoded_vocab：逐 token 写入 int32 长度 + token 字符串 */
int vocab_size = vocab_info.vocab_size;
std::vector<uint8_t> vocab_blob;
for (int i = 0; i < vocab_size; i++) {
    std::string piece = tokenizer->TokenToPiece(i);
    int32_t len = (int32_t)piece.size();
    vocab_blob.insert(vocab_blob.end(), (uint8_t*)&len, (uint8_t*)&len +
sizeof(int32_t));
    vocab_blob.insert(vocab_blob.end(), (uint8_t*)piece.data(),
(uint8_t*)piece.data() + piece.size());
}

/* 2. 初始化 context 级 grammar（一次性） */
std::vector<int32_t> stop_tokens;
for (int i = 0; i < vocab_info.n_special_eos_id; i++) {
    stop_tokens.push_back(vocab_info.special_eos_id[i]);
}
rknn3_grammar_config g_cfg = {};
g_cfg.vocab_size         = vocab_size;
g_cfg.encoded_vocab      = vocab_blob.data();
g_cfg.encoded_vocab_len  = (int32_t)vocab_blob.size();
g_cfg.n_stop_tokens      = (int32_t)stop_tokens.size();
g_cfg.stop_tokens        = stop_tokens.data();
rknn3_init_grammar(ctx, &g_cfg);

/* 3. 设置 session 级约束（JSON Schema） */
const char* schema =
    "{\"type\":\"object\","
    "\"properties\":{\"name\":{\"type\":\"string\"},"
    "\"age\":{\"type\":\"integer\",\"minimum\":0,\"maximum\":150}},"
    "\"required\":[\"name\",\"age\"]}";
rknn3_session_set_grammar(session, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);

/* 4. 正常 session_run，decode 循环自动 mask */
rknn3_llm_infer_param infer = {};
infer.max_new_tokens = 256;
rknn3_llm_tensor input_tensor = {.name = NULL, .prompt = (char*)"Generate a JSON
with name and age.",
                                 .embed = NULL, .tokens = NULL, .n_tokens = 0,
.enable_thinking = false};
rknn3_llm_input input = {};
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input = input_tensor;
rknn3_session_run(session, &input, 1, &infer);
/* 输出形如：{"name":"Alice","age":30} */

/* 5. 清除约束，恢复自由生成 */
rknn3_session_clear_grammar(session);
```

其他约束类型设置示例：

```
/* 正则：限定为日期 YYYY-MM-DD */
rknn3_session_set_grammar(session, "[0-9]{4}-[0-9]{2}-[0-9]{2}",
                          RKNN3_GRAMMAR_REGEX, NULL);

/* EBNF：限定为 yes/no/maybe 三选一 */
const char* ebnf = "root ::= answer\nanswer ::= \"yes\" | \"no\" | \"maybe\"\n";
rknn3_session_set_grammar(session, ebnf, RKNN3_GRAMMAR_EBNF, NULL);

/* 内置 JSON：输出任意合法 JSON 对象（grammar_str 传 NULL） */
rknn3_session_set_grammar(session, NULL, RKNN3_GRAMMAR_BUILTIN_JSON, NULL);
```

Thinking 模式：由用户拼接完整 grammar

开启 thinking 时，不需要新增接口，也不能只把最终答案 grammar 传给 Runtime。调用方应使用现有的 `rknn3_session_set_grammar()` 提交一份覆盖完整生成流的 grammar。对于 XGrammar 后端，推荐使用一个 Structural Tag `sequence` 依次描述：

```
thinking 内容 → thinking 结束/切换标记 → 最终答案
```

例如，模型在生成 `</think>\n\n` 后输出包含整数 `answer` 的 JSON：

```
const char* full_thinking_json =
    "{\"type\":\"structural_tag\",\"format\":{"
    "\"type\":\"sequence\",\"elements\":["
    /* 1. thinking：允许任意长度文本，直到 </think> */
    "{\"type\":\"tag\",\"begin\":\"\","
        "\"content\":{\"type\":\"any_text\"},\"end\":\"</think>\"},"
    /* 2. thinking 与最终答案之间的固定分隔 */
    "{\"type\":\"const_string\",\"value\":\"\\n\\n\"},"
    /* 3. 最终答案 */
    "{\"type\":\"json_schema\",\"json_schema\":{"
        "\"type\":\"object\","
        "\"properties\":{\"answer\":{\"type\":\"integer\"}},"
        "\"required\":[\"answer\"],\"additionalProperties\":false}}"
    "]}}";

int ret = rknn3_session_set_grammar(session, full_thinking_json,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);
if (ret != RKNN3_SUCCESS) {
    /* 处理错误 */
}

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "What is 2 + 2? Reply with JSON containing integer field answer.",
    .embed = NULL,
    .tokens = NULL,
    .n_tokens = 0,
    .enable_thinking = true,
};
/* 生成流结构：...reasoning...</think>\n\n{\"answer\":4} */
```

上例中的空 `begin` 表示 grammar 从首个生成 token 接受任意 thinking 文本；它既兼容 opening marker 已位于 prompt/KVCache 的模板，也允许（但不强制）模型在 `</think>` 前自行生成 opening marker。

若模型使用其他字符串或 special token 作为 thinking 边界，应按实际 chat template 修改 `begin`、`end` 和分隔符。

`rknn3_session_set_grammar()` 每次只接受一种 grammar。这里的“拼接”是调用方在一个 Structural Tag JSON 内使用 `sequence`、`tag`、`json_schema`、`regex` 等节点描述完整语言，不是把多段序列化 JSON 字符串直接相连。关闭 thinking 后，应重新设置只描述最终答案的 grammar；否则完整 grammar 仍会要求生成 thinking 结束标记。

限制输出为 tool_call

Grammar 可以限制工具调用的外层标记和参数 JSON 结构，但不负责注册或执行工具。工具定义、chat template、提示词和输出解析仍由调用方配置；如需使用 SDK 的 Function Calling 流程，参见 4.2.4。

不同模型对 function/tool_call 的序列化协议并不相同。下面列出示例程序 `rknn3_session_test_grammar.cpp` 覆盖的两种常见格式；调用方应根据模型及其 chat template 选择对应 grammar，不能混用。

1. **Native XML（Qwen3.5 / Qwen3-Coder）**

函数名和参数都通过 XML tag 表达：

```
<tool_call>
<function=get_weather>
<parameter=location>
Shanghai
</parameter>
</function>
</tool_call>
```

对应的 Structural Tag 字符串配置如下。函数名使用正则约束，示例参数固定为 `location=Shanghai`：

```
const char* native_xml_tool_call_tag =
    "{\"type\":\"structural_tag\",\"format\":{\"type\":\"sequence\",\"elements\":["
    "{\"type\":\"const_string\",\"value\":\"<tool_call>\\n<function=\"},"
    "{\"type\":\"regex\",\"pattern\":\"[A-Za-z0-9_-]{1,64}\"},"
    "{\"type\":\"const_string\",\"value\":\">\\n\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"<parameter=location>\\nShanghai\\n</parameter>\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"\\n</function>\\n</tool_call>\"}"
    "]}}";
```

1. **JSON-in-XML / Hermes（Qwen3）**

`<tool_call>` 内部使用包含 `name` 和 `arguments` 的 JSON：

```
<tool_call>
{"name": "get_current_temperature", "arguments": {"location": "Shanghai"}}
</tool_call>
```

对应配置使用 `triggered_tags`：遇到 `<tool_call>\n` 后进入指定 tag，并用 JSON Schema 约束 `arguments`：

```
const char* json_in_xml_tool_call_tags =
    "{\"type\":\"structural_tag\",\"format\":{\"type\":\"triggered_tags\","
    "\"triggers\":[\"<tool_call>\\n\"],\"tags\":[{\"type\":\"tag\","
    "\"begin\":\"<tool_call>\\n"
        "{\\\"name\\\": \\\"get_current_temperature\\\", \\\"arguments\\\": \","
    "\"content\":{\"type\":\"json_schema\",\"json_schema\":{"
        "\"type\":\"object\",\"additionalProperties\":true}},"
    "\"end\":\"}\\n</tool_call>\"}],"
    "\"at_least_one\":true,\"stop_after_first\":true}}";
```

上面字符串与参考测试中的协议一致，用于展示 grammar 的组织方式。生产场景应根据工具列表为每个函数设置准确的函数名、参数字段、必填项和 `additionalProperties`，并与 `rknn3_session_set_function_tools()` 使用的工具描述保持一致。如果模型使用特殊 token 或其他 wrapper，也必须相应修改 trigger、tag 边界和固定字符串。

不开启 thinking：直接约束最终 tool_call

根据模型选择上面的 Native XML 或 JSON-in-XML grammar，并设置 `enable_thinking=false`。Grammar 从首个生成 token 起生效，因此输出只能进入所选 tool_call 协议：

```
/* Qwen3.5 / Qwen3-Coder: 选择 Native XML。Qwen3 可改为
 * json_in_xml_tool_call_tags。
 */
const char* selected_tool_call_grammar = native_xml_tool_call_tag;
int ret = rknn3_session_set_grammar(session, selected_tool_call_grammar,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "Call get_weather for Shanghai. Reply only with the tool call.",
    .enable_thinking = false,
};
```

开启 thinking，方案一：手工拼接 thinking + Native XML

将 thinking prefix 和 Native XML tool_call 都写入同一个 `sequence`。这种方式要求 `</think>\n\n` 后立即进入 `<tool_call>\n<function=...>`：

```
const char* full_thinking_native_xml_tool_call =
    "{\"type\":\"structural_tag\",\"format\":{"
    "\"type\":\"sequence\",\"elements\":["
    /* thinking 区域 */
    "{\"type\":\"tag\",\"begin\":\"\","
        "\"content\":{\"type\":\"any_text\"},\"end\":\"</think>\"},"
    "{\"type\":\"const_string\",\"value\":\"\\n\\n\"},"
    /* Native XML tool_call 区域 */
    "{\"type\":\"const_string\",\"value\":\"<tool_call>\\n<function=\"},"
    "{\"type\":\"regex\",\"pattern\":\"[A-Za-z0-9_-]{1,64}\"},"
    "{\"type\":\"const_string\",\"value\":\">\\n\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"<parameter=location>\\nShanghai\\n</parameter>\"},"
    "{\"type\":\"const_string\","
        "\"value\":\"\\n</function>\\n</tool_call>\"}"
    "]}}";

int ret = rknn3_session_set_grammar(session, full_thinking_native_xml_tool_call,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "Call get_weather for Shanghai. Reply only with the tool call.",
    .enable_thinking = true,
};
```

开启 thinking，方案二：完整 grammar + JSON-in-XML `triggered_tags`

对 JSON-in-XML/Hermes 协议，可以把 `triggered_tags` 放在 thinking 前缀之后。需要支持多个函数时，在 `tags` 中为每个函数增加对应的 `begin` 和参数 schema，并配置可以唯一分派的 trigger：

```
const char* full_thinking_json_in_xml_tool_call =
    "{\"type\":\"structural_tag\",\"format\":{"
    "\"type\":\"sequence\",\"elements\":["
    /* thinking 区域 */
    "{\"type\":\"tag\",\"begin\":\"\","
        "\"content\":{\"type\":\"any_text\"},\"end\":\"</think>\"},"
    "{\"type\":\"const_string\",\"value\":\"\\n\\n\"},"
    /* tool_call 区域 */
    "{\"type\":\"triggered_tags\","
        "\"triggers\":[\"<tool_call>\\n\"],"
        "\"tags\":[{\"type\":\"tag\","
            "\"begin\":\"<tool_call>\\n"
                "{\\\"name\\\": \\\"get_current_temperature\\\", \\\"arguments\\\":
\","
            "\"content\":{\"type\":\"json_schema\",\"json_schema\":{"
                "\"type\":\"object\",\"additionalProperties\":true}},"
            "\"end\":\"}\\n</tool_call>\"}],"
        "\"at_least_one\":true,\"stop_after_first\":true}"
    "]}}";

int ret = rknn3_session_set_grammar(session, full_thinking_json_in_xml_tool_call,
                                    RKNN3_GRAMMAR_STRUCTURAL_TAG, NULL);

rknn3_llm_tensor input_tensor = {
    .name = NULL,
    .prompt = "Call get_current_temperature with location=Shanghai. Reply only with
the tool call.",
    .enable_thinking = true,
};
```

`triggered_tags` 的每个 trigger 必须是对应 tag `begin` 的无歧义前缀。不要直接用一个宽松的 `triggered_tags` 同时表达“任意 thinking 文本”和“必须输出一次 tool_call”：

- `at_least_one=false` 允许 trigger 前出现自由文本，但也允许一次 tool_call 都不生成；
- `at_least_one=true` 要求至少生成一个 tag，同时禁止该 `triggered_tags` 节点内部出现前置自由文本；
- 因此，需要保留 thinking 且强制至少一个 tool_call 时，应使用外层 `sequence`，先描述 thinking 和 `</think>\n\n`，再进入 `triggered_tags`。

约束能力与使用边界

Grammar 能提高输出结构的确定性，但不能保证每次推理都 100% 得到符合用户业务意图的 tool_call。应区分“结构合法”和“语义正确”：

- Grammar 只约束生成 token 是否属于所描述的语言。例如，函数名使用正则或 `arguments` 仅被约束为任意对象时，并不代表模型一定选择正确的函数或填入 `location=Shanghai`；工具选择、参数值及其与上下文是否一致仍取决于模型能力、工具描述、提示词、chat template 和采样配置。固定值应在 grammar/schema 中使用常量或枚举表达，并在调用工具前做业务校验。
- 当 run 正常完成 grammar 时，输出结构可以满足对应的 Structural Tag/JSON Schema；但达到 `max_new_tokens` 或 context 上限、用户停止/中止、推理错误等情况仍可能留下未完成的 thinking 或 tool_call。调用方必须检查运行状态，并对完整输出再次解析和校验。
- `triggered_tags` 只在命中 trigger 后约束对应 tag；是否强制出现 tool_call 取决于 `at_least_one` 及它所在的外层结构。宽松配置下，模型可能始终不触发 tool_call。

- thinking 的 opening、结束标记、换行及特殊 token 必须与实际模型模板一致。配置不匹配时，即使 grammar 能成功编译，也可能造成生成质量下降、无自然可选 token 或在 token 预算内无法完成。

因此，Grammar 的作用是让正常完成的输出满足机器可解析的结构约束，而不是替代模型的工具调用判断和业务正确性校验。实际应用应同时优化模型与提示词，并在执行工具前验证函数名、参数 schema、参数取值和权限。

Handle 级手动驱动模式

当需要自定义采样策略或特殊推理流程（如自行实现 beam search、top-k/top-p 采样、logits 后处理）时，session 级的自动 mask 无法介入用户自定义的采样路径，此时改用 handle 级 API 手动编排采样循环。handle 级在协处理器模式下纯 Host 端执行，不经过 Device，用户需自行获取每步 decode 的 logits，并按 `apply_mask -> 自定义 sample -> accept_token` 的顺序驱动。

使用流程

handle 级手动驱动流程如下：



使用示例

手动采样循环示例（JSON Schema 约束，logits 由用户自定义推理路径获取）：

```c
/* 1. 创建 grammar 句柄（须先调用 rknn3_init_grammar） */
rknn3_grammar* grammar = rknn3_grammar_create(ctx);

/* 2. 编译 grammar 绑定到句柄 */
const char* schema =
    "{\"type\":\"object\",\"properties\":{\"name\":{\"type\":\"string\"}}}";
rknn3_grammar_set(grammar, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);

/* 3. 手动 decode 循环：apply_mask -> 自定义 sample -> accept_token */
for (int step = 0; step < max_new_tokens; step++) {
    /* 用户自行获取当前步 logits（来自自定义推理路径） */
    float16* logits     = ...;  /* 当前步 logits，长度为 vocab_size */
    int32_t  vocab_size  = ...;

    /* 3a. 应用 mask：非法 token 的 logit 被置为 -inf */
    rknn3_grammar_apply_mask(grammar, logits, vocab_size, 0);

    /* 3b. 用户自定义采样（greedy / top-k / top-p 等） */
    int32_t token_id = my_sample(logits, vocab_size);

    /* 3c. 通知 grammar 已采样该 token，推进状态机；失败说明采样与约束不一致 */
    if (rknn3_grammar_accept_token(grammar, token_id, 0) != 0) {
        printf("xgrammar AcceptToken rejected token_id=%d\n", token_id);
        break;
    }

    /* 命中 stop token 则结束生成 */
    if (is_stop_token(token_id, stop_tokens, n_stop_tokens)) break;
}

/* 4. 重置状态机，开启下一轮约束生成（不重新编译 grammar） */
rknn3_grammar_reset(grammar);

/* 5. 销毁句柄（clear 先清除约束，destroy 释放句柄） */
rknn3_grammar_clear(grammar);
rknn3_grammar_destroy(ctx, grammar);
```

> **说明**：handle 级与 session 级是两种驱动方式--session 级由 `rknn3_session_run` 的 decode 循环自动调用 `apply_mask` / `accept_token`；handle 级由用户自行编排采样循环。两者共用同一 context 由 `rknn3_init_grammar` 构建的 TokenizerInfo / GrammarCompiler，切换驱动方式无需重新 init。三段调用顺序不可颠倒：`apply_mask` 必须在采样前调用以屏蔽非法 token，`accept_token` 必须在采样后调用以推进状态机，否则状态机与输出不同步。
>
> **说明**：- `rknn3_init_grammar` 每个 context 仅可调用一次；重复调用会销毁旧编译器，使已建 session 的 grammar 句柄悬空，返回错误。 - `encoded_vocab` 必须覆盖词表全部 token；缺漏会导致 mask 不完整，模型可能输出不符合约束的 token。 - 单步 decode 开销约 +0.5~1.0ms/token（仅供参考），首次 `set_grammar` 编译耗时 <100ms（仅供参考），建议在 `session_run` 前一次性设置。 - Grammar 状态由 `session_run` 入口自动重建 fresh matcher，多轮 `keep_history` 推理时无需手动重置；切换约束类型（如 JSON -> Regex）直接调用 `rknn3_session_set_grammar` 即可，无需先 clear。 - 当前模型均为单输出；`llm_output_count > 1` 时仅 `output_idx=0` 的 mask 有效。 - Grammar 与 speculative decoding 暂不兼容，启用 grammar 时应关闭 speculative decoding。

4.3 扩展与安全

4.3.1 自定义算子

概述

RKNN3 提供自定义算子扩展机制，允许用户实现自定义算子在 RK182X 协处理器的 CPU 核心上执行，用于扩展算子实现、减少数据回传、降低主控负载、简化应用开发。典型应用场景包括目标检测模型（YOLOv5/v6/v8）的边界框解码、NMS 等后处理操作，以及模型图中 RKNN3 暂不支持的中间算子。

两类算子

根据算子在模型中的位置和用途，自定义算子分为两类：

类型	plugin_type	位置	输入输出
后处理算子	`RKNN3_OP_PLUGIN_TYPE_POSTPROCESS`	模型末尾	由插件 `get_output_num` / `get_attrs` 自定义
自定义 CPU 算子	`RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP`	计算图任意位置	由模型图决定

两类算子共用同一套插件接口（`rknn3_custom_op` 结构体）、编译方式和加载 API（`rknn3_register_custom_ops_plugins()`），区别在于 `plugin_type` 字段及对应的回调职责。

插件接口定义

自定义算子需实现 `rknn3_custom_op` 结构体定义的接口，该结构体在 `rknn3_api.h` 中定义：

```
typedef struct _rknn3_custom_op
{
    const char *op_type;                    // 算子类型名称
    rknn3_op_plugin_type plugin_type;       // 插件类型
    rknn3_op_target_type target;            // 执行后端（CPU）
    uint32_t version;                       // 版本号
    const char *author;                     // 作者信息
    const char *description;                // 描述信息

    // 生命周期回调函数
    int (*init)(rknn3_custom_op_context *op_ctx);     // [可选] 初始化
    int (*prepare)(rknn3_custom_op_context *op_ctx,   // [可选] 准备
                   rknn3_tensor *inputs, uint32_t n_inputs,
                   rknn3_tensor *outputs, uint32_t n_outputs);
    int (*compute)(rknn3_custom_op_context *op_ctx,   // [必需] 计算
                   rknn3_tensor *inputs, uint32_t n_inputs,
                   rknn3_tensor *outputs, uint32_t n_outputs);
    int (*deinit)(rknn3_custom_op_context *op_ctx);   // [可选] 反初始化

    // 后处理插件专用接口
    int (*get_output_num)(rknn3_custom_op_context *op_ctx);  // [可选] 获取输出数量
    int (*get_attrs)(rknn3_custom_op_context *op_ctx,        // [可选] 获取输入输出属性
                     rknn3_tensor_attr *input_attrs, uint32_t n_inputs,
                     rknn3_tensor_attr *output_attrs, uint32_t n_outputs);
} rknn3_custom_op;
```

关键数据结构：

- `rknn3_custom_op_context`：算子上下文，包含 RKNN3 context 句柄、私有数据指针（`priv_data`，框架管理，用户勿改）、用户数据指针（`user_data`，用户管理）等。自定义 CPU 算子还可通过其 `get_param` 接口读取算子属性。

- `rknn3_tensor`：张量结构，包含内存信息（`rknn3_tensor_mem`）和属性信息（`rknn3_tensor_attr`），插件通过 `inputs[i].mem->virt_addr` 读取输入、`outputs[j].mem->virt_addr` 写入输出。
- `rknn3_tensor_attr`：张量属性，包括 shape / dtype / layout、量化参数等。

插件类型与执行后端枚举：

```
typedef enum _rknn3_op_plugin_type {
    RKNN3_OP_PLUGIN_TYPE_POSTPROCESS = 0,  // 后处理算子（模型末尾）
    RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP = 1,    // 自定义 CPU 算子（图任意位置）
} rknn3_op_plugin_type;

typedef enum _rknn3_op_target_type {
    RKNN3_OP_TARGET_TYPE_CPU = 1,          // CPU 后端
} rknn3_op_target_type;
```

本节后续按算子类型分别介绍实现步骤、编译加载与示例：4.3.1.1 后处理算子、4.3.1.2 自定义 CPU 算子。

4.3.1.1 后处理算子

后处理算子位于模型末尾，用于将后处理计算（如 YOLO 边界框解码、NMS）从主控 SoC 迁移到协处理器端执行，减少数据回传、降低主控负载。其输出数量与属性由插件通过 `get_output_num` / `get_attrs` 自定义。

4.3.1.1.1 插件实现步骤

自定义算子插件开发流程如下图：

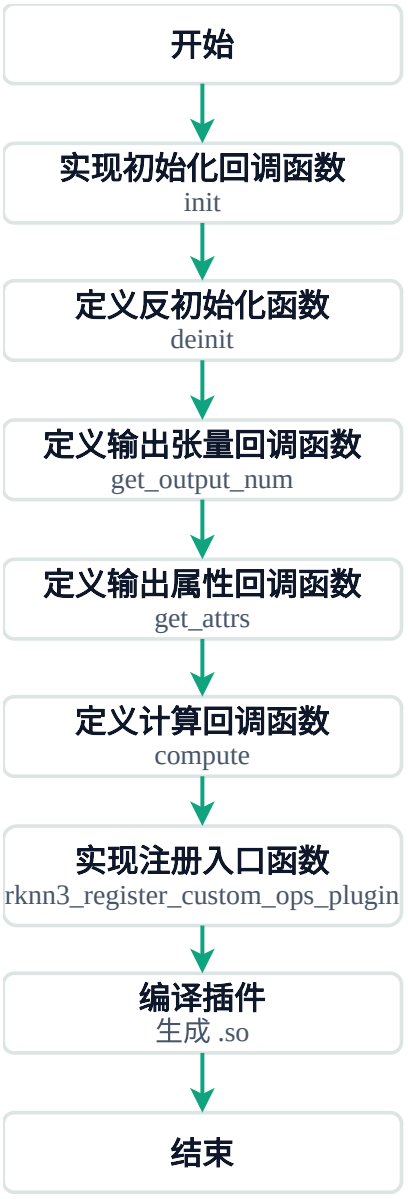


图4-1 自定义算子插件开发流程

如图所示，插件实现依次完成各回调函数，组装为算子结构体后注册，最后编译为 `.so`。下面以 YOLO 后处理插件（`YoloPostprocess`）为例说明各步骤：

- **实现初始化回调函数**：`init` 在算子初始化时调用，分配私有数据（如阈值、模型尺寸等），存入 `op_ctx->priv_data`，供后续回调使用。

```
static int yolo_postprocess_plugin_init(rknn3_custom_op_context *op_ctx)
{
    rknn3_yolo_postprocess_context *pp_ctx = malloc(sizeof(...));
    memset(pp_ctx, 0, sizeof(...));
    pp_ctx->conf_threshold = 0.25;
    pp_ctx->nms_threshold  = 0.45;
    op_ctx->priv_data = pp_ctx;
    return 0;
}
```

- **定义反初始化函数**：`deinit` 在算子销毁时调用，释放 `init` 中分配的私有数据等资源。

```
static int yolo_postprocess_plugin_deinit(rknn3_custom_op_context *op_ctx)
{
    if (op_ctx->priv_data) {
        free(op_ctx->priv_data);
        op_ctx->priv_data = NULL;
    }
    return 0;
}
```

- **定义输出张量回调函数**：`get_output_num` 返回算子输出张量数量，后处理插件的输出数量由插件自定义。

```
static int yolo_postprocess_plugin_get_output_num(rknn3_custom_op_context *op_ctx)
{
    return 1;   /* 输出 1 个张量：检测结果 */
}
```

- **定义输出属性回调函数**：`get_attrs` 定义输出张量的 shape、dtype、aligned_size 等属性。

```
static int yolo_postprocess_plugin_get_attrs(rknn3_custom_op_context *op_ctx,
                                             rknn3_tensor_attr *input_attrs,
uint32_t n_inputs,
                                             rknn3_tensor_attr *output_attrs,
uint32_t n_outputs)
{
    output_attrs[0].n_dims  = 3;
    output_attrs[0].shape[0] = input_attrs[0].shape[0];  /* batch */
    output_attrs[0].shape[1] = 256;   /* 最大检测数 */
    output_attrs[0].shape[2] = 6;     /* score, class_id, x1, y1, x2, y2 */
    output_attrs[0].dtype    = RKNN3_TENSOR_FLOAT32;
    return 0;
}
```

- **定义计算回调函数**：`compute` 是必需回调，执行算子的实际计算逻辑，从 `inputs[i].mem->virt_addr` 读取输入，处理后写入 `outputs[j].mem->virt_addr`。

```
static int yolo_postprocess_plugin_compute(rknn3_custom_op_context *op_ctx,
                                           rknn3_tensor *inputs, uint32_t n_inputs,
                                           rknn3_tensor *outputs, uint32_t
n_outputs)
{
    int result_count = post_process(op_ctx->priv_data, inputs, n_inputs, outputs,
n_outputs);
    return (result_count >= 0) ? 0 : -1;
}
```

- **实现注册入口函数**：将上述回调组装到 `rknn3_custom_op` 结构体（指定 `op_type`、`plugin_type`、`target` 及各回调指针），再通过固定名称的入口函数 `rknn3_register_custom_ops_plugin()` 注册，框架按 `op_index` 依次调用返回已注册的算子。一个 `.so` 可注册多个算子。

```
static rknn3_custom_op yolo_postprocess_op = {
    .op_type        = "YoloPostprocess",
    .plugin_type    = RKNN3_OP_PLUGIN_TYPE_POSTPROCESS,
    .target         = RKNN3_OP_TARGET_TYPE_CPU,
    .init           = yolo_postprocess_plugin_init,
    .compute        = yolo_postprocess_plugin_compute,
    .deinit         = yolo_postprocess_plugin_deinit,
    .get_output_num = yolo_postprocess_plugin_get_output_num,
    .get_attrs      = yolo_postprocess_plugin_get_attrs,
};

static rknn3_custom_op* registered_ops[] = { &yolo_postprocess_op, NULL };

rknn3_custom_op* rknn3_register_custom_ops_plugin(int op_index)
{
    if (op_index < 0 || registered_ops[op_index] == NULL) {
        return NULL;
    }
    return registered_ops[op_index];
}
```

- **编译插件**：将上述 C 源码编译为 `.so` 动态库，编译要求与脚本见 4.3.1.1.2。

4.3.1.1.2 插件编译和加载使用

编译要求

- 必须纯 C 语言（不支持 C++ 类/模板/命名空间/异常）；
- 编译器为 RISC-V 工具链 `riscv64-unknown-elf-gcc`（RK182X CPU 核心专用）；
- 关键编译选项：`-mcpu=c908v -mrvv-v0p10-compatible -march=rv64gcv -mabi=lp64d -O2 -g2 -mcmodel=medany -fpic -fno-plt`；
- 链接选项：`-nostartfiles -nostdlib -static-libgcc -e 0 -Wl,-r,-z,max-page-size=1024`，产物为 `.so`，用 `riscv64-unknown-elf-strip` 裁剪。

```
CC=riscv64-unknown-elf-gcc
STRIP=riscv64-unknown-elf-strip
CFLAGS="-mcpu=c908v -mrvv-v0p10-compatible -march=rv64gcv -mabi=lp64d -O2 -g2 \
        -mcmodel=medany -fpic -fno-plt -D_POSIX_C_SOURCE=1 -I${RKNPU3_INCLUDE}"
LDFLAGS="-mcpu=c908v -O2 -g2 -Wl,-r,-z,max-page-size=1024 -fpic \
         -nostartfiles -nostdlib -static-libgcc -e 0"
${CC} ${LDFLAGS} -o libpostprocess_yolov5_rk182x.so *.o
${STRIP} --strip-unneeded -R .hash libpostprocess_yolov5_rk182x.so
```

加载与使用

插件通过 `rknn3_register_custom_ops_plugins()` 加载，必须在 `rknn3_model_init()` 之后、`rknn3_run()` 之前。第三个参数 `size` 为保留字段，当前传 `0` 即可。加载后框架自动重建算子分派表，`rknn3_run()` 时自动调用插件 compute。

API 接口：

```
int rknn3_register_custom_ops_plugins(rknn3_context ctx,
                                      const char* plugin_path,
                                      int64_t size);
```

使用流程（以 YOLOv5 后处理插件为例）：

```
/* 1. 初始化并加载模型 */
rknn3_init(&ctx, NULL);
rknn3_load_model_from_path(ctx, "yolov5.rknn", "yolov5.weight");
rknn3_model_init(ctx, &config);

/* 2. 加载后处理插件（必须在 model_init 之后） */
const char* plugin_path = "./libpostprocess_yolov5_rk182x.so";
ret = rknn3_register_custom_ops_plugins(ctx, plugin_path, 0);
if (ret != RKNN3_SUCCESS) {
    printf("Plugin load failed! ret=%d\n", ret);
    return -1;
}

/* 3. 查询插件输出信息 */
rknn3_input_output_num io_num;
rknn3_query(ctx, RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM, &io_num, sizeof(io_num));
rknn3_tensor_attr output_attrs[io_num.n_output];
for (int i = 0; i < io_num.n_output; i++) {
    output_attrs[i].index = i;
    rknn3_query(ctx, RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR,
                &output_attrs[i], sizeof(rknn3_tensor_attr));
}

/* 4. 执行推理（框架自动调用插件 compute） */
rknn3_run(ctx, inputs, n_inputs, outputs, n_outputs);

/* 5. 读取插件输出结果（已包含后处理后的检测框） */
float* result = (float*)outputs[0].mem->virt_addr;
```

> **说明**：同 context 可多次加载，后加载覆盖同 `op_type` 的前者（先 dlclose 旧插件调 deinit）。后处理插件加载后可用 `RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM` / `RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR` 查询输出数量与属性，据此分配输出内存。该编译与加载方式适用于所有自定义算子，自定义 CPU 算子同样遵循。

4.3.1.1.3 示例

YOLOv5 后处理示例位于 `rknn3-model-zoo/examples/yolov5/cpp/libpostprocess_rk182x/`，含 Anchor-based（yolov5）与 Anchor-free（yolov8）两套实现。典型配置 `MAX_OBJ_NUM=256`、`OBJ_CLASS_NUM=80`、`NMS_THRESH=0.45`、`BOX_THRESH=0.25`，输出格式 `[N, 256, 6]`（score, class_id, x1, y1, x2, y2）。

4.3.1.2 自定义 CPU 算子

自定义 CPU 算子（`RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP`）可位于模型计算图任意位置，替代/补充 RKNN3 不支持的算子。工具链转换时无法映射到 NPU 的节点保留为 CustomOperator，由运行时在协处理器 CPU 核心执行。与后处理算子的主要差异：

对比项	后处理算子	自定义 CPU 算子
位置	模型末尾	计算图任意位置
输入输出	由插件 `get_output_num` / `get_attrs` 自定义	由模型图决定，插件不实现这两个回调
算子属性	无	来自原始框架（ONNX），通过 `get_param` 读取

4.3.1.2.1 插件实现步骤

自定义 CPU 算子的实现步骤与后处理算子类似（init / deinit/compute/注册入口），但有两点关键差异：

- **不实现 `get_output_num` / `get_attrs`**：输入输出由模型图决定，无需插件自定义。
- **通过 `get_param` 读取算子属性**：算子属性来自原始框架（ONNX），`rknn3_custom_op_context` 含 `rknn_ctx`、`priv_data`（勿改）、`user_data`（用户管理）、`get_param`。`get_param` 支持 6 种 dtype：`i`(int32)、`f`(float)、`s`(char[])、`is`(int32[])、`fs`(float[])、`ss`(char[] NUL 分隔)。

`compute` 回调典型三步：读属性 -> 取输入做布局/类型转换（NC1HWC2+FP16 解包为 NCHW+FP32）-> 计算回写（打包回 NC1HWC2）。

```
/* op_type 必须与模型中算子类型名完全一致（区分大小写） */
static rknn3_custom_op my_op = {
    .op_type     = "RknnCustomOpExam",
    .plugin_type = RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP,
    .target      = RKNN3_OP_TARGET_TYPE_CPU,
    .init        = my_op_init,
    .compute     = my_op_compute,
    .deinit      = my_op_deinit,
};
static rknn3_custom_op* registered_ops[] = { &my_op, NULL };
```

4.3.1.2.2 插件编译和加载使用

自定义 CPU 算子的编译要求、加载 API 与使用流程与后处理算子完全相同，详见 4.3.1.1.2。区别仅在于加载后无需查询 `RKNN3_QUERY_POSTPROCESS_*`（自定义 CPU 算子的输入输出由模型图决定，通过常规 `RKNN3_QUERY_IN_OUT_NUM` / `RKNN3_QUERY_OUTPUT_ATTR` 查询即可）。

4.3.1.2.3 示例

完整示例见 `rknn3-runtime/examples/rknn3_custom_op_demo/`（RknnCustomOpExam: `y=clamp(x*scale+shift, min, max)`），演示了 `get_param` 读取属性、`compute` 执行计算的完整流程。FP16 与 FP32 转换用 `float16.h` 的 `fp16_to_fp32` / `fp32_to_fp16`。

4.3.2 模型加密部署

RKNN SDK 支持离线加密 RKNN 模型，并在运行时进行解密。

> 模型加密只是模型保护的一部分，无法完全杜绝被破解的风险，建议用户尽量避免模型被第三方直接接触，并在系统层面做好整体防护。

适用场景

保护模型知识产权，防止 `.rknn` / `.weight` 被非法复制分发。采用混合加密架构：

- **AES-256-CTR**（对称）：加密模型本体，高性能支持随机访问解密；
- **RSA-OAEP**（非对称）：保护 AES 密钥，安全分发。

密钥体系：用户密钥（AES-256，用户生成，加解密模型文件）+ RK 密钥对（RSA，RK 预置，公钥加密用户密钥，芯片内置私钥解密）。

使用方式

- **加密**：用 `rknn3_model_encrypt.py` 加密模型，输出加密模型与密钥信封：

```
python3 rknn3_model_encrypt.py \
    --model model.rknn --weight model.weight \
    --rk-pubkey rk_public_key.pem --output-dir ../encrypted
# 输出: model.rknn.enc, model.weight.enc, user_key_iv.enc（RSA 加密的密钥信封）
```

- **加载**：在 `rknn3_init()` 之后、`rknn3_load_model_from_path()` 之前设置解密密钥，运行时自动解密，后续流程与非加密模型相同：

```
rknn3_init(&ctx, NULL);
rknn3_set_decrypt_key_from_path(ctx, "user_key_iv.enc");   /* 必须在 load_model 之前
*/
rknn3_load_model_from_path(ctx, "model.rknn.enc", "model.weight.enc");
rknn3_config config = {0};
config.run_core_mask = 0xff;  /* 按实际核心数配置 */
rknn3_model_init(ctx, &config);
/* 后续推理与非加密模型相同 */
```

> **说明**：安全特性包括双层加密、密钥仅芯片内置私钥可解、权重密文传输（PCIe / USB / Ethernet 链路不暴露明文）、解密后密钥内存中短暂存在使用后清除。完整示例见 `rknn3-runtime/examples/rknn3_encrypt_demo/`。

5. 问题诊断与优化

本章介绍模型部署过程中常见问题的诊断方法与优化手段，涵盖精度、性能、内存三类核心问题，以及常见错误处理。问题排查的总思路是：先确认环境与版本一致，再通过模拟器验证模型正确性，最后在板端定位 Runtime 或部署问题。

5.1 精度问题诊断

适用场景

模型转换或量化后推理结果与原始模型偏差过大，需要定位是配置错误、量化误差、前后处理错误，还是 Runtime 实现问题。精度排查按量化流程分为内部量化精度诊断与 RKQuantizer 量化精度诊断两类，两者的排查阶段略有不同。

5.1.1 内部量化精度诊断

内部量化流程的精度排查遵循"模拟器 FP16 精度 -> 模拟器量化精度 -> 连板精度 -> Runtime 精度"的递进流程，前一级正确是后一级正确的前提。排查步骤如下图：

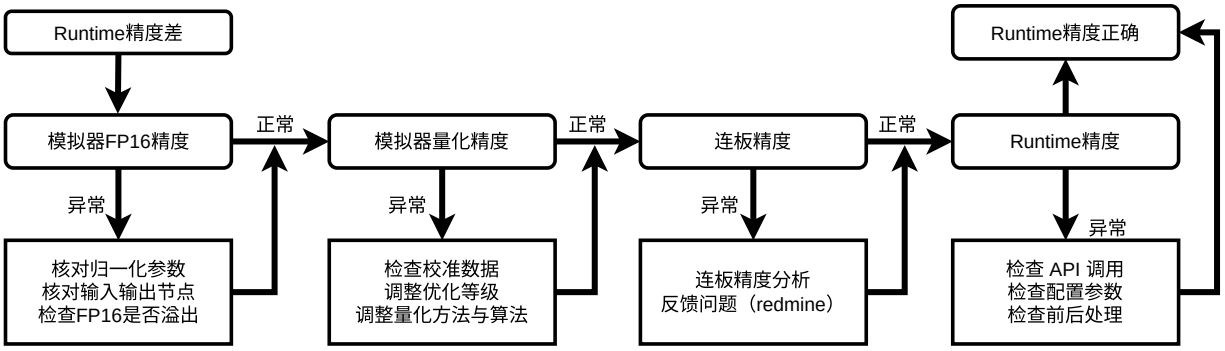


图5-1 内部量化精度排查步骤

判断精度以余弦距离为基本依据，最终需在数据集上验证。下面依次介绍各阶段的方法。

阶段一：模拟器 FP16 精度排查

不开量化（默认 FP16）在模拟器推理，确认基础正确性。重点排查：

- **核对归一化参数**：`mean_values` / `std_values` 须与训练时一致，注意 cv2 读取的 BGR 与 NHWC 默认值不一致问题。N 通道输入为 `[[c1, c2, ..., cn]]`；多输入时为 `[[...], [...]]`，每个子列表对应一个输入：

```
rknn.config(mean_values=[[128, 128, 128, 128]],   # 4 通道单输入
            std_values=[[1, 1, 1, 1]],
            target_platform='rk1820', core_num=1)
```

- **核对输入输出节点**：`load_onnx()` 的输入输出节点名、`input_size_list`（输入尺寸）、`data_format` 须与模型定义一致。
- **检查 FP16 是否溢出**：中间 Tensor 超 -65504~65504，通过精度分析查看是否出现 `inf`，可通过加 BN 层修改模型结构解决。若 FP16 推理结果即与原始模型差异较大且非配置/溢出问题，可能是 Runtime 精度差，直接进入阶段四 Runtime 精度排查。

阶段二：模拟器量化精度排查

FP16 正确后开启量化排查。重点排查：

- **检查校准数据**：校正集须覆盖实际应用场景的数据分布，一般 20~200 组。校正数据可用图片（jpg/png，OpenCV 读取）或 npy（numpy 读取），非 RGB/BGR 图片输入建议用 npy。`dataset.txt` 每行一组输入，多输入时空格分隔：

```
sampleA_in0.npy sampleA_in1.npy sampleA_in2.npy
sampleB_in0.npy sampleB_in1.npy sampleB_in2.npy
```

3 通道图像输入且量化数据为图片格式时，须确认模型输入是 RGB 还是 BGR，设置 `quant_img_RGB2BGR`。该参数仅控制量化过程中读取校正集图像时是否转换通道，不影响推理：模型用 RGB 训练则设 `False` 或不设，推理输入 RGB；模型用 BGR 训练则设 `True`，推理同样输入 BGR。若量化数据用 npy 格式，建议不设此参数避免混乱。

- **调整优化等级**：`optimization_level` 默认 3（速度优先），调小（如 0）可禁用影响精度的优化。
- **调整量化方法与算法**：若精度不足，按以下顺序优化：
- **切换量化方法**：w8a8 由 `layer` 改 `channel`；w4a16 由 `channel` 改 `group32`。
- **切换量化算法**：`normal` -> `kl_divergence`（feature 分布不均匀时改善好）-> `mmse`（迭代裁剪权重，耗时内存大增）。
- **混合量化**：对敏感层用高精度（如 w8a16），其余层用低精度（如 w4a16），通过 `op_quantized_dtype` 指定各层量化类型。

量化算法与校正数据：

算法	特点	推荐数据量
`normal`（默认）	常规校正，速度快	20~200 组
`kl_divergence`	feature 分布不均匀时改善较好，耗时略高于 normal	20~200 组
`mmse`	迭代中间层裁剪权重，可能提升精度但耗时与内存大增	20~50 组

建议先用 `normal`，精度不佳再尝试 `kl_divergence` 或 `mmse`。量化算法仅影响转换阶段，不影响运行性能。

若 `single` 列某些层掉精度严重且无法改善，可能是权重数值分布不适合量化，需修改模型结构或重新训练。

阶段三：连板精度排查

模拟器正常后，连板推理（指定 `target` + `core_mask`）对比模拟器结果。重点排查：

- **连板精度分析**：差异大时，进行连板精度分析（`rknn.inference(accuracy_analysis=True)`，需 `profile_mode=True`），对比模拟器与板端的逐层输出差异。重点检查前后处理与模拟器是否一致（归一化、通道顺序、数据类型）、C API 输入输出配置（归一化是否重复、通道顺序、`rknn3_create_mem()` 大小与数据格式）。
- **反馈问题（redmine）**：差异持续无法消除时，将精度分析结果反馈 NPU 团队（redmine）。

阶段四：Runtime 精度排查

连板正常但板端部署推理异常时，做板端精度分析，查 `runtime_error` 的 `single_sim` 列（cos 偏低/euc 偏高显黄红色）。重点排查：

- **检查 API 调用**：API 调用顺序、参数传递是否正确。
- **检查配置参数**：`core_mask`、内存分配、数据类型等配置是否与模型匹配。
- **检查前后处理**：输入前处理（归一化、通道顺序、数据类型）与输出后处理（反量化、业务逻辑）是否与模拟器一致。

若 `single_sim` 异常导致 `entire` 与模拟器差异扩大，可能是 Runtime 该层实现问题，反馈 NPU 团队。

5.1.2 RKQuantizer 量化精度诊断

RKQuantizer 量化流程输出的已是伪量化模型（权重已完成 fake-quant），无需再排查 FP16 精度。精度排查遵循"量化精度 -> 连板精度 -> Runtime 精度"的递进流程。排查步骤如下图：

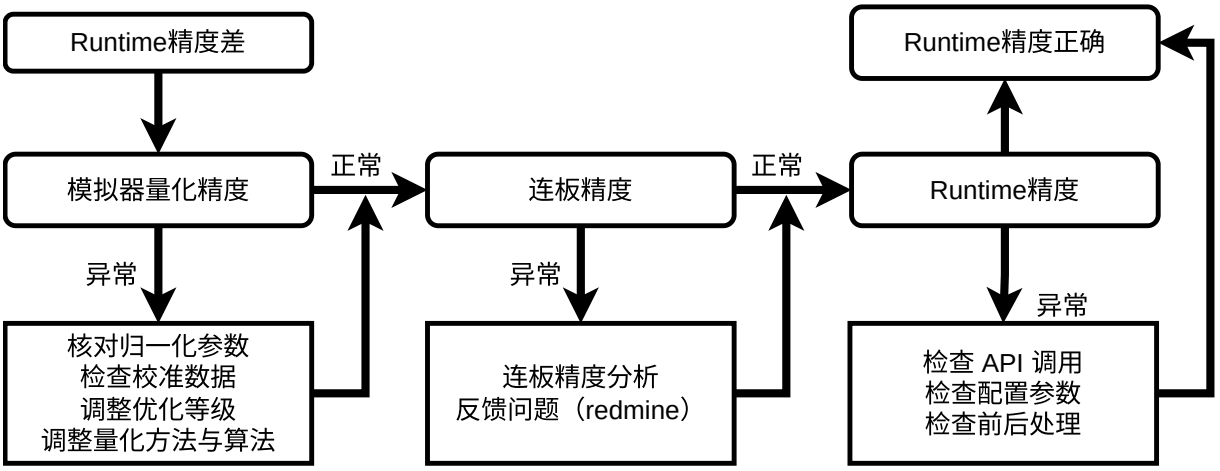


图5-2 RKQuantizer 量化精度排查步骤

阶段一：量化精度排查

RKQuantizer 量化后的模型直接进行量化精度排查。重点排查：

- **核对归一化参数**：`mean_values` / `std_values` 须与训练时一致，注意 cv2 读取的 BGR 与 NHWC 默认值不一致问题。
- **检查校准数据**：RKQuantizer 的 GRQ 算法需校准数据统计各层激活分布并对权重微调，校准数据以 JSON 文件提供，须覆盖实际应用场景的数据分布，一般 20~200 个样本。校准数据的具体格式与准备方法见 2.3.4 RKQuantizer 量化流程。
- **调整优化等级**：RKNN 转换阶段 `rknn.config()` 的 `optimization_level` 默认 3（速度优先），调小（如 0）可禁用影响精度的优化。注意 RKQuantizer 量化流程转换时 `quantized_algorithm` 必须设为 `normal`。
- **调整量化方法与算法**：RKQuantizer 支持 GRQ 和 Normal 两种算法，GRQ 为首选。若精度不足，可调整 `quantized_dtype`（w4a16 / w6a16 / w8a16）或使用混合精度（`layer_quant_config` / `auto_hybrid_rate`），详见 2.3.4。

LLM / MLLM/VIT 量化建议：采用 `quantized_dtype='w4a16'`、`quantized_method='group32'`，需进一步提升 w4a16 精度可先用 RKQuantizer GRQ 量化。

若 `single` 列某些层掉精度严重且无法改善，可能是权重数值分布不适合量化，需修改模型结构或重新训练。

阶段二：连板精度排查

与内部量化的连板精度排查一致，见 5.1.1 阶段三。

阶段三：Runtime 精度排查

与内部量化的 Runtime 精度排查一致，见 5.1.1 阶段四。

5.2 性能分析与优化

适用场景

模型推理速度不达预期，需要定位是 NPU 算力瓶颈、带宽瓶颈、CPU 算子拖累，还是前后处理/传输耗时，并针对性优化。

性能分析工具

工具/接口	作用	使用方式
`rknn.eval_perf(log_level=)`	Toolkit 性能评估，输出 Op Time Ranking	需先 `config(profile_mode=True)`
`rknn3_profile_ops(ctx, ..., log_level)`	板端逐层算子性能	log_level 0 汇总/1 逐算子/2 含 Cycle 带宽
`rknn3_session_query_state()`	LLM Session 性能统计	查询 `RKLLMRunState`
Lite `session_run` 返回值	LLM Python 性能统计	返回 `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`

LLM 性能关键指标：TTFT（首 token 延迟）、TPOT（每 token 耗时）、Prefill TPS、Decode TPS。通过 `llm_start_time` / `llm_end_time` + 首 token 时间计算各阶段耗时与吞吐。

性能优化手段

- **多核配置**：计算量大的模型用多核（`core_num>1`），小网络单核（多核切换需 CPU 介入反而变慢）。`core_mask` 必须与转换时 `core_num` 匹配。
- **量化加速**：量化同时提升带宽与算力利用率，是性能优化的基础手段。
- **CPU 算子 NPU 化**：大部分性能问题源于 CPU 算子（尺寸超限/未支持/硬件限制）。通过 `eval_perf` / `profile_ops` 识别 CPU 算子，改用 NPU 支持的等价算子或调整尺寸。
- **图级优化**：硬件 Fuse（Conv+Relu/Clip/Add）、算法等效子图单 OP 化、同类算子合并。
- **算子级优化**：DDR 对齐（Channel 按 Int8=32/FP16=16 对齐，HW 4 对齐）、规避低效算子（Transpose/Reshape/Resize/Sigmoid/Tanh）、卷积尺寸优化（Channel<256 利用率高，输出 HW>=32）。
- **LLM 专项优化**：`keep_one_logit` 减少 Prefill 阶段 lm_head 计算、`kvcache_dtype` 选 Int4/Int8 减少 KVCache 访问、多核配置加速 prefill。

性能分析基准

性能数据有意义的前提是**定频**（未定频波动大）。板端用 `opp_dump` 查 CPU/NPU 频率，`srv_set_rate` / `npu_set_rate` 设定频率。用 `rknn-smi info -w` 查 NPU 负载，负载低说明 NPU 在等待，需优化前后处理或多线程。

> **说明**：NPU 算子瓶颈看 DDR Cycles vs NPU Cycles，DDR > NPU 为带宽瓶颈，反之为算力瓶颈。部署耗时分三方面：用户应用（前后处理，可用 Matmul API 加速）、输入输出传输（协处理器模式，选合适 tensor 数据类型/排布）、推理（多次取平均）。

5.3 内存分析与优化

适用场景

板端内存有限，多模型并存或大模型部署时内存不足，需要分析内存占用构成并优化。LLM 模型因 KVCache 占用大，内存优化尤为关键。

内存分析工具

工具/接口	作用
`rknn.eval_memory()`	Toolkit 内存评估，输出 Device Memory 与 Per-Core Memory Allocation 表
`rknn3_query(ctx, RKNN3_QUERY_CORE_MEM_SIZE, ...)`	板端查询各 NPU 核心 weight/internal 占用
`rknn3_profile_mem(ctx)`	板端打印各核 Command/Weight/Internal/KVCache 及设备内存统计

`eval_memory` 输出的 Per-Core Memory Allocation 含 Command（寄存器配置）、Weight（权重）、Internal（中间 Tensor）、KVCache（LLM 专有）、Total 各项。

内存构成

运行时内存 = 权重 + internal tensor + 寄存器配置 + 输入输出 tensor，LLM 额外有 KVCache。内存在 `rknn3_model_init()` / `rknn3_session_init()` 时创建。

内存优化手段

- **Internal 内存复用**：多模态串行推理（vision -> LLM）时，同核心的 internal 内存可复用（见 4.1.4），总块数 = vision 核数 + LLM 核数 - 重叠核数。
- **多 Session 共享**：同一 context 下多 Session 共享权重/internal/LoRA，仅 KVCache 独立，比多 context 省内存。
- **KVCache 优化**：`kvcache_buffer_len`（32 对齐，按需设置）、`kvcache_dtype`（内存占用 FP16 > Int8_to_F16 > Int4_to_F16）、`position_embeddings_host_storage`（存 Host 端省设备内存）。
- **Tie Word Embedding**：复用 lm_head 权重，省 `embed.bin`，降低 Host 端内存（见 4.2.9）。
- **llm_head 设备指定**：`llm_head.device` 可指定到主控 SoC（rk3588/rk3576），减少协处理器内存。
- **core_mask 控制**：`core_num` / `core_mask` 决定权重/internal 在多少核分配，直接影响 Total 内存，按需配置而非全核。

> **说明**：动态 Shape 模型不支持权重共享与上下文复用（`rknn3_dup_context()`），内存复用受限。板端内存可用 `rknn.eval_memory()` 查看 Total 评估是否够用。

5.4 常见问题与错误处理

版本与兼容性

- **版本不一致**：rknn3-toolkit、通信服务（rknn3_transfer_proxy / rknn3-server）、Runtime 库必须同版本，否则模型转换正常但板端跑不起来，可能出现 `protocol fingerprint mismatch`，或报 `Invalid RKNN format`。版本查询命令：

```
pip list | grep rknn3-toolkit                                  # Toolkit 版本
strings xxx.rknn | grep rknn3-toolkit                          # 模型转换时版本
strings librknn3_api.so | grep -i "librknn3_api version"       # Runtime 版本
strings rknn3_transfer_proxy | grep "Transfer version"         # 协处理器模式通信服务版本
rknn3-server -v                                                # 非协处理器模式
```

- **跨平台不兼容**：RK1820 / RK1828 / RK3572 模型互不兼容，需按目标平台重新转换。

工具安装

- **依赖冲突导致安装失败**：所有依赖已安装但版本不匹配时，pip 环境检查会阻断安装，可加 `--no-deps` 跳过：`pip install rknn3-toolkit-*.whl --no-deps`，但需自行确保运行依赖完整。
- **无 ARM Linux 版本**：RKNN3-Toolkit 仅提供 x86_64 版本，无 ARM Linux 版本。

模型加载失败

报错	原因	解决
`Error parsing message`	ONNX 模型损坏	重新下载并校验 MD5
`Not support input data type 'float16'`	PyTorch float16 权重不支持	改 float32
自定义输出节点报错	outputs 节点名无效	核对节点名
4 维以上 Op 报错	暂不支持	手工去掉或简化
动态图报错（如 NonZero）	导致动态图	替换/移除该 OP

建议转换前用 `onnx-simplifier` 预处理模型。

模型转换问题

- **onnxsim 简化失败**：onnx-simplifier 对 shape 推导失败或动态图的模型可能简化失败，此时可直接用原始 ONNX 转换，onnxsim 非必须。
- **mean_values 通道数报错**（如 `expect N`）：模型输入非 3 通道图像（如 1x32 的非图像数据）时，`mean_values` 通道数须与实际输入通道一致；若模型不需要 mean/std，可不设置。
- **RKNN 模型比原始模型大**：正常现象。RKNN 模型除权重和图结构外，还含 NPU 寄存器配置信息，且可能做 OP 拆解以提升运行效率。

推理结果错误

- **模拟器与板端差异**：硬件驱动差异导致少量不同属正常，差异大则反馈 NPU 团队。
- **连板与板端差异大**：查预处理、数据类型、layout（NCHW / NHWC）一致性。
- **耗时 >20s 且结果错误**：通常 NPU Hang，更新 Toolkit / Runtime。
- **3 维输入连板错误**：NPU 3 维支持不完善，改 4 维或更新版本。
- **结果每次不一致**：板端 NPU 内核驱动 bug，更新驱动+Toolkit。
- **`do_quantization=False` 结果全 nan**：中间层超 FP16 范围，加 BN 层。
- **QAT 模型不一致**：PyTorch 推理需设 `engine='qnnpack'`。

模拟器与连板推理

- **三种推理方式区分**：模拟器推理（Linux x86_64 上 RKNN3-Toolkit 模拟器，输出未必与板端一致）、连板推理（PC 调 Toolkit Python API，模型在板端 NPU 执行）、板端推理（板端调 C API 执行）。模拟器结果仅供初步验证，精度/性能以连板或板端为准。
- **连板比板端慢**：连板推理有 PC↔板端的数据传输开销，NPU 实际推理性能以板端 C API 为准。
- **连板日志查看**：模型初始化与推理在板端完成，日志产生在板端，通过串口进系统执行 `export RKNN_LOG_LEVEL=5` 后再连板调试，板端错误信息显示在串口窗口。
- **data_format 默认值**：`rknn.inference()` 连板时默认 `NHWC`，ONNX 4 维输入默认 NCHW，推理时内部做 NCHW→NHWC 转换；若显式设 `data_format='NCHW'` 则不做转换。

模型评估问题

- **`Invalid RKNN format`**：板端报此错通常是 `rknn.config()` 的 `target_platform` 未设或设错，或 Toolkit 与 Runtime 版本不兼容。检查 target_platform 并确保版本一致。
- **inference 耗时与 eval_perf 不一致**：`rknn.inference()` 通过 PC + adb 连板，有固定数据传输开销；真实帧率建议直接在板端用 C API 测试。
- **Resize OP 导致精度下降**：NPU 暂不支持硬件级 Resize，转换工具将 Resize 转为 ConvTranspose 有精度损失。可尽量避免 Resize（改用 ConvTranspose 再训练），或 `rknn.config()` 设 `optimization_level=2` 让 Resize 走 CPU（精度不掉但性能下降）。
- **perf_debug 影响性能数据**：`rknn.init_runtime()` 开启 `perf_debug` 时会添加调试代码、可能禁用并行机制，耗时比关闭时多。perf_debug 用于定位耗时占比大的层，非用于绝对性能评估。

core_mask 不匹配

报错 `core_mask ff is not match with npu core number 1!`：`rknn3_model_init()` 的 `run_core_mask` 与转换时 `core_num` 不一致。按正确核心数配置：RK1820 / RK1828 为 0x1~0xff，RK3572 仅 0x1。

内存不足

- **量化时被 kill**：Toolkit 量化内存占用大，增大电脑内存或虚拟内存/交换分区。
- **板端内存不足**：用 `rknn.eval_memory()` 查 Total，按 5.3 手段优化。

返回值检查

所有 API 调用需检查返回值，`RKNN3_SUCCESS`(=0) 表示成功。统一模式：

```
int ret = rknn3_init(&ctx, NULL);
if (ret != RKNN3_SUCCESS) {
    printf("rknn3_init failed! ret=%d\n", ret);
    return -1;
}
```

`rknn3_session_init()` 失败返回 NULL。资源释放顺序：先销毁 Session（`rknn3_session_destroy()`），再销毁上下文（`rknn3_destroy()`）；通用模型场景还需在销毁上下文前释放自行分配的 tensor 内存（`rknn3_destroy_mem()`）。

C API 使用

- **`rknn3_create_mem()` 分配大小**：输入内存一般用 `rknn3_tensor_attr` 的 `aligned_size` 和 `core_id` 进行分配，确保大小与对齐满足 NPU 要求。

- **输入数据填充顺序**：四维输入按 `NHWC`（`[batch, height, width, channel]`）填充；非四维输入按模型原始形状填充，`data_format` 设为 `undefined`。

日志与调试

- 连板推理日志在板端，串口进系统后 `export RKNN_LOG_LEVEL=5` 获取详细错误。
- 板端性能 Profile 用 `RKNN_LOG_LEVEL=4`。
- Docker 重启后连板卡在初始化：rknn3_transfer_proxy 异常退出致 RKNN Server 连接状态未重置，重启开发板。

LLM 与 Session

- **首 token 很慢（TTFT 高）**：prefill 阶段需处理完整输入序列，计算量大。可减小 `max_context_len`、开启 KVCache Checkpoint（仅 Linear/Sliding Attention 模型）加速长前缀恢复。
- **多轮对话越来越慢**：上下文增长导致 KVCache 增大。用 `RKNN3_KVCACHE_POLICY_RECURRENT` 策略自动复写，设 `n_keep` / `n_keep_aligned` 保留 system prompt 不被复写。
- **`rknn3_session_init()` 返回 NULL**：检查 `vocab_info` 是否正确填充（`vocab_size` / `special_eos_id` 等）、`max_context_len` 是否过大导致内存不足、模型是否正确加载。
- **`result_callback` 不触发**：确认已通过 `rknn3_session_set_callback()` 注册回调，`RKLLMCallback` 结构体已正确初始化（`memset` 清零后填充）。

内存与性能

- **LLM 模型加载内存不足**：`kvcache_buffer_len` 过大会增加 KVCache 内存，按实际需求设置（32 对齐）；`kvcache_dtype` 选 `Int4_to_F16` 或 `Int8_to_F16` 比 `Float16` 占用更低；`position_embeddings_host_storage=True` 将位置编码存 Host 端省设备内存。
- **多模型并存内存不足**：多模态串行推理可用 Internal 内存复用（见 4.1.4）；`llm_head.device` 可将 lm_head 计算卸载到主控 SoC 减少协处理器内存。

部署模式

- **协处理器模式连不上板端**：确认 `rknn3_transfer_proxy` 已在板端启动、PCIe / USB / Ethernet 连接正常、`device_id` 正确（通过 `rknn3_find_devices()` 或 `get_devices_id()` 查询）。
- **RK3572 与 RK182X 模型能否通用**：不能。不同平台的 RKNN 模型互不兼容，需按目标平台重新转换（`target_platform` 参数）。

模型加密

- **加密模型加载失败**：确认 `rknn3_set_decrypt_key_from_path()` 已在 `rknn3_load_model_from_path()` 之前调用；密钥信封文件（`user_key_iv.enc`）与加密模型配套使用；密钥信封由对应 `rk_public_key.pem` 加密生成，不可混用。

多模态

- **MLLM 图像输入后输出乱码**：检查占位标记（`image_start` / `image_end` / `image_content`）是否与模型 tokenizer 特殊 token 一致；`image_embed` 的维度（`n_image_tokens × embedding_dim`）和数量（`n_image`）是否与 prompt 中 `<image>` 标签数量匹配。

> **说明**：遇 `Meet unsupport xxx operator` 表示 Runtime 不支持该算子，更新工具链到最新版本。rkllm3-server 的 HTTP 错误码遵循 OpenAI 规范（401 authentication_error、400 invalid_request_error、501 not_supported_error），完整错误码见《RKLLM3_Server使用指南与API_参考》。